Verification and Testing Methodologies Real Time Embedded Systems

Agenda

- Introduction: Test and Verification
- Verification
  - Introduction and Basic Operations on Temporal Logic
  - Syntax and Semantics of CTL
  - CTL Model Checking Algorithm
  - RTCTL for Real Time ES
- Test
  - Functional vs. Structural Test
  - Combination and Sequential Circuit Test
  - Delay Fault Testing for Real Time ES

Verification vs. Test

- Verifies correctness of design i.e., check if the design meets the specifications.
- Simulation or formal methods.
- Performed once prior to manufacturing.
- Required for reliability of design.

- Checks correctness of manufactured hardware.
- Two-stage process:
  - 1. Test generation: CAD tools executed once during design for ATPG
  - 2. Test application: TPs tests applied to ALL hardware samples
- Test application performed on every manufactured device.
- Responsible for reliability of devices.

Design Cycle

Specification
Design
Verification
Implementation
Testing
Installation/marketing
Maintenance

Verification is used to capture the bugs at the early phase of design cycle.

Simulation

Simulation:

Exhaustive Simulation

Non exhaustive Simulation

Number of test cases are exponential to the number of state variables.

Non Exhaustive Simulation

Instead of using all possible combinations, simulation is done for some selected input combinations.

To find the appropriate subset is a complex problem.

- Test case generation

We may not cover all possible error cases.

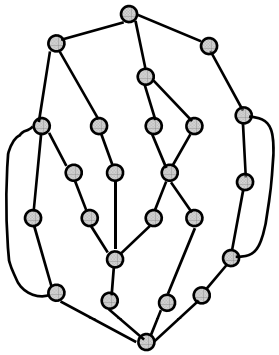
Non Exhaustive Simulation

Non Exhaustive Simulation: **Pentium Bug**

The intel pentium ™ processor for IBM compatible PC was first introduced into the market in May of 1993.

A year later an estimated two million had been sold, it was discovered that there was a flaw in the hardware of floating point division.

Basic picture of Model Checking (Formal Verification)



Finite-state model

Verification tool

OK

or

Error trace

```
Line 5: …
Line 12: …
Line 15:…
Line 21:…
Line 25:…
Line 27:…
   …
Line 41:…
Line 47:…
```

□ (Φ→◇ Ω)

Specification

Logic for Formal verification

- Propositional Logic
  - p: 'I qualified GATE exam in 2010'
  - q: 'I prepared well'
  - r: 'I got a non-scholarship seat in IIT Guwahati '
  - Dually, the formula denotes conjunction of *p* and *r*: 'I qualified GATE exam in 2010 and I got a non-scholarship seat in IIT Guwahati.' (may be his score was not high!!!)
- Predicate Logic
  - *W(X):* '*X* gets fail grade."
  - *R(X)*: "*X* is working hard",
  - In predicate logic, we also use two quantifiers, one is universal quantifier, for all ($\forall$), and the other is existential quantifier, there exists ($\exists$).
  - We can write:$\forall x\ [R(x) \rightarrow U(x)]$ for all those who work hard will get good marks.

Temporal Logic

- The truth value of a temporal logic is defined with respect to a model.
- Temporal logic formula is not *statically* true or false in a model.
- The models of temporal logic contain several states and a formula can be true in some states and false in others.

Temporal Logic

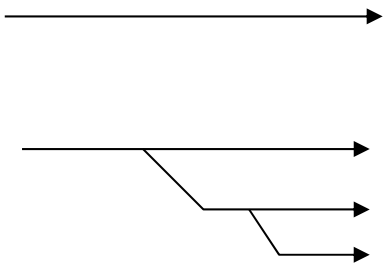
In temporal logic we can express statements like:

- "I am *always* happy",
- "I will *eventually* be happy",
- "I will be happy *until* I do something wrong“
- "I am happy."

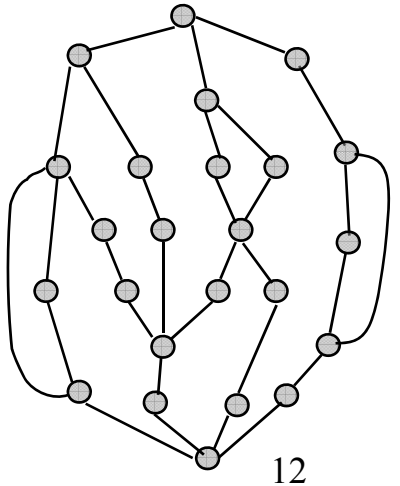
Temporal logic

Branching vs. linear time:

- *Linear time* logic thinks of time as a set of paths, where a path is a sequence of time instances.



- *Branching-time* logic represents time as a tree, rooted at the present instance and branching out into the future.



Temporal Operator

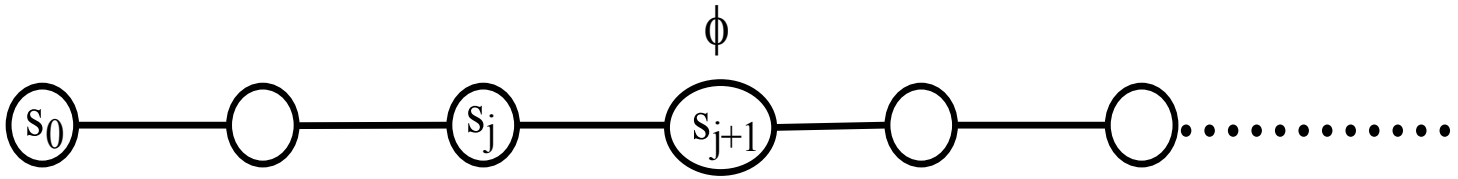
Operator	Textual Notation	Meaning
○	X φ	Φ holds at next state
◊	F φ	Φ eventually holds
□	G φ	Φ holds globally
U	φ **U** ψ	Φ holds until ψ holds

Temporal Operator

Next : X φ

$(M, S_j) \models X\,\varphi \Leftrightarrow (M, S_{j+1}) \models \varphi$

State S_j satisfies as its next state S_{j+1} satisfies φ .

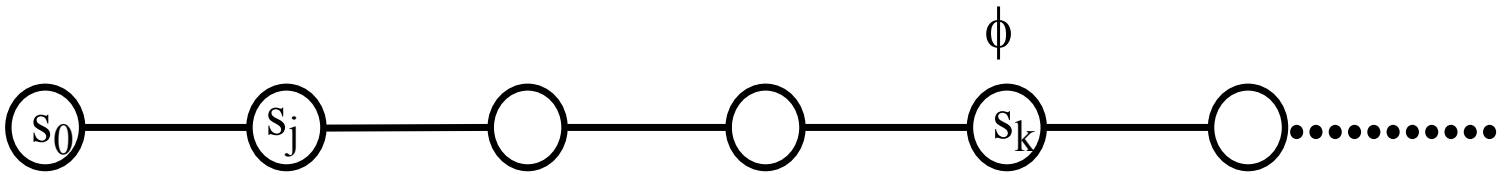


Temporal Operator

Future : F φ

$(M, S_j) \models F\,\varphi \Leftrightarrow \exists k, k \geq j, (M, S_k) \models \varphi$

state S_j satisfies as future state S_k satisfies φ .

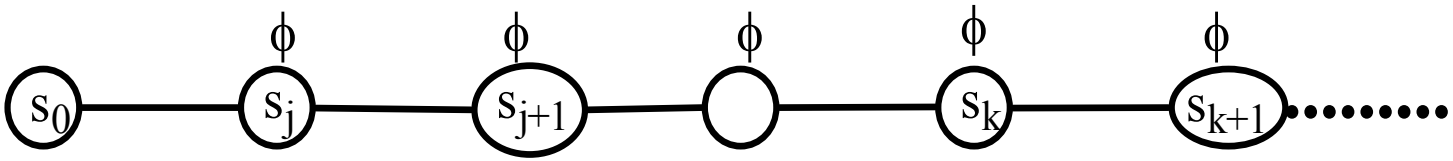


Temporal Operator

Globally : G φ

$(M, S_j) \models G\varphi \Leftrightarrow \forall k, k \geq j, (M, S_k) \models \varphi$

S_j satisfies **G** φ as all states satisfies φ.



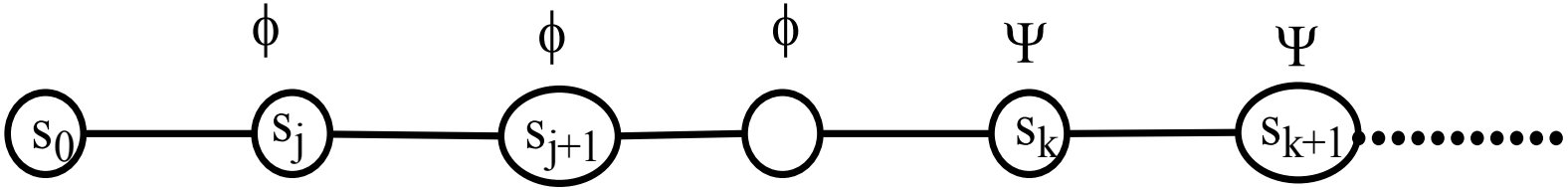
Temporal Operator

Until : $\Psi \ U \ \varphi$

$$(M, S_j) \models \Psi \cup \varphi \Leftrightarrow \exists k, (M, S_K) \models \varphi \text{ and}$$

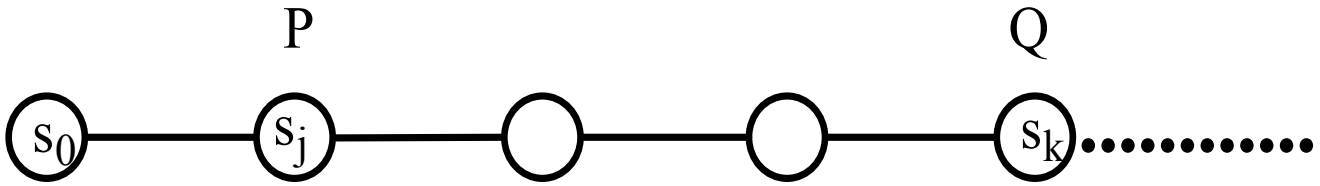
$$\forall k, j < k \ (M, S_j) \models \Psi$$

S_j satisfies $(\Psi \ U \ \varphi)$ because Ψ is true for all states S_i $j<=i<k$ and then φ is true for state S_k .

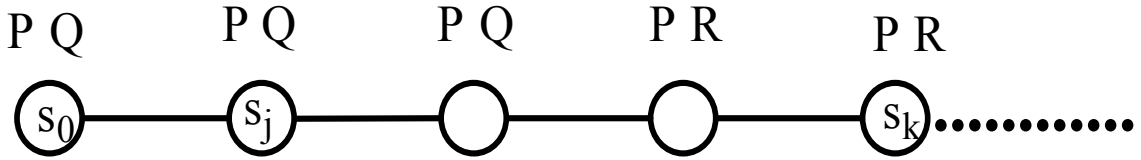


Example

($P \rightarrow FQ$): If P is true in a state then in a future state Q is true.



($P \wedge (Q\ U\ R)$): P holds in a state and also $Q\ U\ R$ (Q until R) holds in the state



Syntax of CTL

Syntax of CTL

A CTL formula comprises

1. Atomic propositions such as *{p, q, r.....}*
2. Path Quantifiers *{A,E}*
    a. *A* : all paths starting from a given state.
    b. *E* : there exists at least one path from a given state.
3. Propositional logic operators such as AND ($\wedge$), OR ($\vee$), NOT ($\neg$)
4. Temporal operators *{X,F,G,U}*
    a. NE**X**T: next states of current state.
    b. FUTURE: any one of future states from the current state.
    c. GLOBAL: all future states from the current state.
    d. UNTIL: Some CTL formula holds until another CTL formula, from the current state.

CTL Formulas

We can define CTL formulas as:

$\Phi ::= \bot \mid \top \mid P \mid (\neg\varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \rightarrow \varphi) \mid AX\varphi \mid EX\varphi \mid AF\varphi \mid EF\varphi \mid AG\varphi \mid EG\varphi \mid A[\varphi\ U\ \varphi] \mid E[\varphi\ U\ \varphi];$

where

- The symbol $\top$ means truth value 'true' and symbol $\bot$ means truth value 'false'.
- P ranges over a set of atomic propositions

CTL Formulas

AGf_1: Always globally f_1 holds.

EGf_1: There exists a path where f_1 holds globally

AFf_1: f_1 holds in all path in future.

EFf_1: There exists a path in which f_1 holds in future.

CTL Formulas

AX f_1 means: holds in state $s°$ iff f_1 holds in all successor states of $s°$

EX f_1 means: There exists a successor such that f_1 holds

A[f_1 U f_2] means: always until, in all paths such that f_1 holds until is f_2 satisfied.

E[f_1 U f_2] means: There exists a path such that f_1 holds until is f_2 satisfied.

Temporal structure

- The semantics of CTL is defined over a model M, which is defined as 3-tuple $M = (S, \rightarrow, L)$

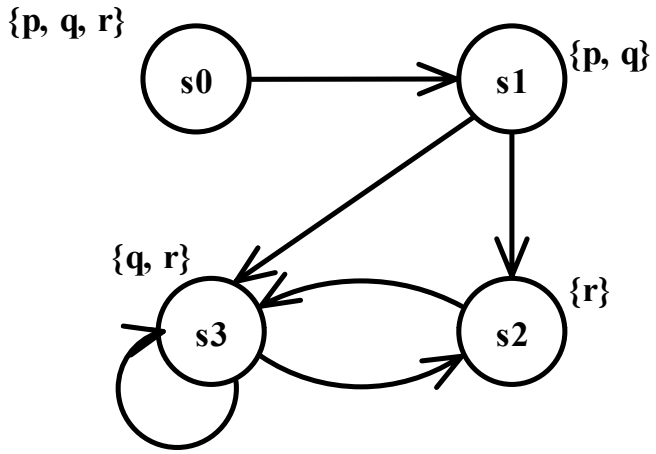
- **Definition**: A **temporal structure** $M := (S, \rightarrow, L)$ consists of
  1. A finite set of states S
  2. A transition relation $\rightarrow \subseteq S \times S$ with $\forall s \in S \; \exists s' \in S : (s, s') \in \rightarrow$
  3. A labeling function $L: S \rightarrow \wp(V)$, with being the set of propositional variables (atomic formulas)

- This structure is often called Kripke structure.

Kripke structure

- A Kripke structure is similar to a state transition diagram, with
  - All states must have at least one outgoing edge.
  - Each state is labeled with one of the element of the power set of atomic propositions.

Kripke structure



- S={s0,s1,s2,s3}
- $\rightarrow$ ={{s0, s1}, {s1,s2}, {s1, s3}, {s2,s3}, {s3,s2},{s3,s3}}.
- L: L(s0)={p,q,r}, L(s1)={p, q}, L(s2)={r}, L(s3)={q, r}.

Illustration of basic CTL temporal operators

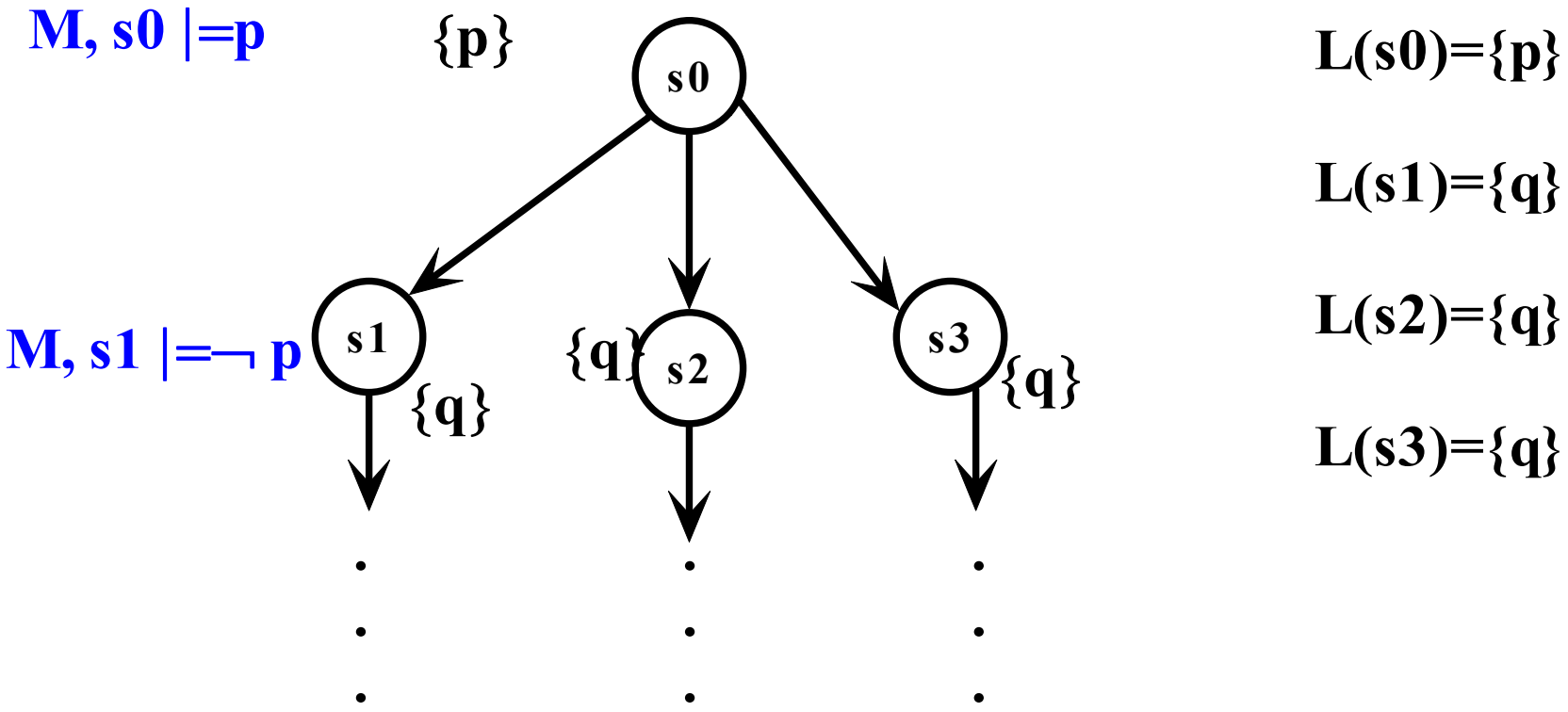
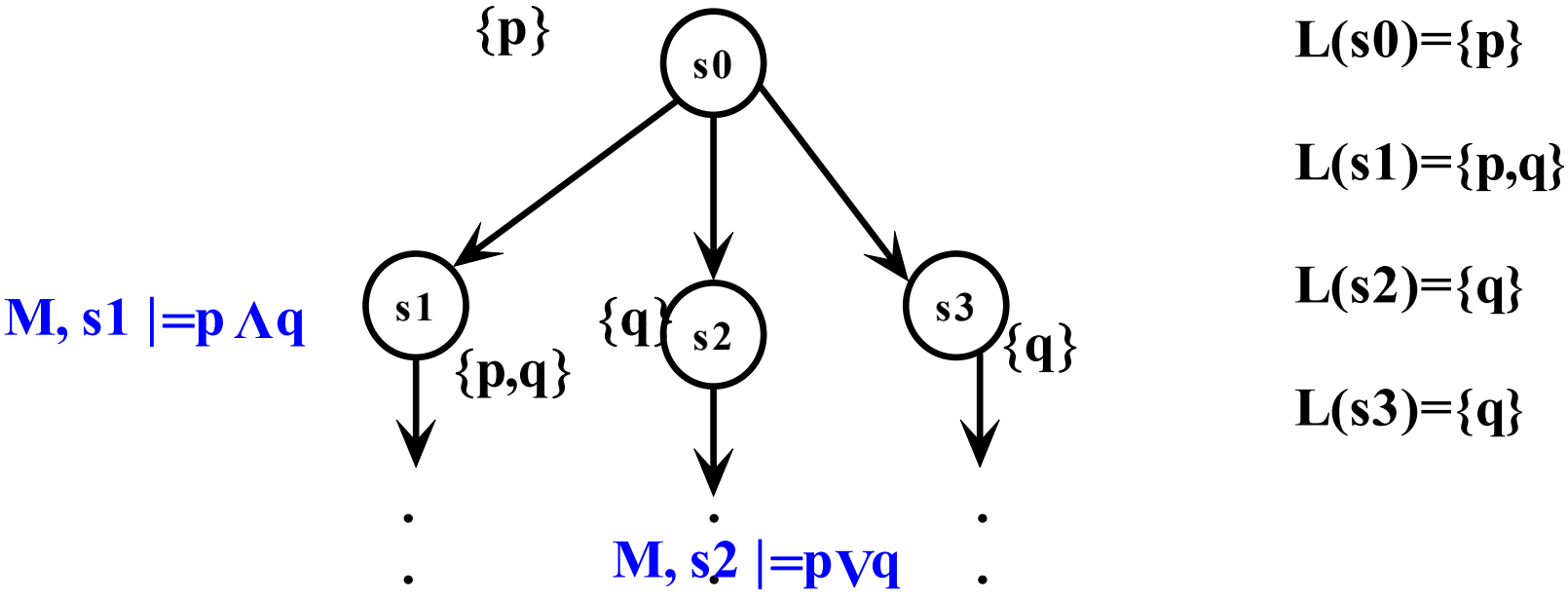


Illustration of basic CTL temporal operators

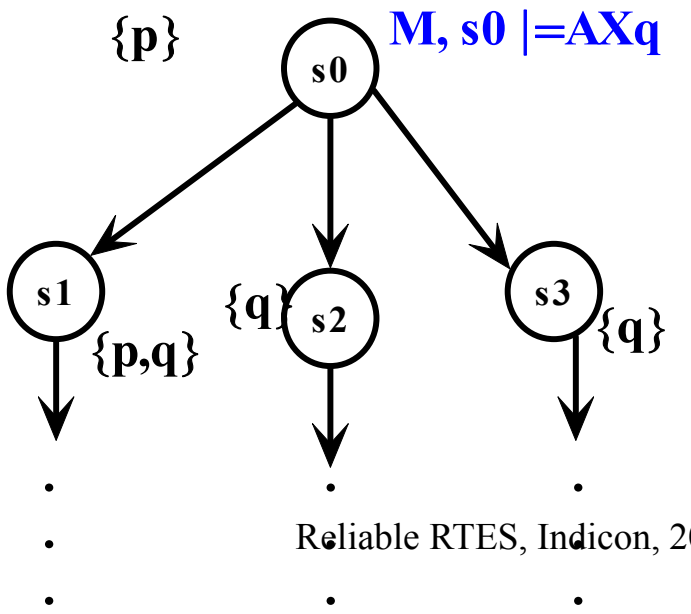


$M, s \models \varphi 1 \rightarrow \varphi 2$ iff $M, s \not\models \varphi 1$ or $M, s \models \varphi 2$;

$\varphi 1 \rightarrow \varphi 2$ is satisfied at s if in s either $\varphi 1$ is not satisfied or $\varphi 2$ is satisfied.

Illustration of basic CTL temporal operators

$M, s \models AX\varphi$ iff for all s1 such that s $\rightarrow$ s1, we have M, s1 $\models \varphi$; $AX\varphi$ is satisfied at s if in all next states of s, φ is satisfied.



L(s0)={p}

L(s1)={p,q}

L(s2)={q}

L(s3)={q}

Illustration of basic CTL temporal operators

M, s ⊨ *EXφ* iff for one state s1 such that s → s1 we have *M,* s1 ⊨ φ; *EXφ* is satisfied at *s,* if in some next state of *s,* φ is satisfied.

s0 ⊨ *EXp*.

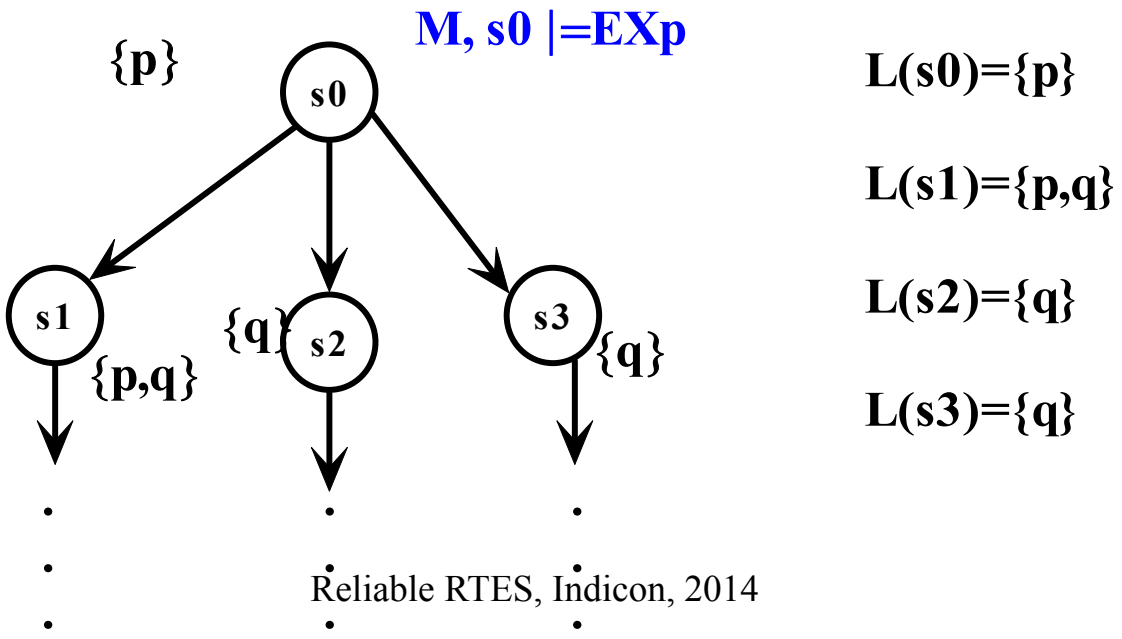


Illustration of basic CTL temporal operators

$M, s \models AG\varphi$ holds iff for all paths s1 → s2 → s3 → . . ., where s=s1, and all s_i along the path, $M, s_i \models \varphi$. $AG\varphi$ is satisfied at s if all states of all paths from s satisfies φ.

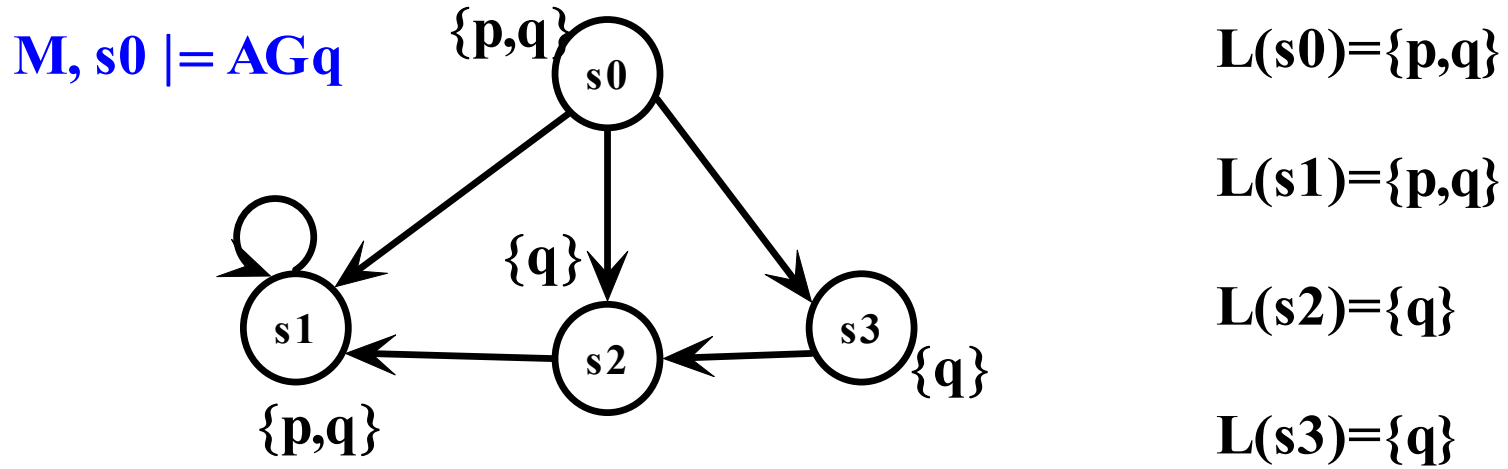
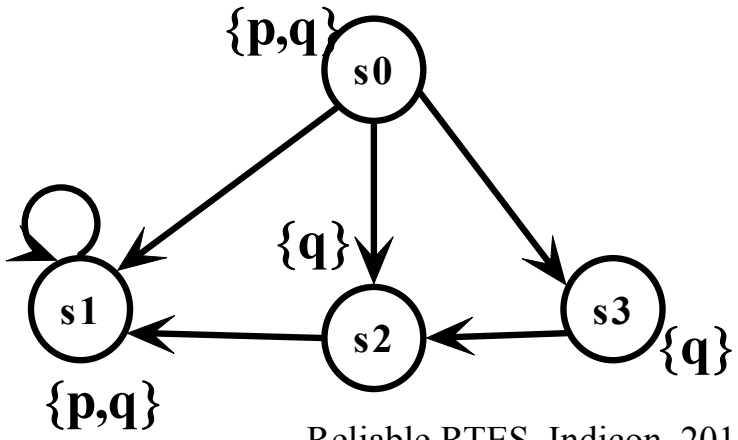


Illustration of basic CTL temporal operators

M, s ⊨ *EGφ* holds iff there is a path s1 → s2 → s3 → . . ., where *s*=s1, and all s_i along the path, *M*, s_i ⊨ *φ*. *EGφ* is satisfied at *s* if all states of at least one path from *s* satisfies *φ*.

state s0 satisfies *EGp* (the path s0 → s1 → s1 → ….).



L(s0)={p,q}

L(s1)={p,q}

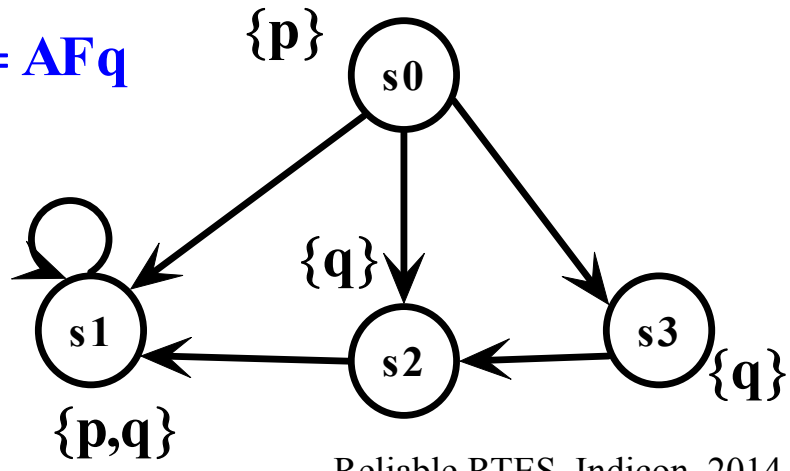
L(s2)={q}

L(s3)={q}

Illustration of basic CTL temporal operators

$M, s \models AF\varphi$ holds iff for all paths s1 → s2 → s3 → . . ., where s=s1, and for at least one s_i along the path, $M, s_i \models \varphi$. $AF\varphi$ is satisfied at s if some "future" state of all paths from s satisfies φ.

M, s0 |= AFq



L(s0)={p}

L(s1)={p,q}

L(s2)={q}

L(s3)={q}

Illustration of basic CTL temporal operators

M, s ⊨ *A[φ1 U φ2]* holds iff for all paths s1 → s2 → s3 → . . ., where *s*=s1, *φ1 U φ2* is satisfied, i.e., there is some s_i along the path, such that *M,* s_i ⊨ *φ2*, and for each $j < i$, we have *M,* s_j ⊨ *φ1*

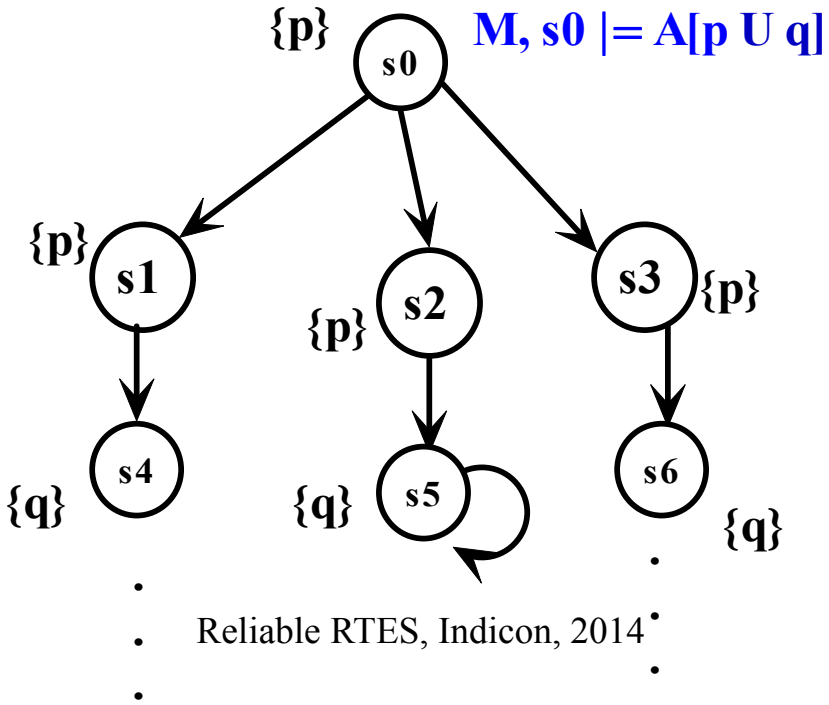
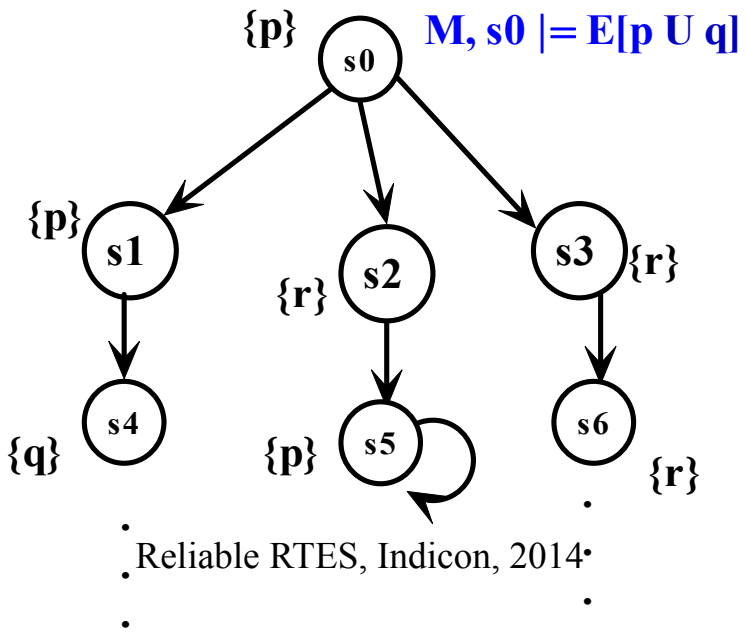


Illustration of basic CTL temporal operators

$M, s \models E[\varphi 1\ U\ \varphi 2]$ holds iff for at least one path s1 → s2 → s3 → . . ., where s=s1, $\varphi 1\ U\ \varphi 2$ is satisfied, i.e., there is some s_i along the path, such that $M, s_i \models \varphi 2$, and for each $j < i$, we have $M, s_j \models \varphi 1$



Verification Technique: Model checking (CTL)

- **Process of Model Checking:**
  - Modeling
  - Specification
  - Verification Method: Model Checking Algorithm

Labeling Algorithm

CTL model checking algorithm basically works by iteratively determining (i.e., labeling) states which satisfy a given CTL formula.

The basic input/output of labeling algorithm are as follows:

INPUT : A **CTL model *'M'* = *(S, →, L)*** where S is the set of states, → is the transition relation and L is the labeling function and a **CTL formula Φ**.

OUTPUT : The set of states of M which satisfy Φ.

Labeling Algorithm

- The adequate set of temporal operators for CTL is AF, EU and EX.
- First, we write the given formula Φ in terms of the connectives AF, EU and EX along with other logical connectives and truth value T.

Labeling Algorithm

- Atomic proposition
  - p: label state s with p if $p \in L(s)$
- Logical connectives
  - $p \wedge q$: label s with $p \wedge q$ if s is already labeled with p and q

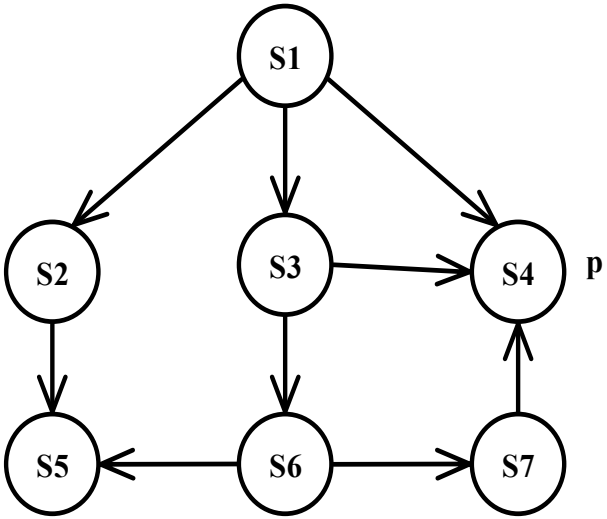
Labeling Algorithm

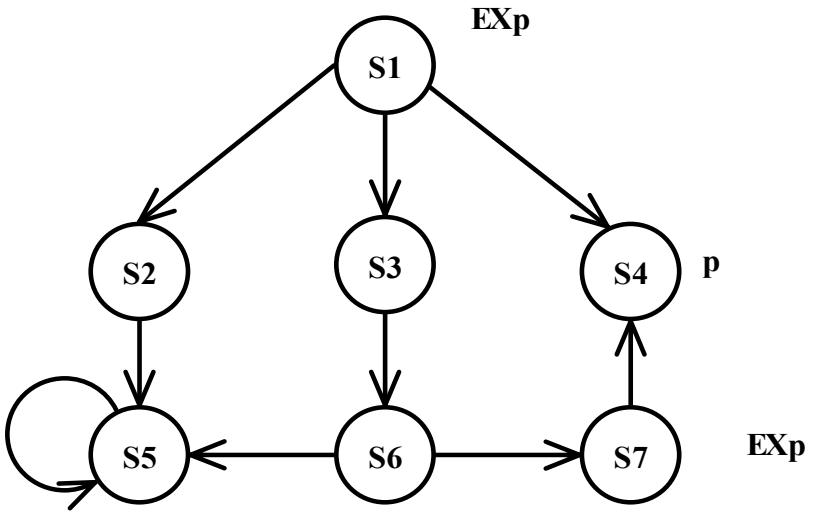
Temporal Operator:

EX p

Label any state with EX p if one of its successor is labeled with p

Labeling Algorithm Example
EF p





Labeling Algorithm

Function $SAT_{EX}(p)$

/* determines the set of states satisfying EXp */

local var X,Y

begin

X := SAT(p)

Y := $\{s_0 \in S \mid s_0 \rightarrow s_1$ for some $s_1 \in X\}$

return Y

end

Labeling Algorithm
AF p

Temporal Operator:

AF p

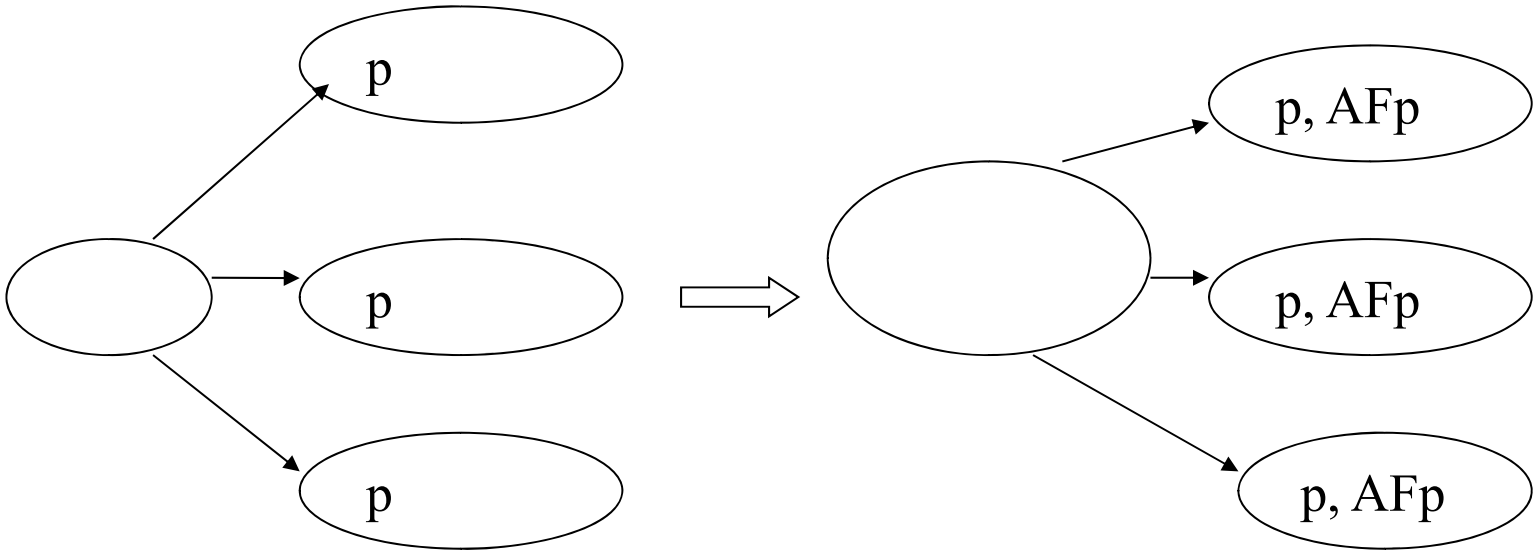
- If any state s is labeled with p, label it with AF p
- Repeat: label any state with AF p if all successor states are labeled with AF p until there is no change.

$$AF\ p \equiv p \vee AX\ AF\ p$$

Labeling Algorithm
AF p

Function $SAT_{AF}(p)$

/* determines the set of states satisfying AFp */

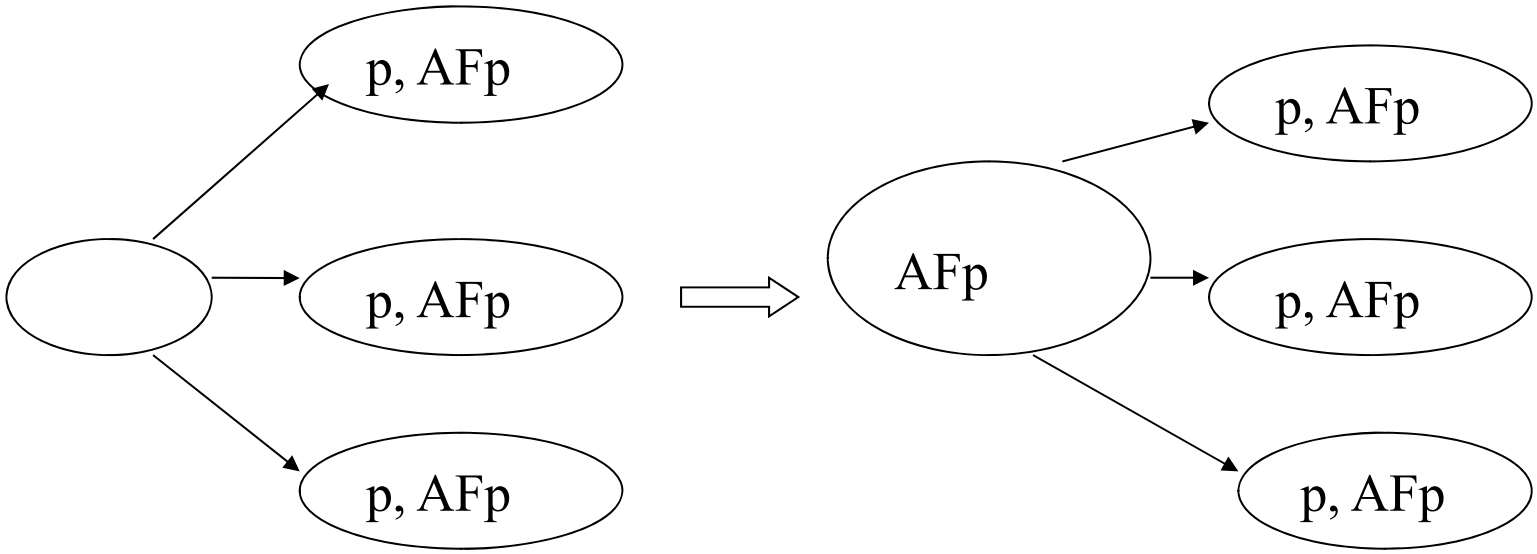


…Until no change

Labeling Algorithm
AF p

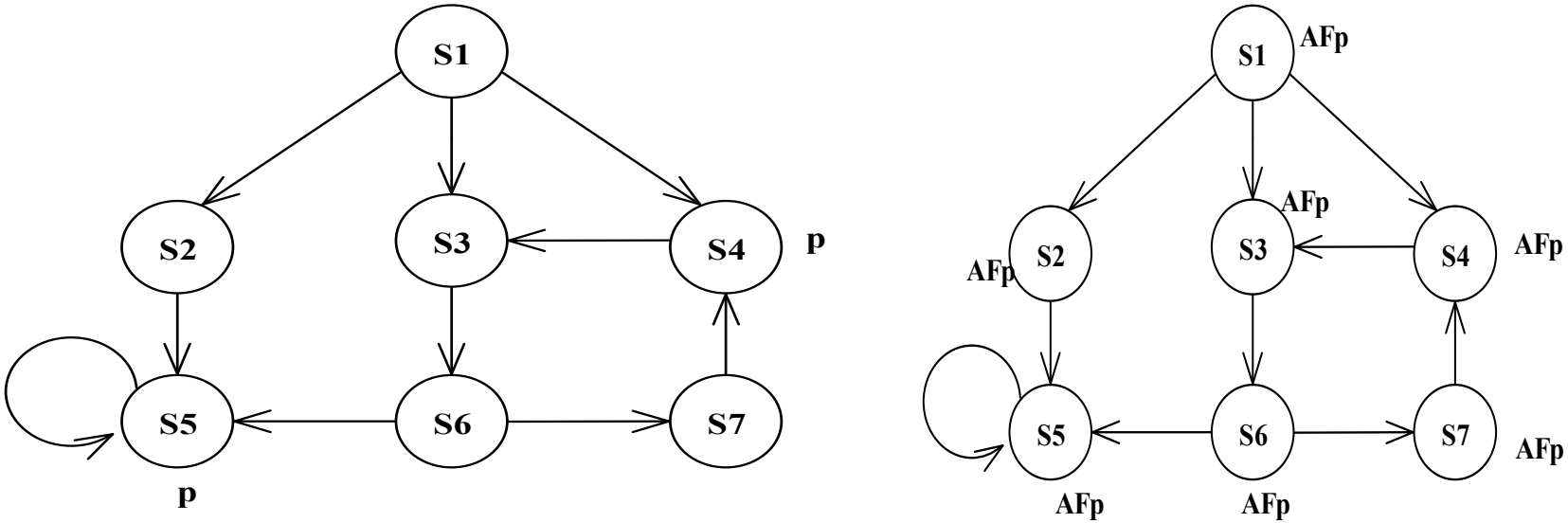
Function $SAT_{AF}(p)$

/* determines the set of states satisfying AFp */



…Until no change

Labeling Algorithm
AF p



Labeling Algorithm
E(p U q)

Temporal Operator:

E(p U q)

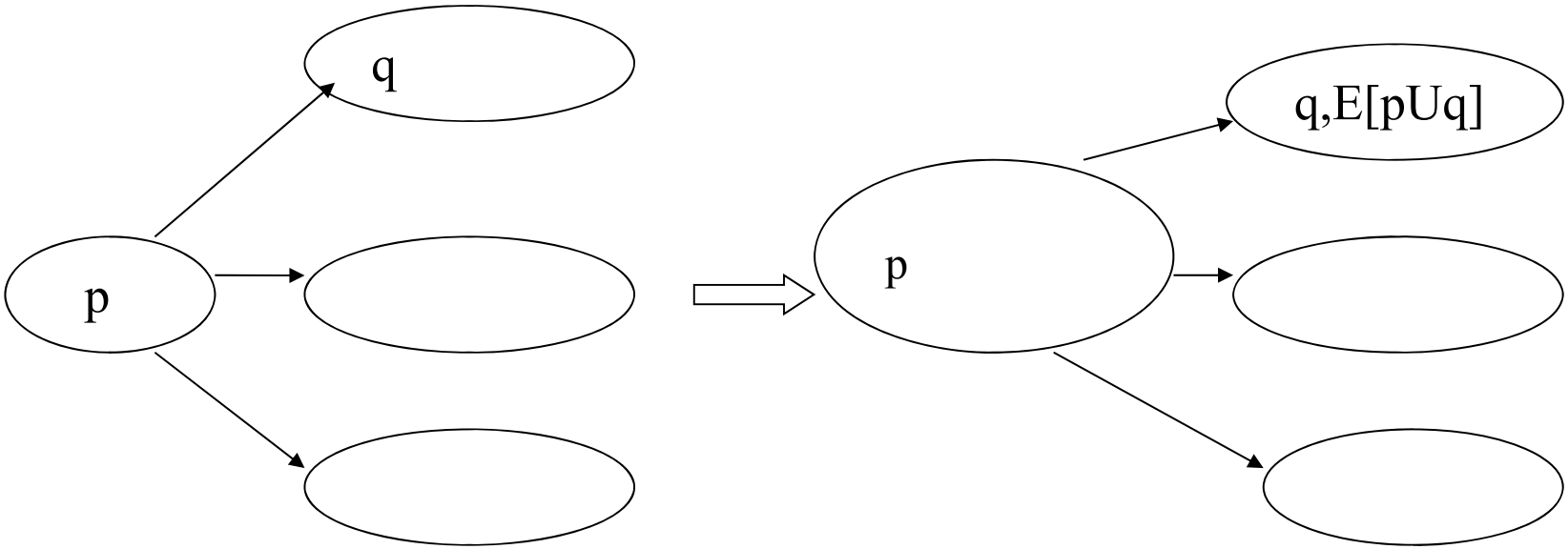
- If any state s is labeled with q, label it with E(p U q)
- Repeat: label any state with E(p U q) if it is labeled with p and at least one of its successor is labeled with E(p U q) until there is no change.

$$E[p\ U\ q] \equiv q \vee (p \wedge EX\ E[p\ U\ q])$$

Labeling Algorithm
E(p U q)

Function $SAT_{EU}(p,q)$

/* determines the set of states satisfying E(p U q) */

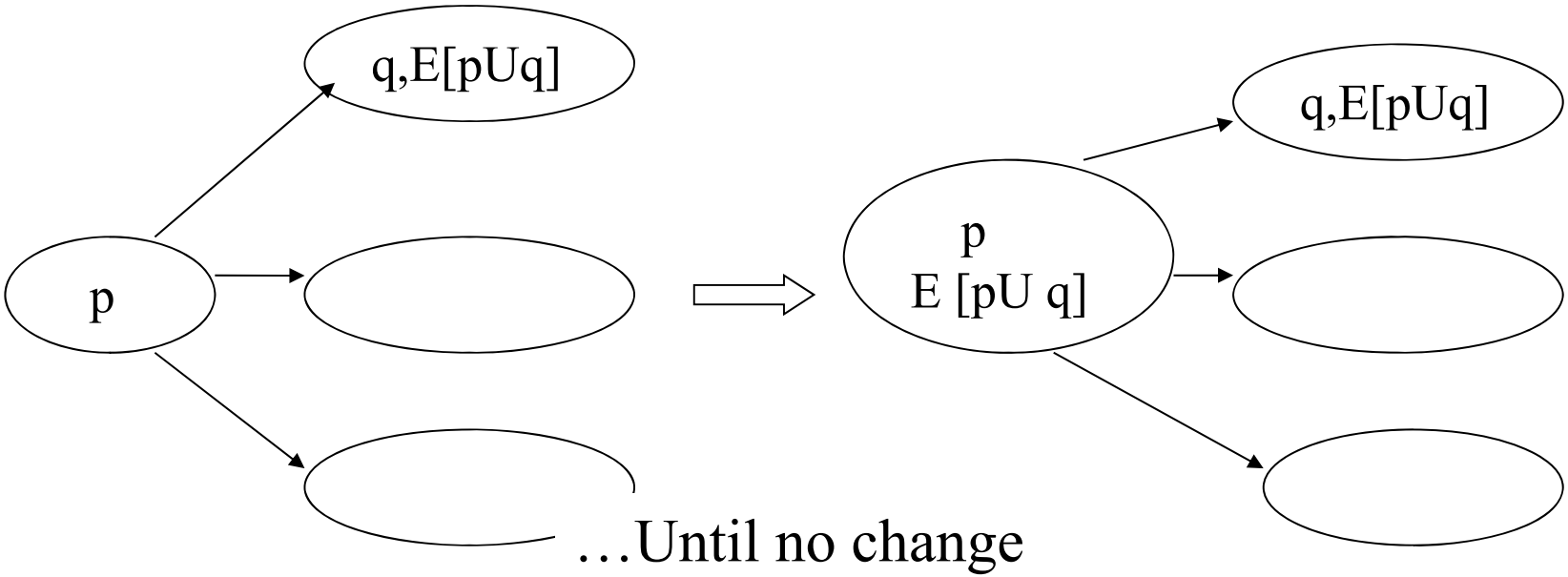


…Until no change

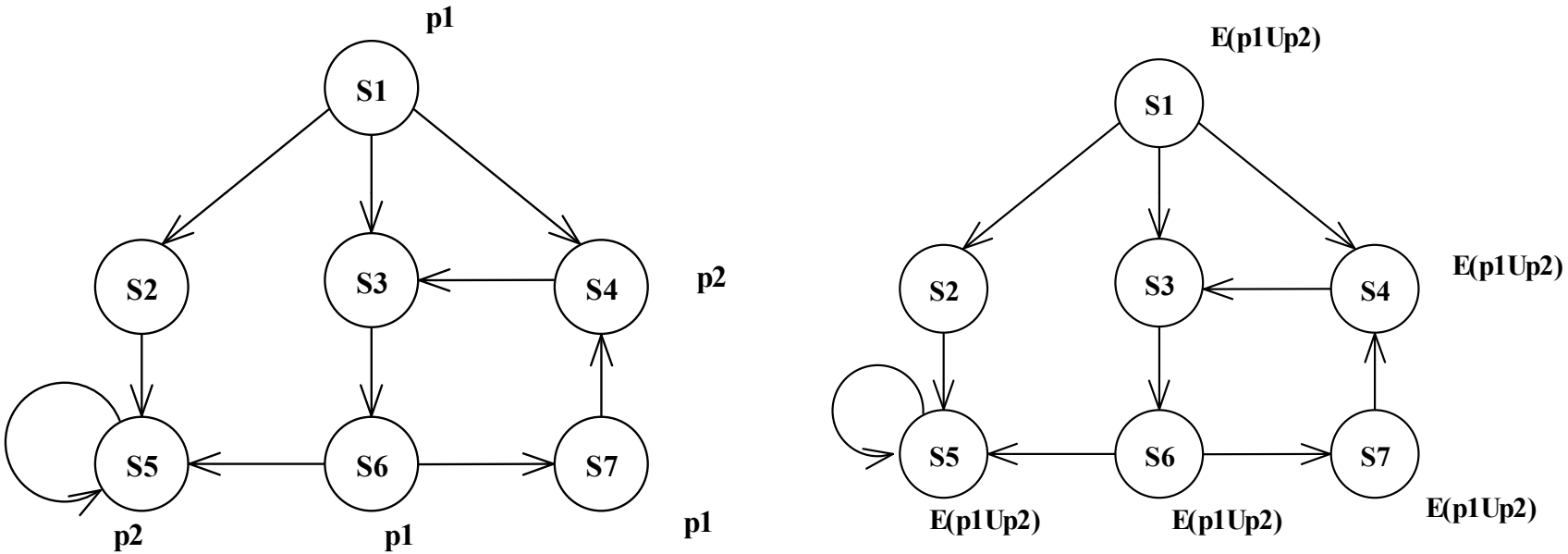
Labeling Algorithm
E(p U q)

Function $SAT_{EU}(p,q)$

/* determines the set of states satisfying E(p U q) */



Labeling Algorithm
E(p1 U p2)



Labeling Algorithm
E(p U q)

Function $SAT_{EU}(p,q)$

/* determines the set of states satisfying E(p U q) */
local var W,X,Y
begin
    W := SAT(p), X := S, Y := SAT(q)
    repeat until X = Y
    begin
        X := Y
        Y := Y $\cup$ (W $\cap$ {s | exists s' such that s $\rightarrow$ s' and s' $\in$ Y}
    end
    return Y
end

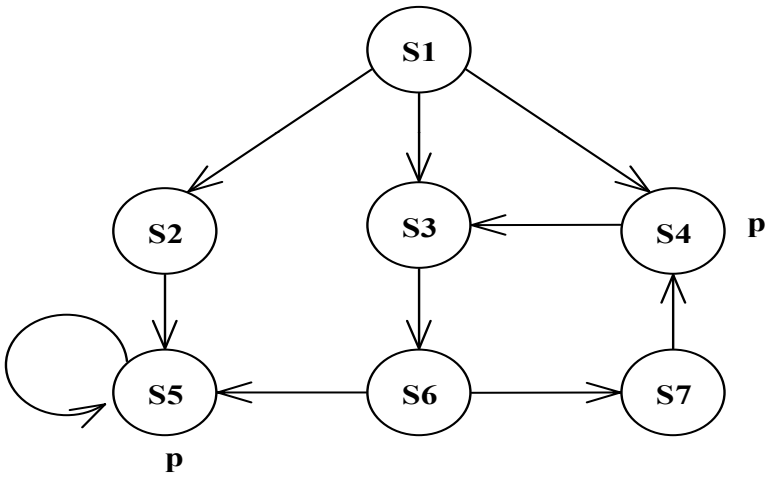
Labeling Algorithm AGp

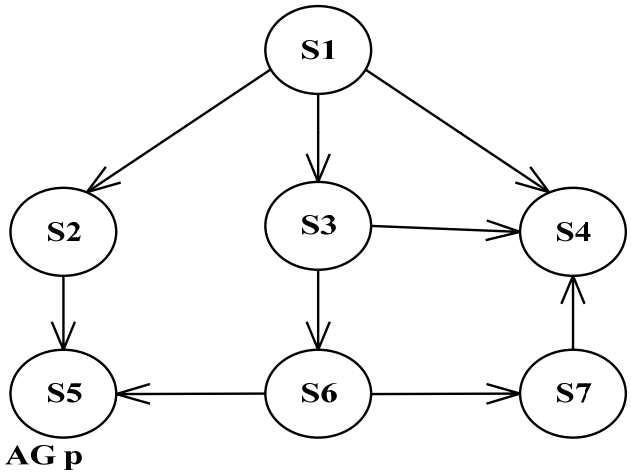
Step1: Label all the states with AGp.

Step2: If any state s is not labeled with p, delete the label AGp.

Step3: Repeat: delete the label AGp from any state if all of its successors are not labeled with AGp until there is no change.

Labeling Algorithm AGp

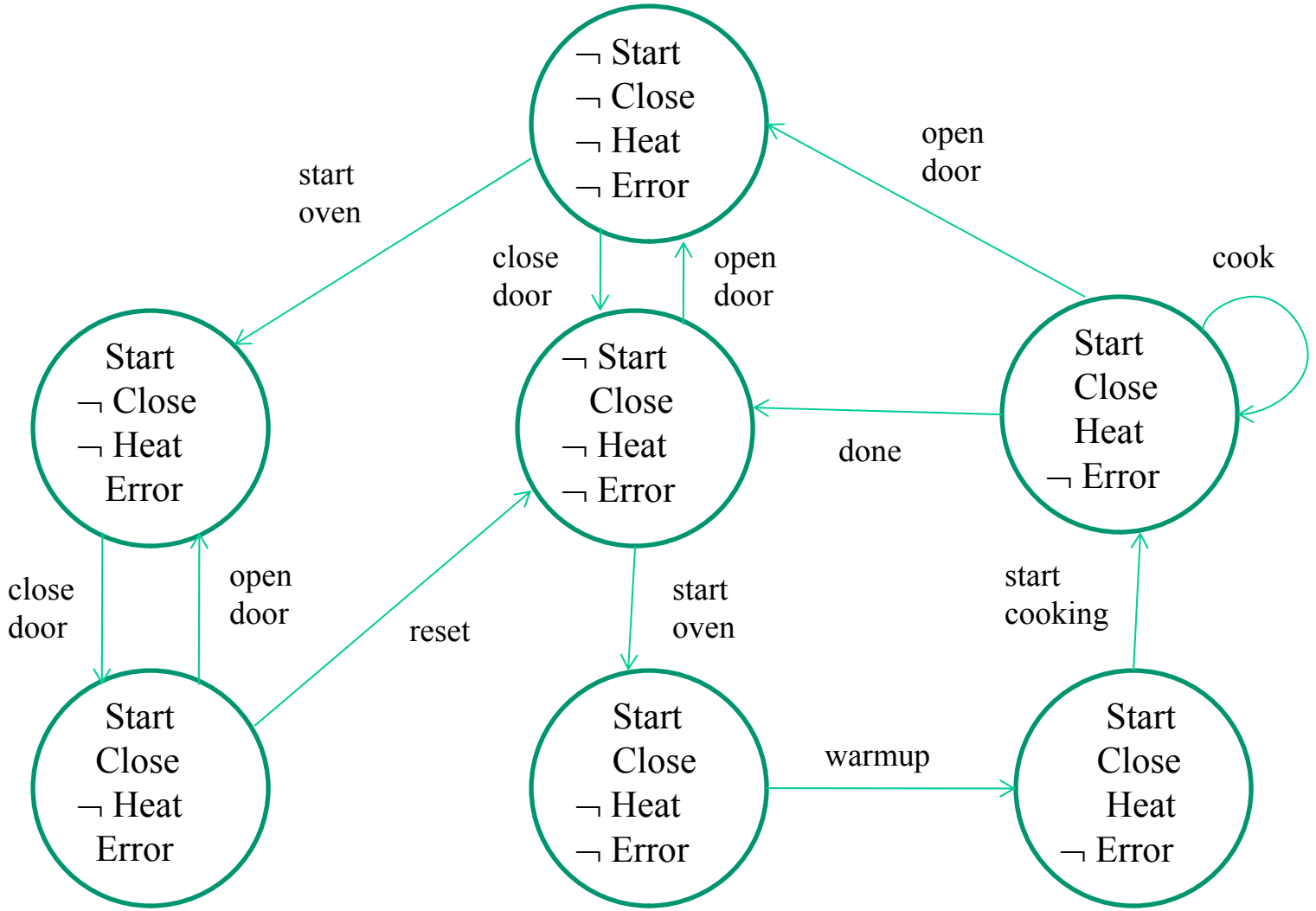




Example

- Design a controller for a microwave oven.
  - Door of the oven: either open or close
  - Start of the oven
  - reset

Model


¬ Start
¬ Close
¬ Heat
¬ Error
start
oven
open
door
close
door
open
door
cook
Start
¬ Close
¬ Heat
Error
¬ Start
Close
¬ Heat
¬ Error
Start
Close
Heat
¬ Error
done
close
door
open
door
reset
start
oven
start
cooking
Start
Close
¬ Heat
Error
Start
Close
¬ Heat
¬ Error
warmup
Start
Close
Heat
¬ Error

Conditions to be Checked

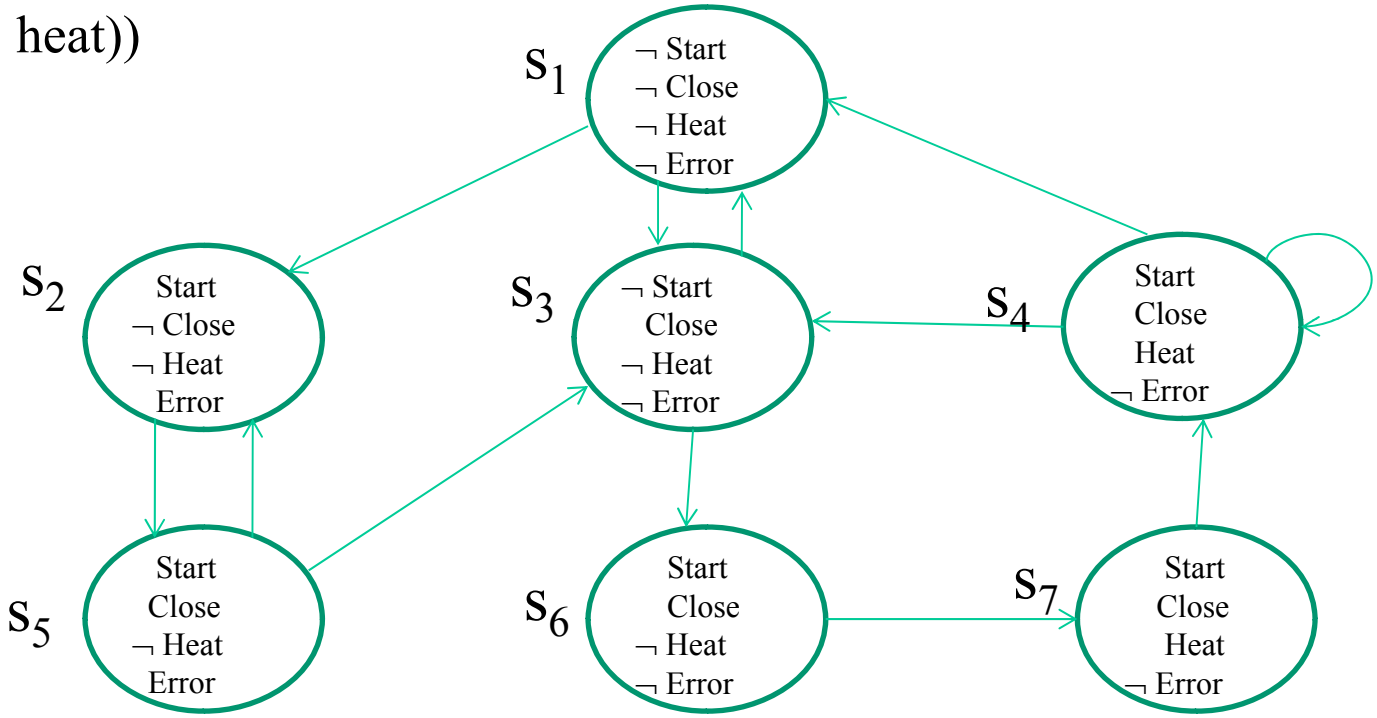
- Microwave oven should not heat up with its door open.
- Once we start the oven, eventually it must start heating.

CTL formulas for the conditions

- Microwave oven should not heat up with its door open.
  - AG ($\neg$ ($\neg$ close $\wedge$ heat))
- Once we start the oven, eventually it must start heating.
  - AG(start $\rightarrow$ AF heat)

Model Checking for the conditions

AG (¬ (¬ close ∧ heat))



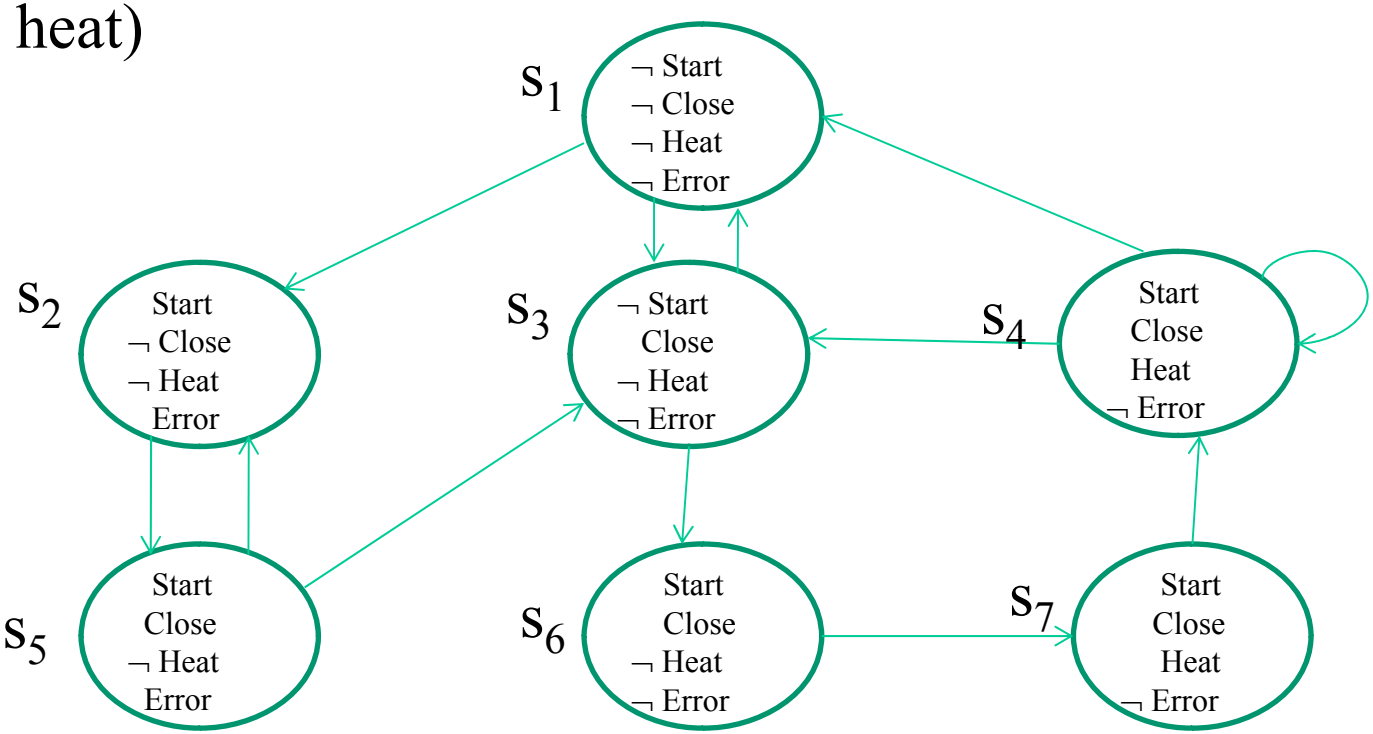
(¬ close ∧ heat) S1={}

¬(¬ close ∧ heat) S2=S

AG (¬ (¬ close ∧ heat)) S2=S

Model Checking for the conditions

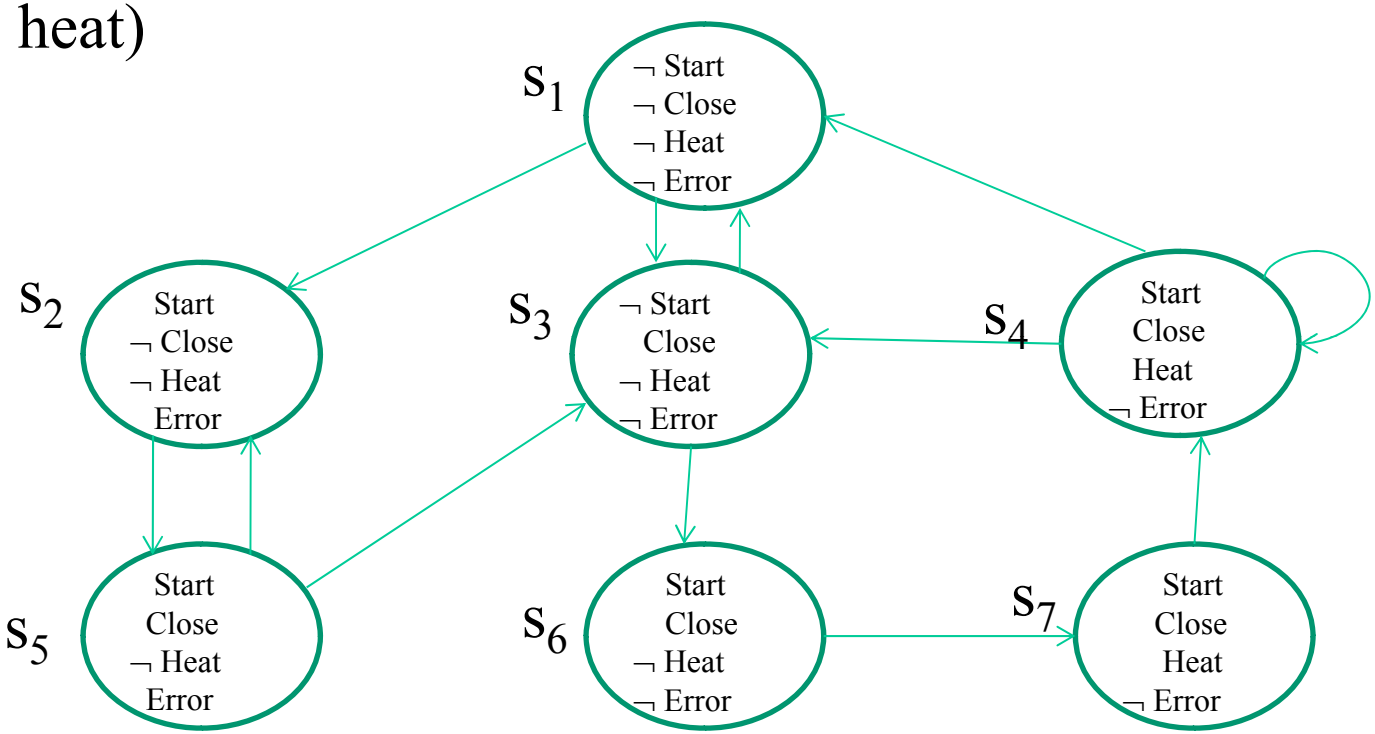
AG(start $\rightarrow$ AF heat)



(heat) $S1=\{s_4, s_7\}$

Model Checking for the conditions

AG(start $\rightarrow$ AF heat)

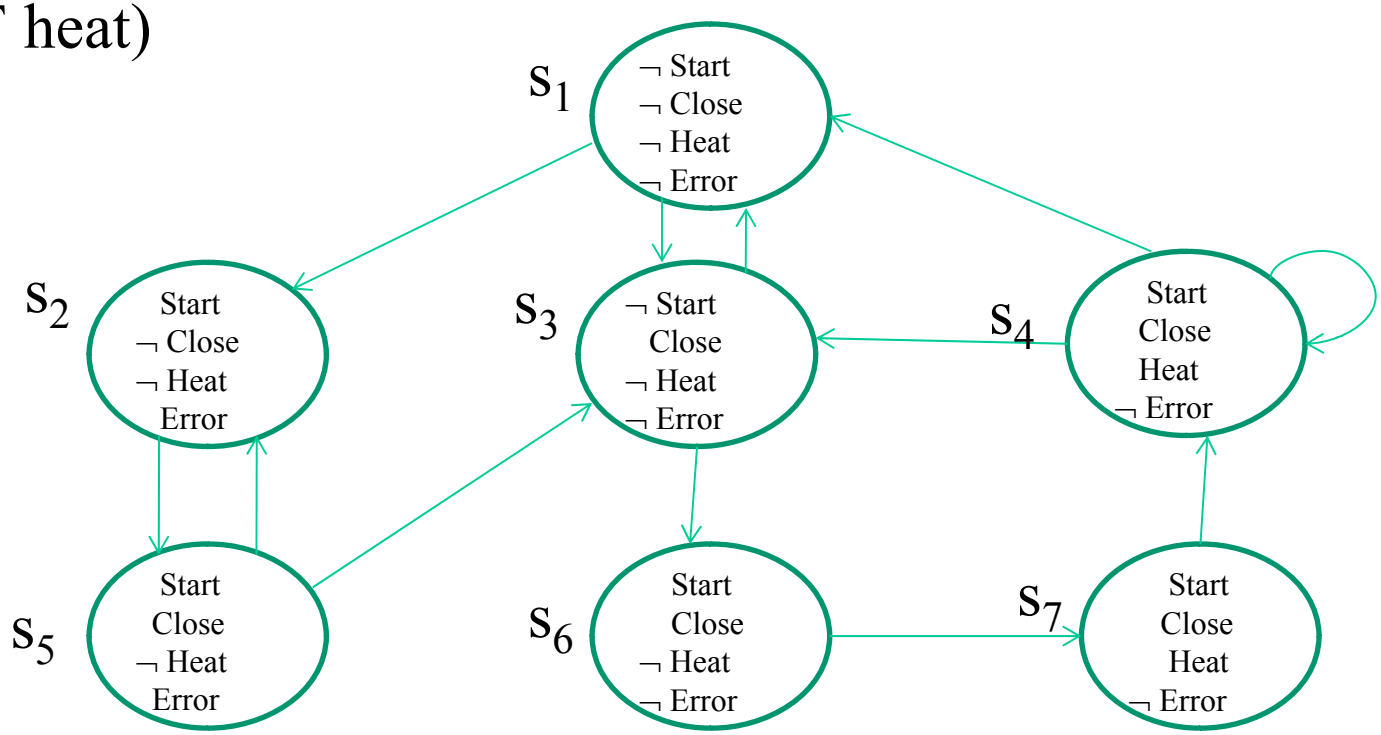


heat $\quad S1=\{s_4, s_7\}$

AF heat

Model Checking for the conditions

AG(start $\rightarrow$ AF heat)

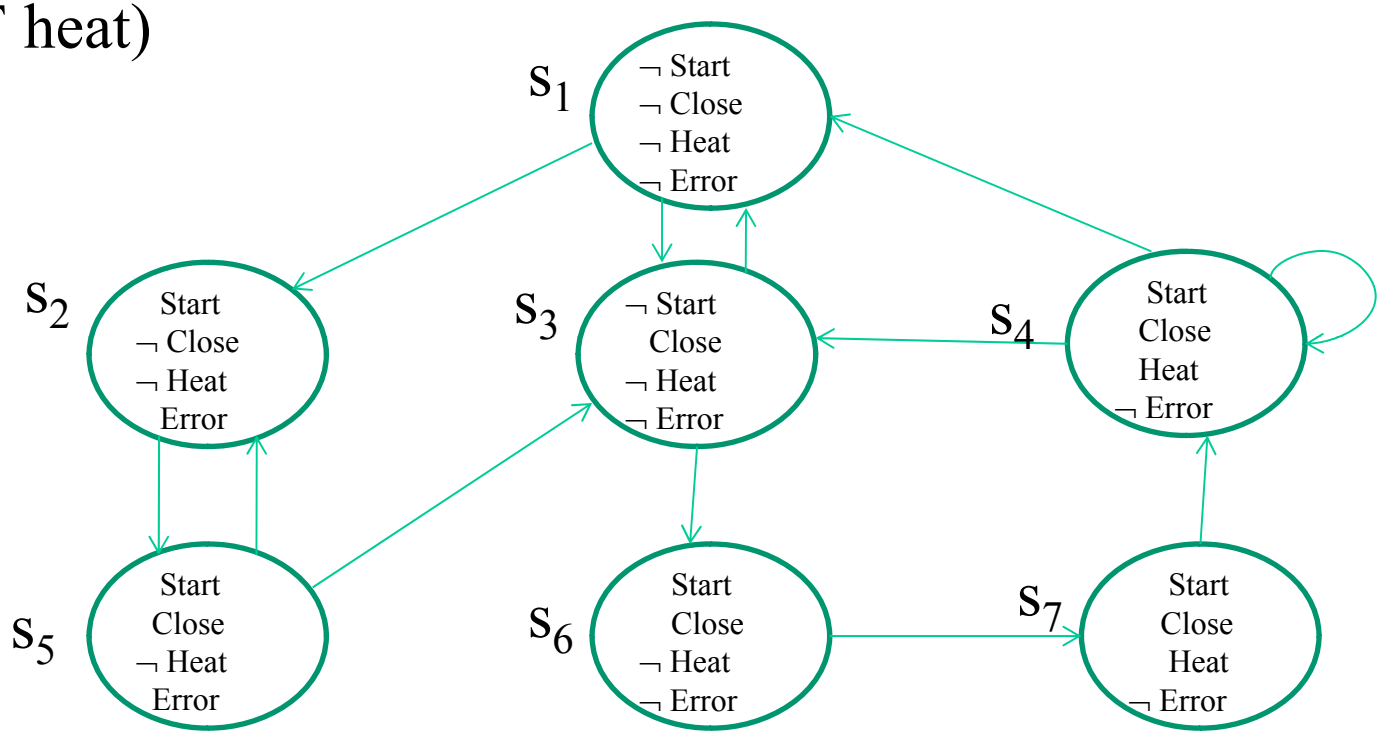


heat $S1=\{s_4, s_7\}$

AF heat $S1=\{s_4, s_7\}$

Model Checking for the conditions

AG(start $\rightarrow$ AF heat)

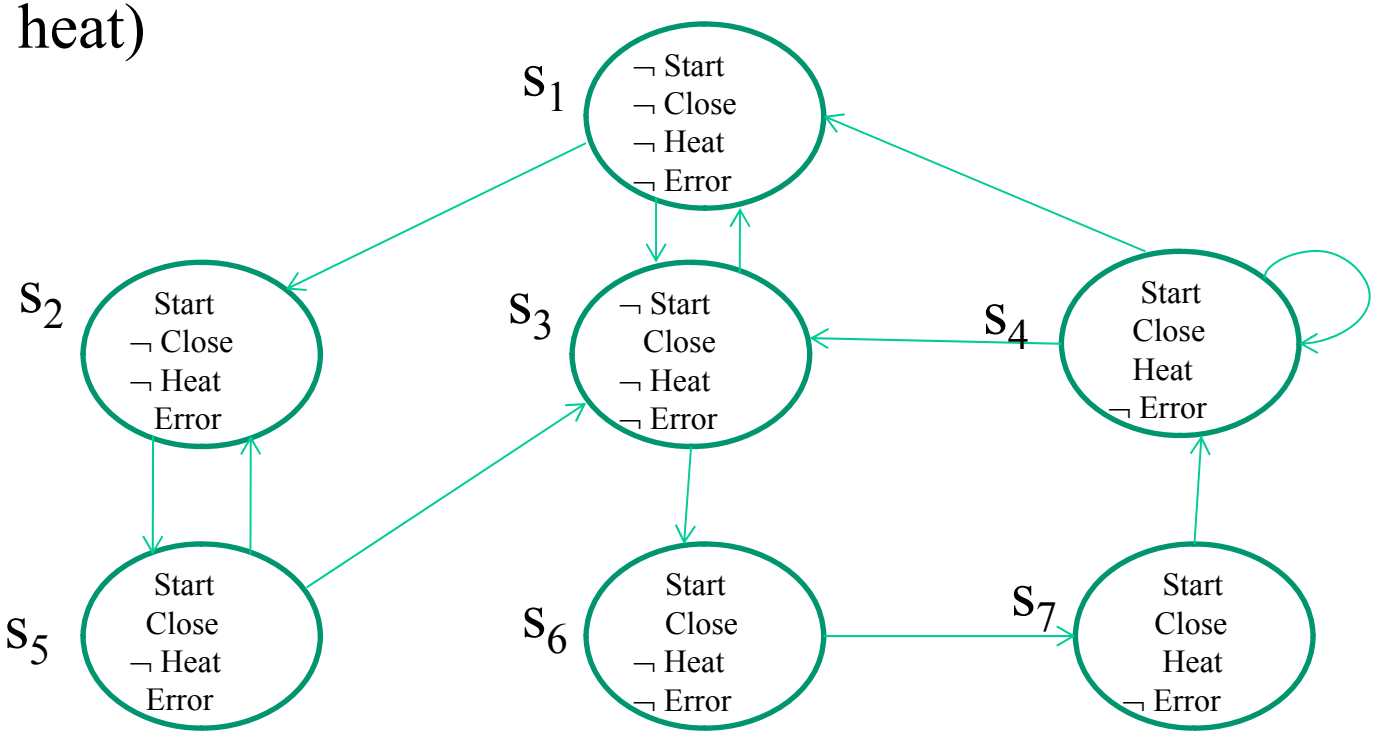


heat $S1=\{s_4, s_7\}$

AF heat $S1=\{s_4, s_7\}$ $S2=\{s_4, s_7, s_6\}$

Model Checking for the conditions

AG(start $\rightarrow$ AF heat)

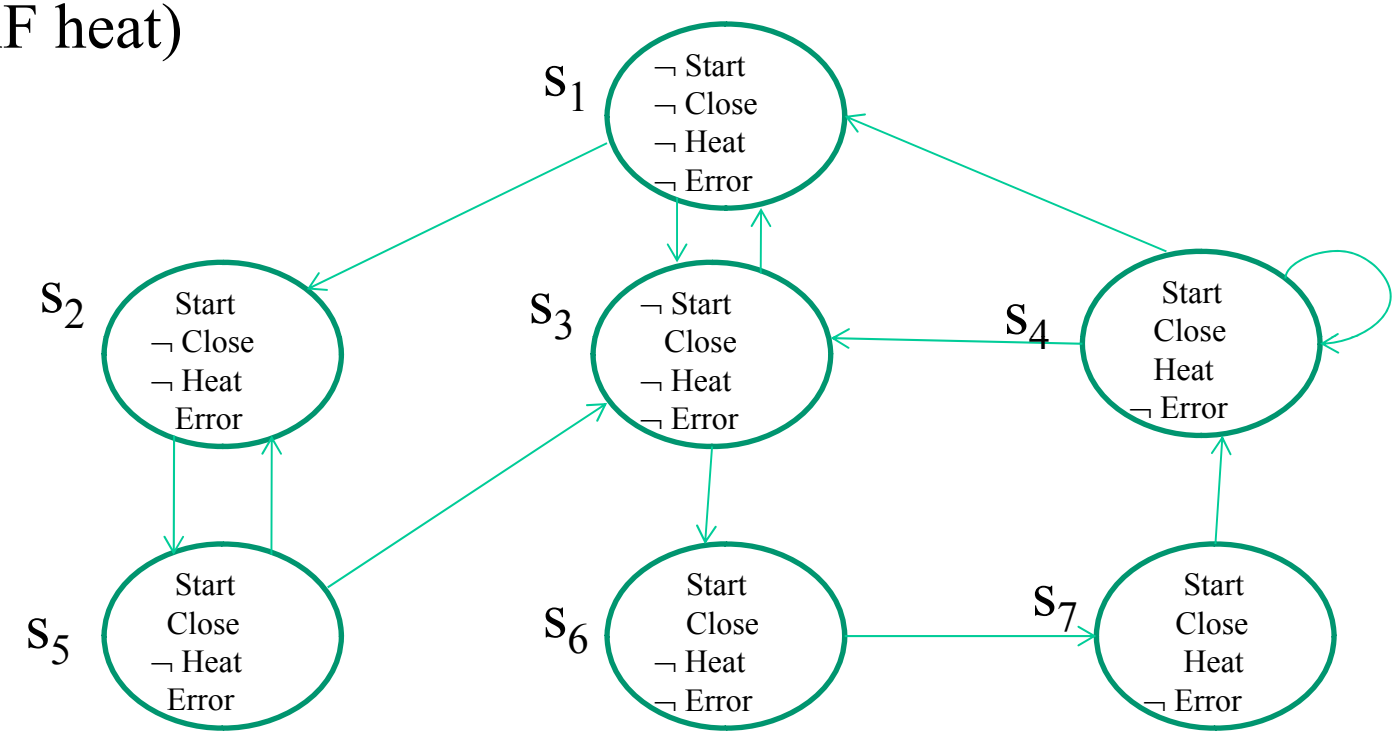


heat $S1=\{s_4, s_7\}$

AF heat $S1=\{s_4, s_7\}$ $S2=\{s_4, s_7, s_6\}$ $S3=\{s_4, s_7, s_6\}$

Model Checking for the conditions

AG(start $\rightarrow$ AF heat)



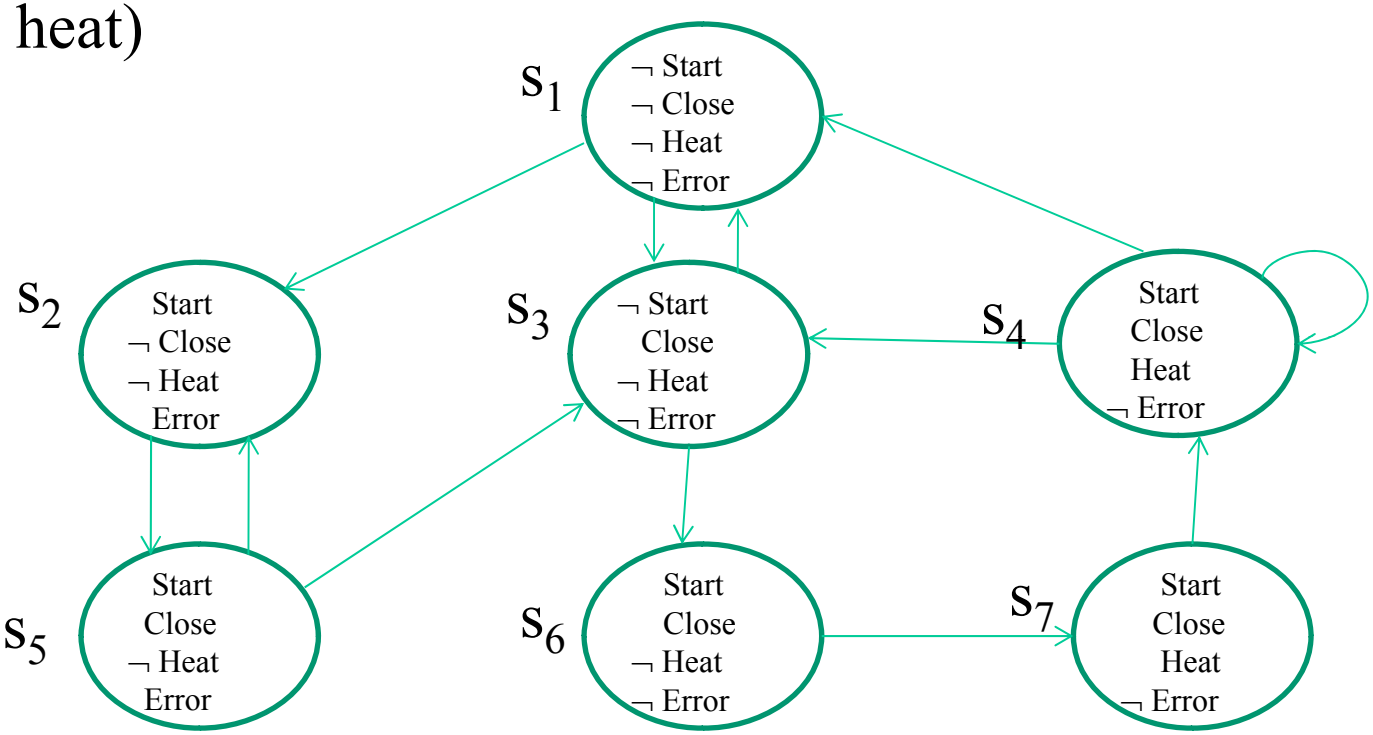
heat $S1=\{s_4, s_7\}$

AF heat $S2=\{s_4, s_7, s_6\}$

start $S3=\{s_4, s_7, s_6, s_2, s_5\}$

Model Checking for the conditions

AG(start $\rightarrow$ AF heat)



heat $S1=\{s_4, s_7\}$

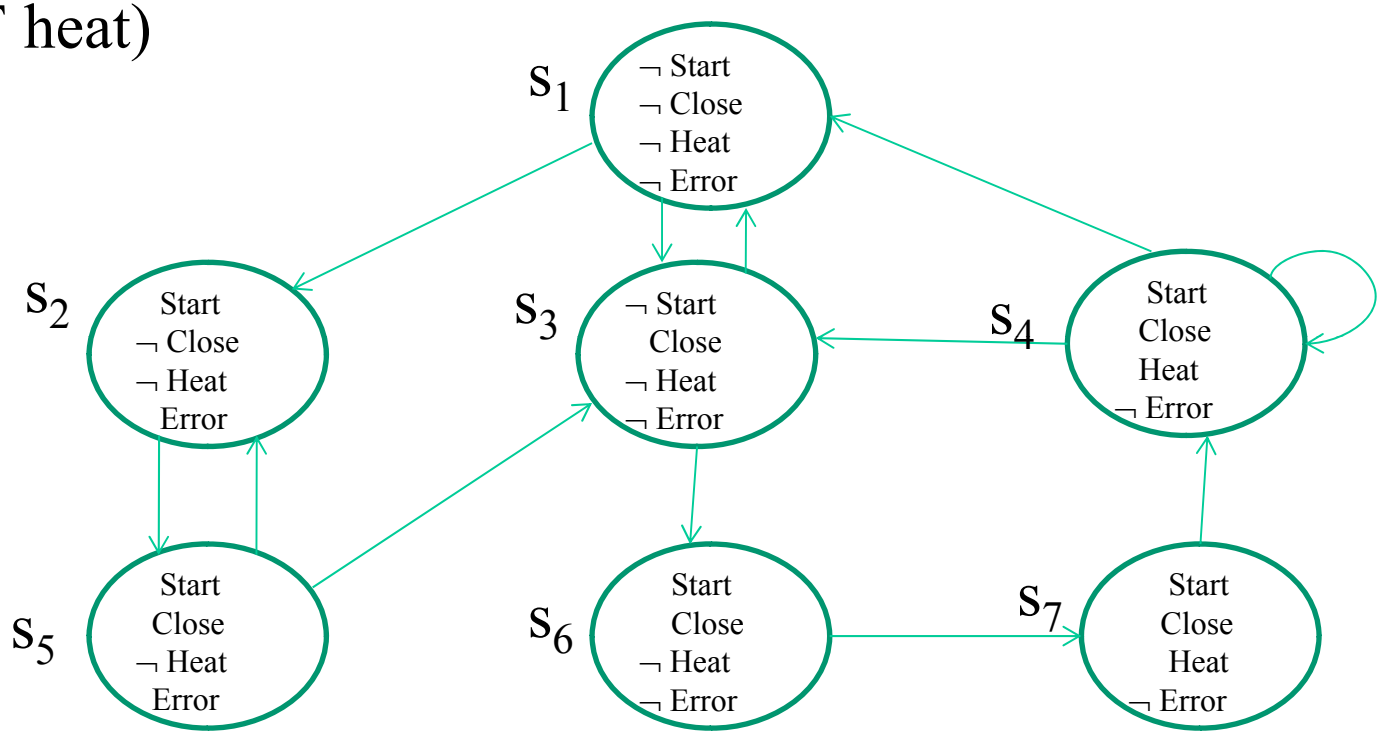
start $S3=\{s_4, s_7, s_6, s_2, s_5\}$

AF heat $S2=\{s_4, s_7, s_6\}$

(start $\rightarrow$ AF heat) $S4=\{s_4, s_7, s_6, s_3, s_1\}$

Model Checking for the conditions

AG(start $\rightarrow$ AF heat)

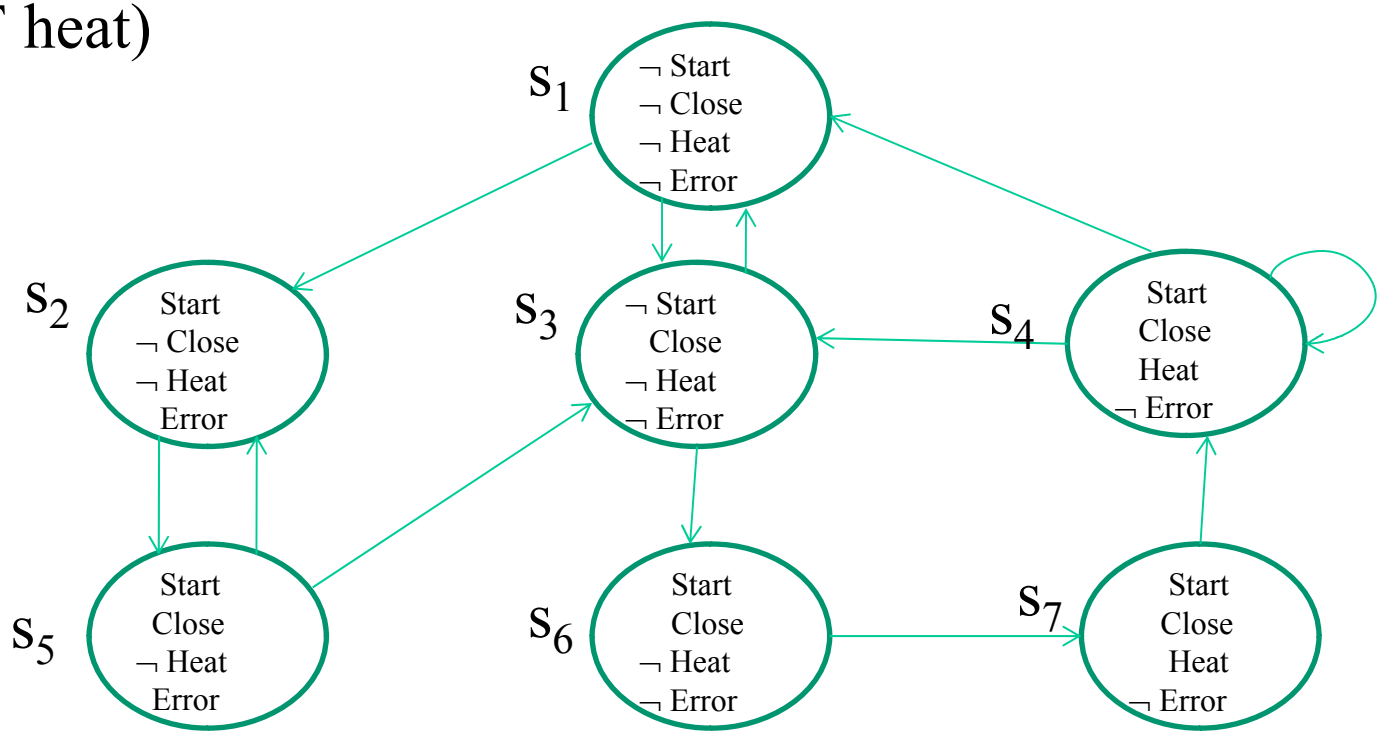


AG(start $\rightarrow$ AF heat) $S5=\{s_4, s_7, s_6, s_3, s_1\}$

$S6=\{s_4, s_7, s_6, s_3\}$

Model Checking for the conditions

AG(start $\rightarrow$ AF heat)

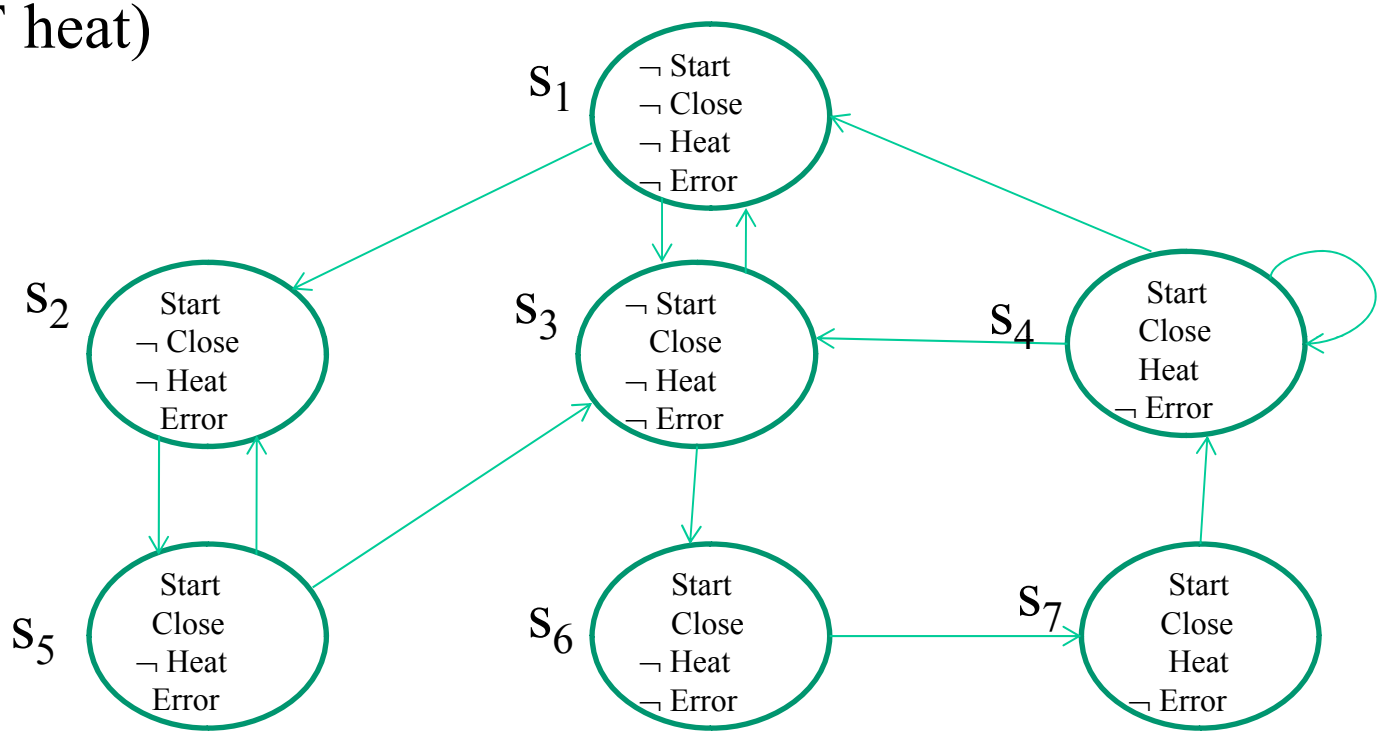


AG(start $\rightarrow$ AF heat) $S6=\{s_4, s_7, s_6, s_3\}$

$S7=\{s_4, s_7, s_6\}$

Model Checking for the conditions

AG(start $\rightarrow$ AF heat)

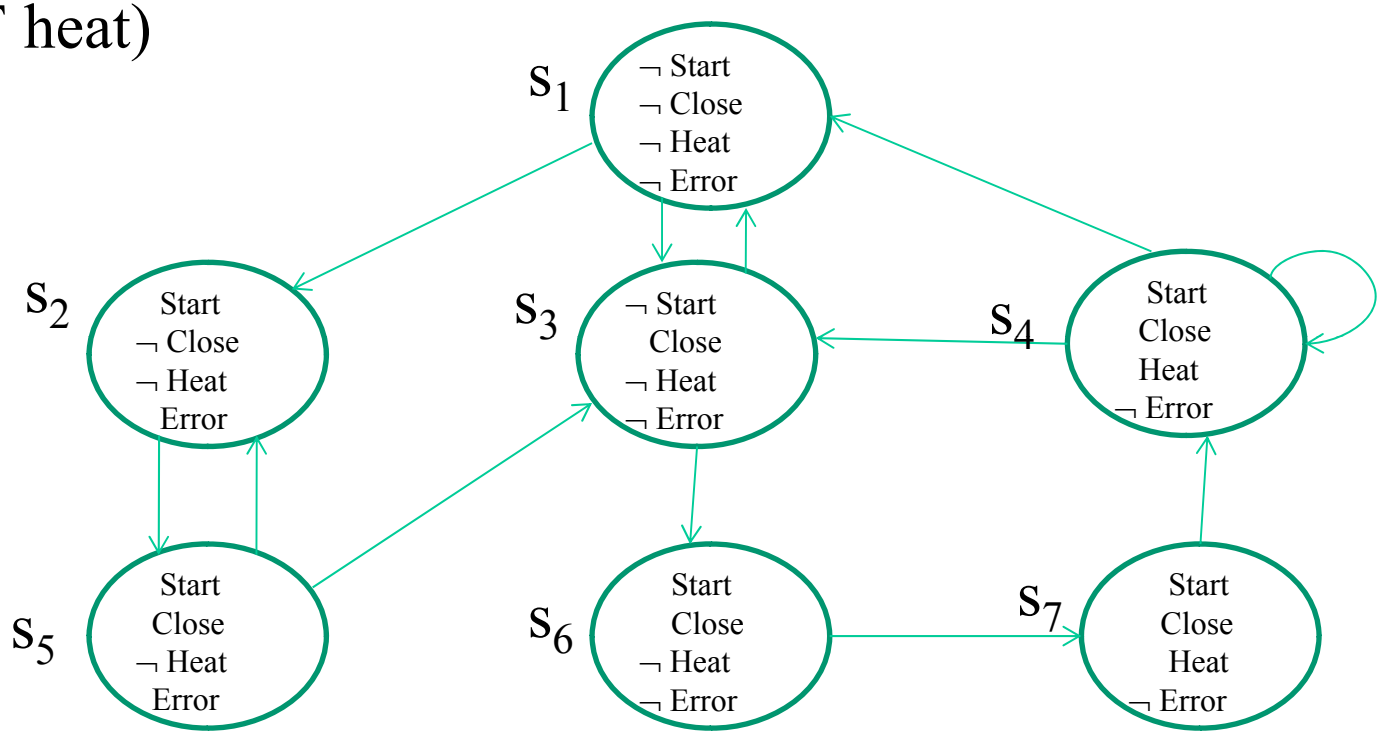


AG(start $\rightarrow$ AF heat) $S7=\{s_4, s_7, s_6\}$

$S8=\{s_7, s_6\}$

Model Checking for the conditions

AG(start $\rightarrow$ AF heat)

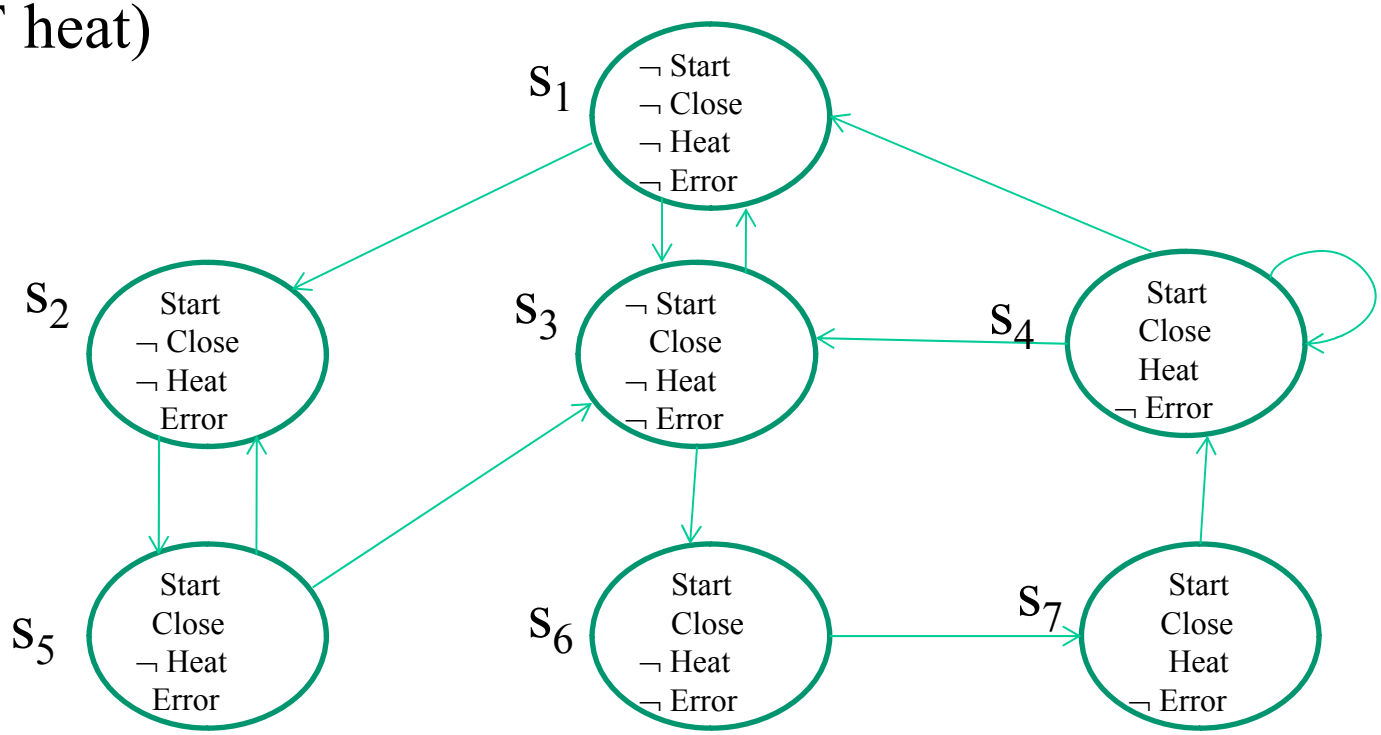


AG(start $\rightarrow$ AF heat) $S8=\{ s_7 , s_6 \}$

$S9=\{ s_6 \}$

Model Checking for the conditions

AG(start $\rightarrow$ AF heat)



AG(start $\rightarrow$ AF heat) S9={ s_6 }

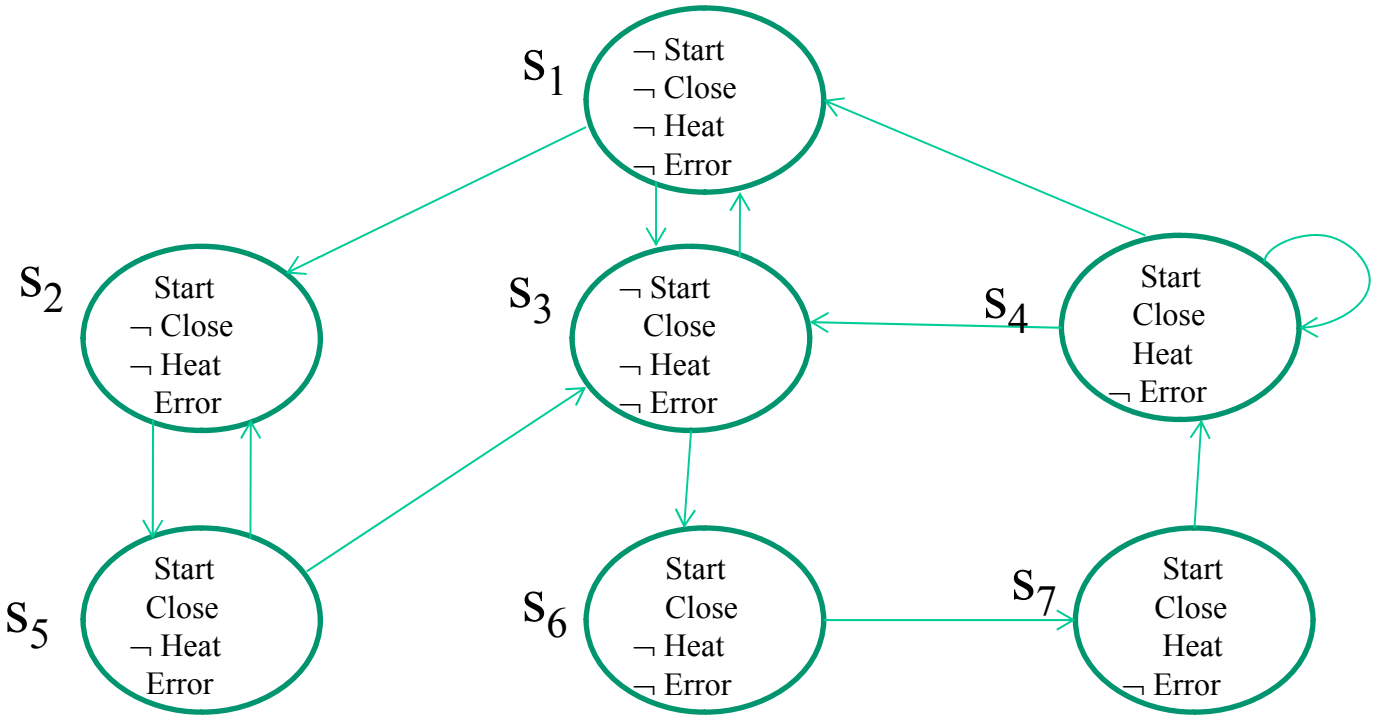
S10={ }

What if Error

- The given specification is not true
  - AG(start $\rightarrow$ AF heat)
- What to do
  - Revisit the design
  - Look for correct sequence of operation
  - AG((start $\wedge$ ($\neg$ error))$\rightarrow$ AF (heat $\wedge$ ($\neg$ error)))
  - AG((start $\wedge$ (close))$\rightarrow$ AF (heat $\wedge$ (close)))

Model Checking for the conditions

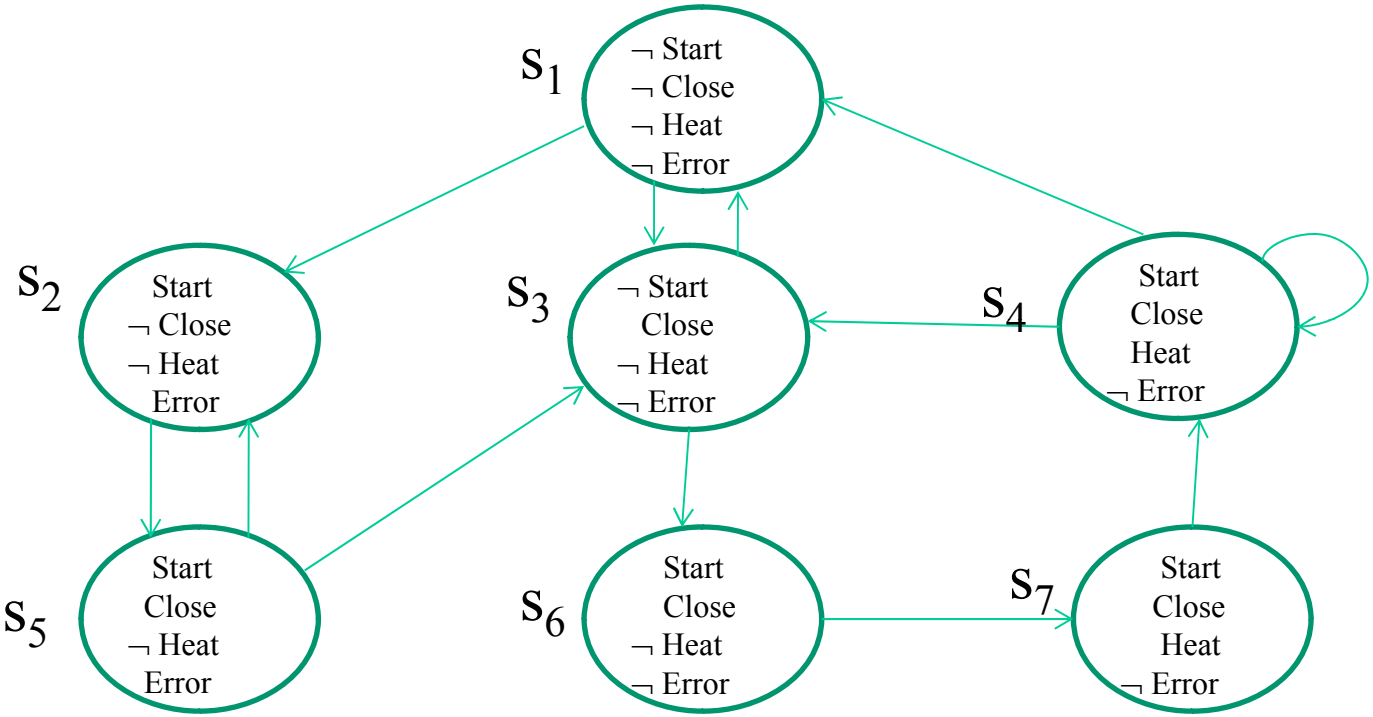
$AG((start \wedge (\neg\ error)) \rightarrow\ AF\ (heat \wedge (\neg\ error)))$



$heat \wedge (\neg\ error)$ $S1 = \{s_4, s_7\}$

Model Checking for the conditions

AG((start $\wedge$ ($\neg$ error))$\rightarrow$ AF (heat $\wedge$ ($\neg$ error)))

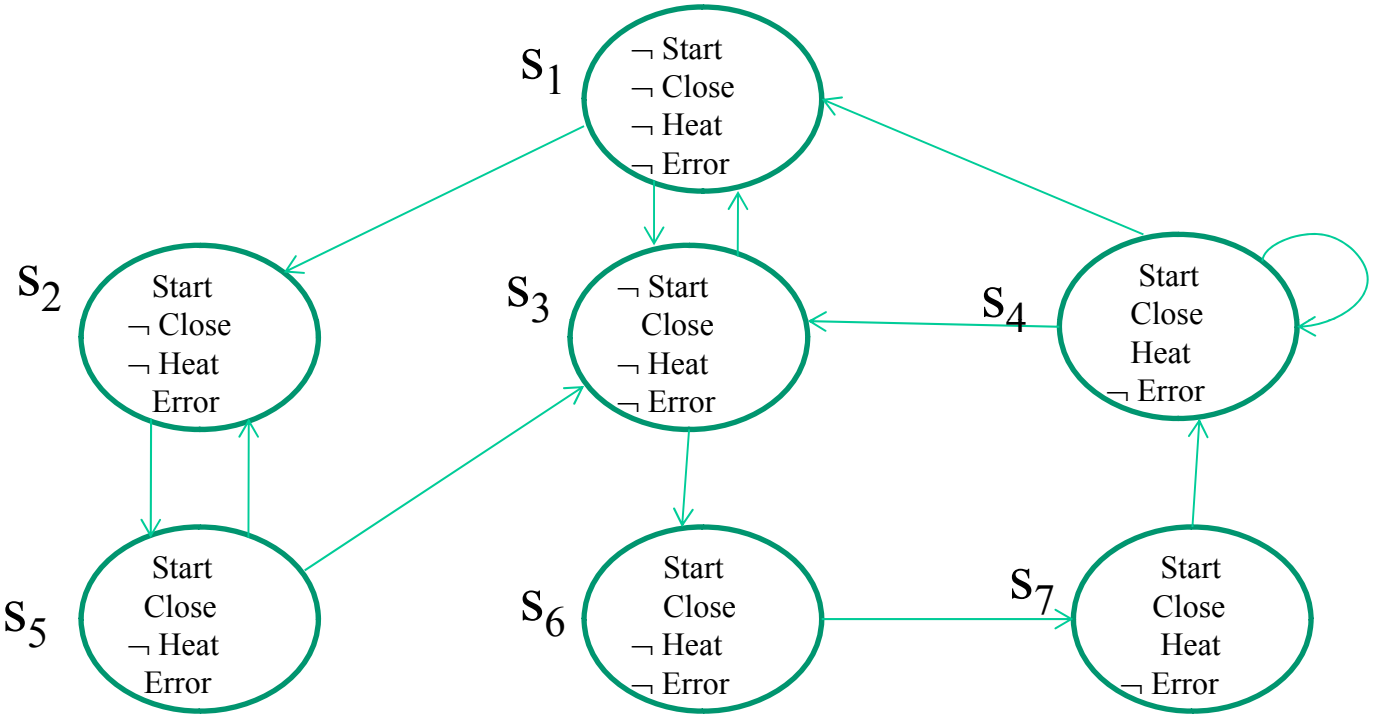


heat $\wedge$ ($\neg$ error) S1=$\{s_4, s_7\}$

AF (heat $\wedge$ ($\neg$ error))

Model Checking for the conditions

$AG((start \wedge (\neg\ error)) \rightarrow\ AF\ (heat \wedge (\neg\ error)))$

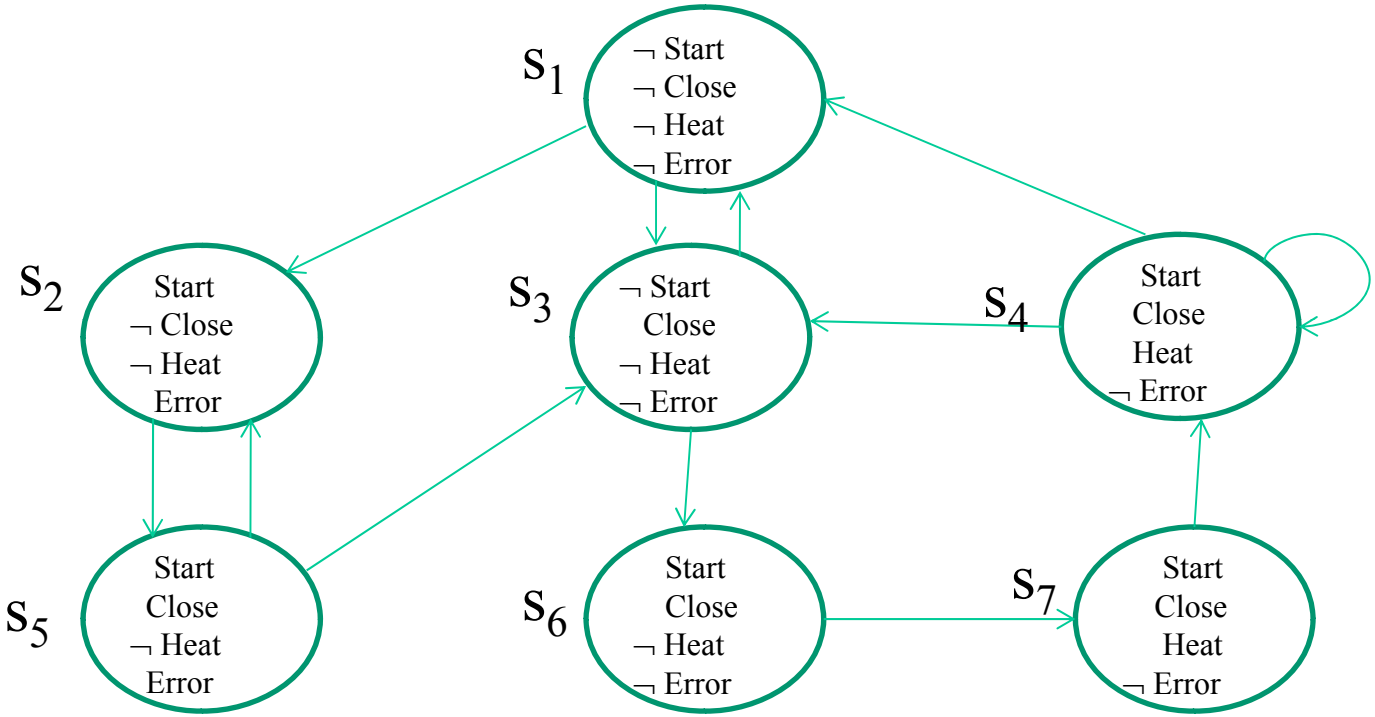


$heat \wedge (\neg\ error)$ $S1=\{s_4, s_7\}$

$AF\ (heat \wedge (\neg\ error))$ $S1=\{s_4, s_7\}$

Model Checking for the conditions

AG((start $\wedge$ ($\neg$ error))$\rightarrow$ AF (heat $\wedge$ ($\neg$ error)))

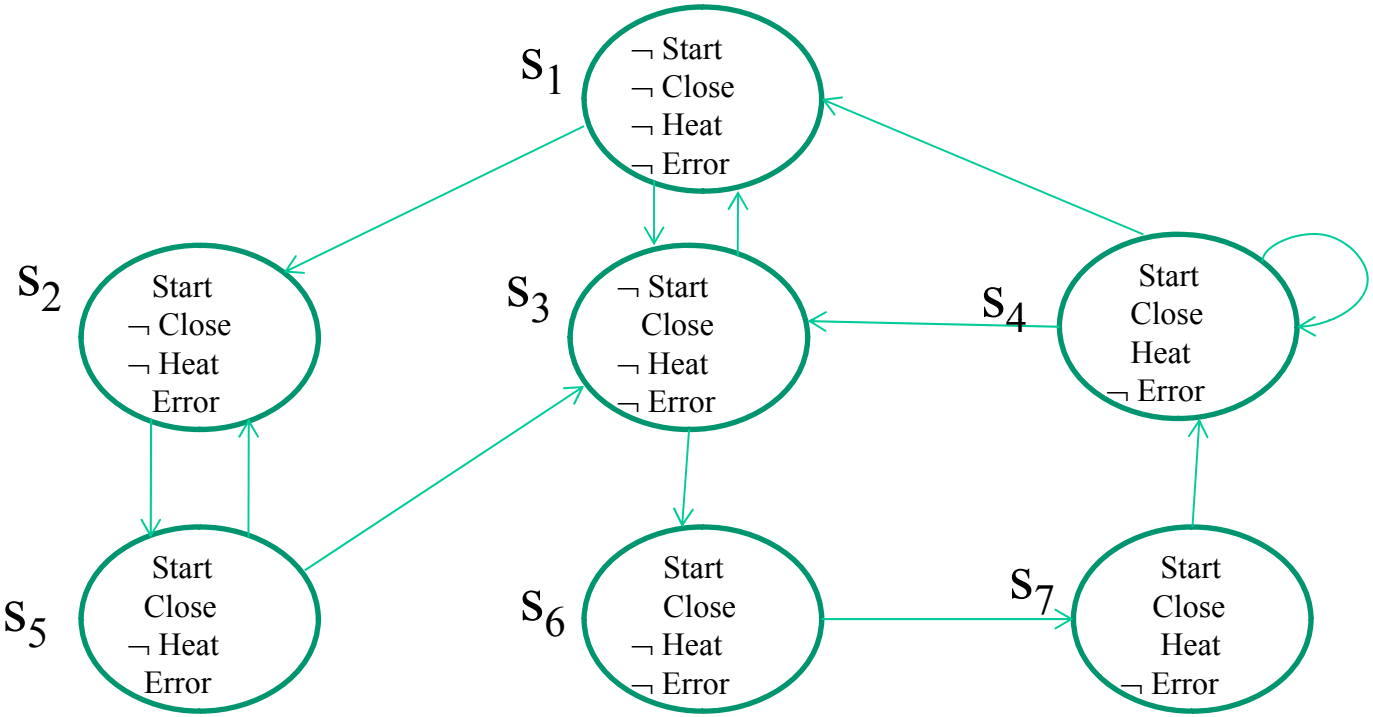


heat $\wedge$ ($\neg$ error) S1={s_4, s_7 }

AF (heat $\wedge$ ($\neg$ error)) S1={s_4, s_7 } S2={s_4, s_7 , s_6 }

Model Checking for the conditions

AG((start $\wedge$ ($\neg$ error))$\rightarrow$ AF (heat $\wedge$ ($\neg$ error)))

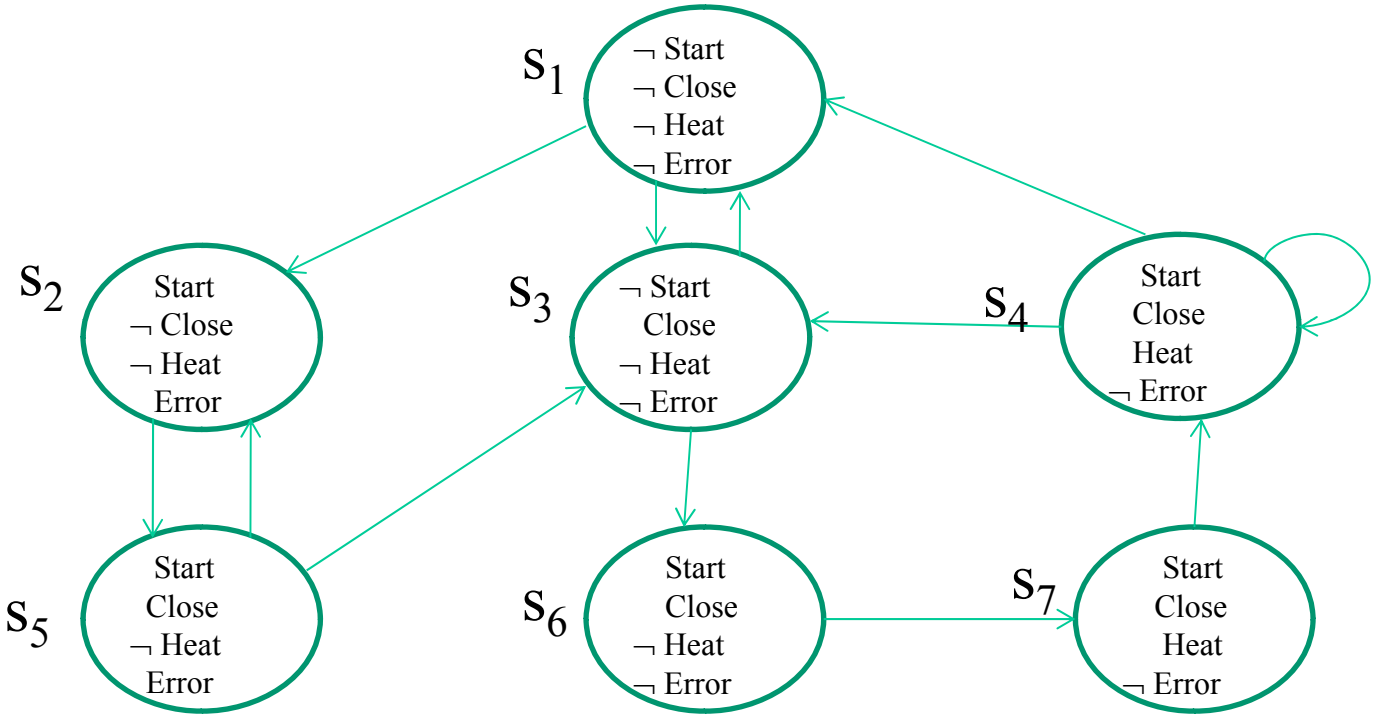


heat $\wedge$ ($\neg$ error) S1={s_4, s_7 }

AF (heat $\wedge$ ($\neg$ error)) S1={s_4, s_7 } S2={s_4, s_7 , s_6 } S3={s_4, s_7 , s_6 }

Model Checking for the conditions

AG((start ∧ (¬ error))→ AF (heat ∧ (¬ error)))



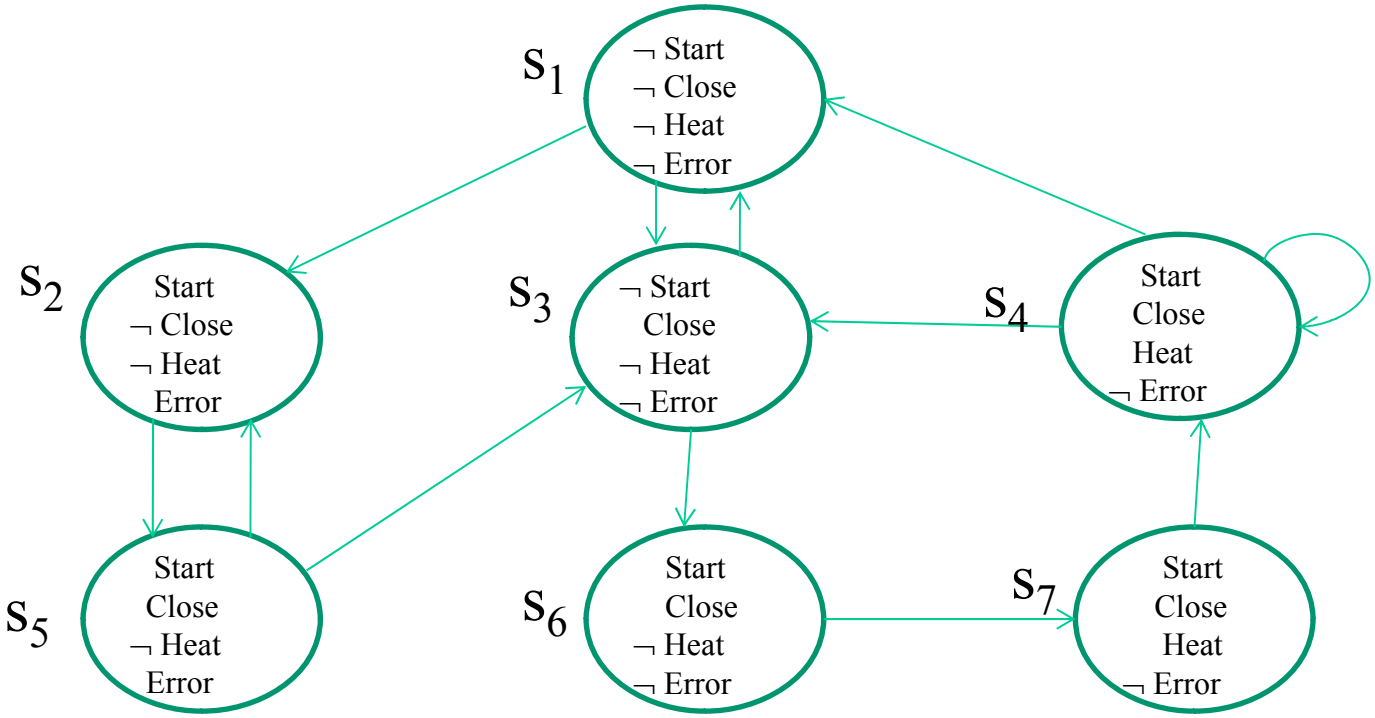
(start ∧ (¬ error)) $S1=\{s_4, s_7\}$

AF (heat ∧ (¬ error)) $S2=\{s_4, s_7, s_6\}$

(start ∧ (¬ error)) $S3=\{s_4, s_7, s_6\}$

Model Checking for the conditions

$AG((start \wedge (\neg\ error)) \rightarrow\ AF\ (heat \wedge (\neg\ error)))$



$(heat \wedge (\neg\ error)))$ $S1=\{s_4, s_7\}$

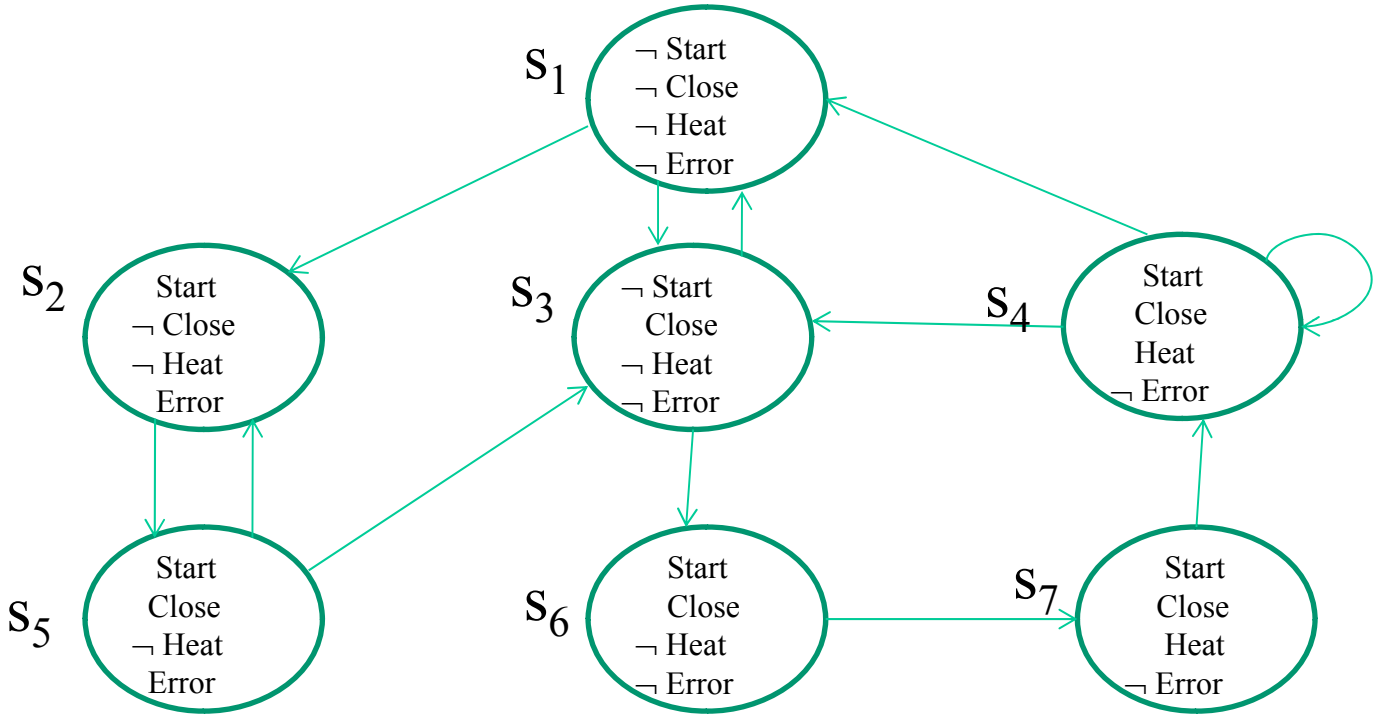
$(start \wedge (\neg\ error))$ $S3=\{s_4, s_7, s_6\}$

$AF\ (heat \wedge (\neg\ error))$ $S2=\{s_4, s_7, s_6\}$

$(start \wedge (\neg\ error)) \rightarrow\ AF\ (heat \wedge (\neg\ error))$ $S4=\{s_4, s_7, s_6, s_1, s_3, s_2, s_5\}$

Model Checking for the conditions

$AG((start \wedge (\neg\ error)) \rightarrow\ AF\ (heat \wedge (\neg\ error)))$

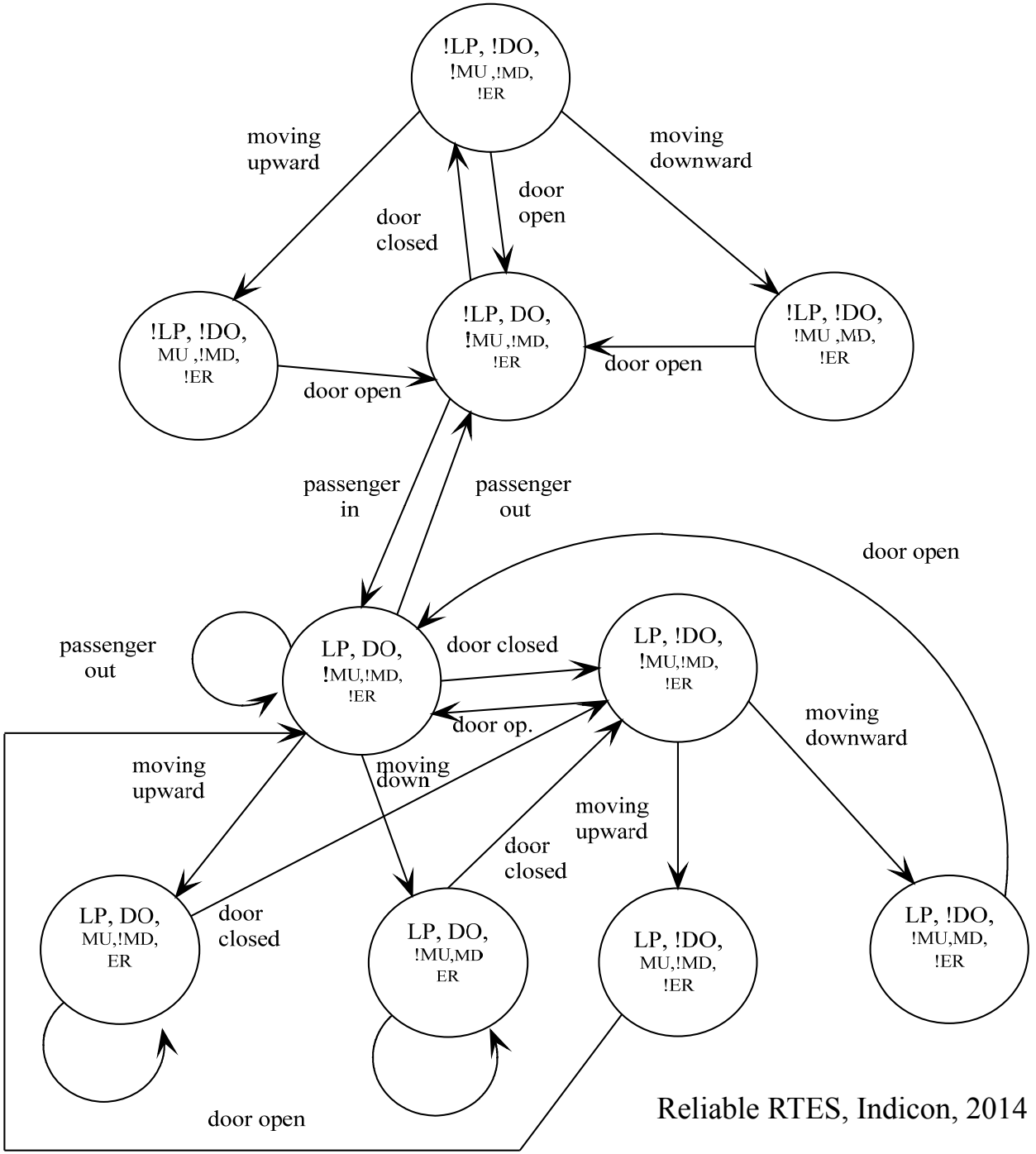


$S5=\{s_4, s_7, s_6, s_1, s_3, s_2, s_5\}$

$AG((start \wedge (\neg\ error)) \rightarrow\ AF\ (heat \wedge (\neg\ error)))$

True in all states

Example 2: Lift Control



Example 2: Lift Control

MU: elevator is moving in the upward direction.
MD: elevator is moving in the downward direction.
DO: door is open.
LP: elevator is loaded with passengers,
ER: some error occurred.

Model Checking for the conditions

- Specification:
  - **If the lift is moving then in the next state it will stop at somewhere and the door will open**.

(MU MD ~ER) → AX(DO ~MU ~MD ~ER).

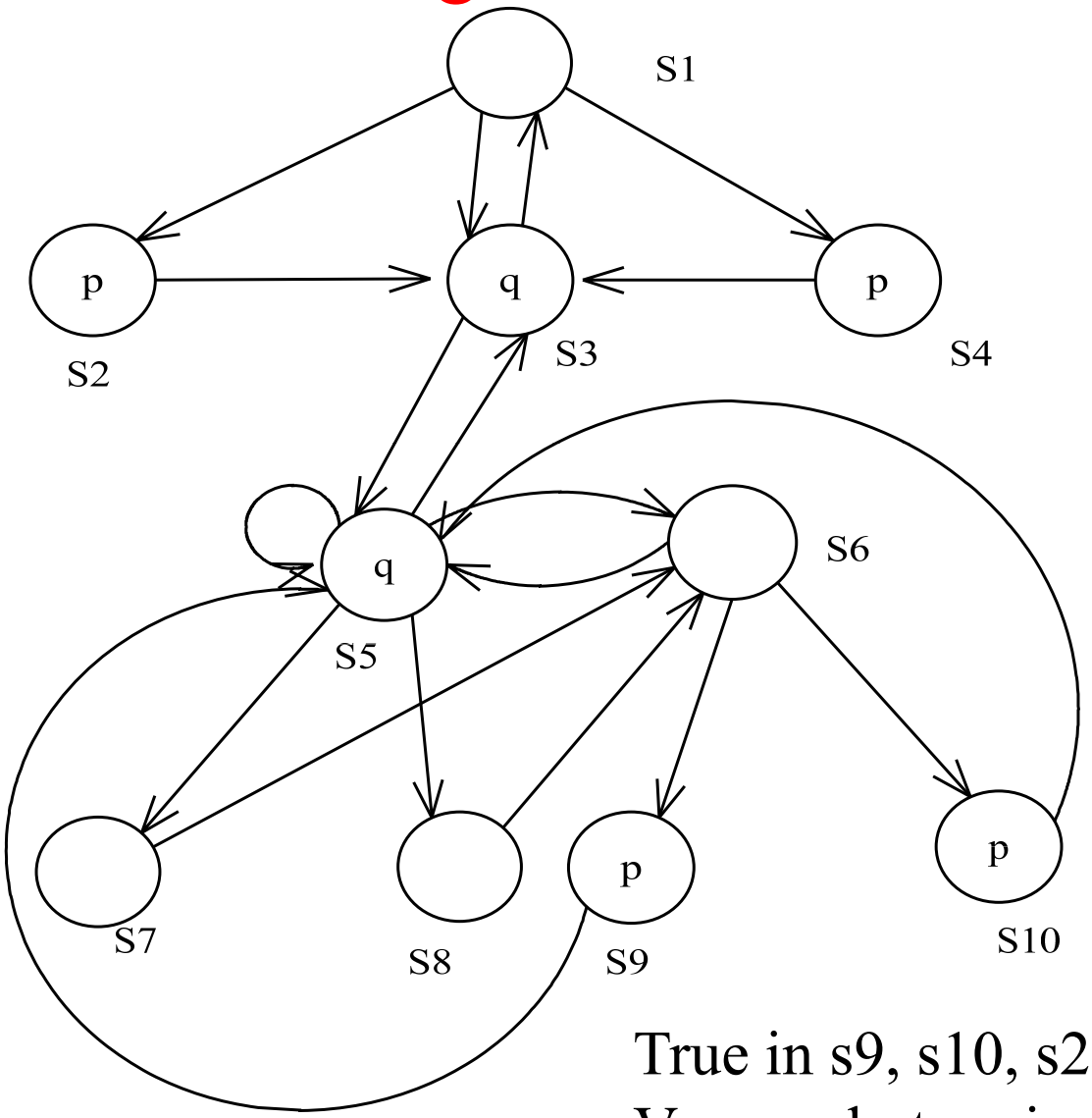
Model Checking for the conditions

$(MU \ \ MD \ \ \sim ER) \rightarrow AX(DO \ \ \sim MU \ \ \sim MD \ \ \sim ER)$.

Let p=(MU MD ~ER), so SAT(p)={S2,S4,S9,S10}.

q=(DO ~MU ~MD ~ER) So SAT(q) is {S3,S5}.

Model Checking for the conditions



True in s9, s10, s2, s4
Vacuously true in all other states

Model Checking for the conditions

- Specification:
  - Always it is possible to reach the state where the lift is static, doors are closed and waiting for the passenger.

p=EF(~DO ~LP ~MU ~MD ~ER)

Model Checking for the conditions

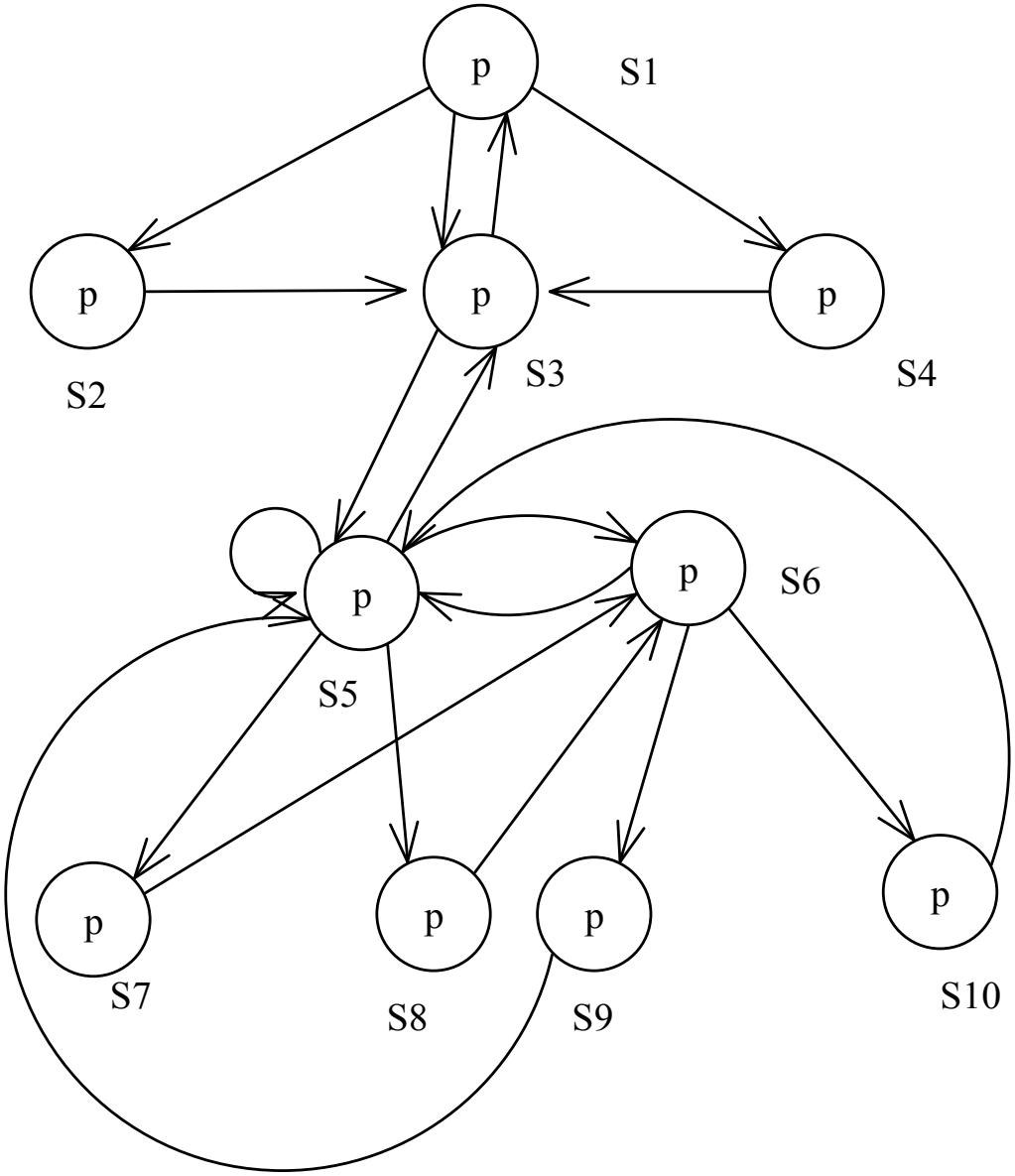
- Here, Y = SAT((~DO ~LP ~MU ~MD ~ER)= {S1}
- Loop 1
  - X = Y = {S1}
  - Y = Y {S3} = {S1,S3}
- Loop 2
  - X = Y = {S1,S3}
  - Y = Y {S1,S3,S2,S4,S5} = {S1,S2,S3,S4,S5}

Model Checking for the conditions

- Loop 3
  - X = Y = {S1,S2,S3,S4,S5}
  - Y = Y { S1,S2,S3,S4,S5,S6,S9,S10} = { S1,S2,S3,S4,S5, S6,S9,S10}
- Loop 4 -
  - X = Y = { S1,S2,S3,S4,S5, S6,S9,S10}
  - Y = Y {S1,S2,S3,S4,S5,S6,S9,S10,S7,S8} = {S1,S2,S3,S4,S5,S6, S7,S8,S9,S10}

- So we have EF(~DO ~LP ~MU ~MD ~ER) = {S1,S2,S3,S4,S5,S6, S7,S8,S9,S10}. Let p= EF(~DO ~LP ~MU ~MD ~ER). So, p is satisfied in every state of the model

Model Checking for the conditions



What about RT ES ?
Quantitative Timings!!!
RTCTL and PRTCTL logic

Characteristics of RTCTL logic

What can RTCTL be applied to?

RTCTL logic may be applied to specify time dependencies not only qualitatively (like in CTL), but also quantitatively:

- To give a maximal time distance between two propositions on every /some path of states.

  - Globally in future p holds, within k time units $AF^{\leq k} p$

Syntax and Semantics of RTCTL logic

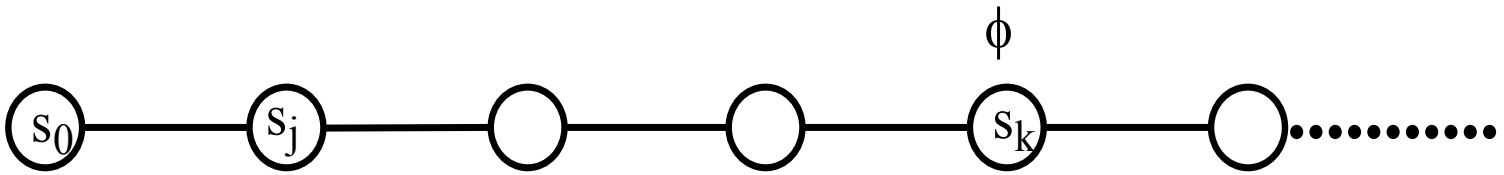
- Time structure (Kripke structure) of RTCTL
- Time bounds for temporal operators

RTTCL Temporal Operator

Future : $F^{\leq l}\ \varphi$

$(M, S_j) \models F^{\leq l}\ \varphi \Leftrightarrow \exists k, k \geq j, k\text{-}j \leq l, (M, S_k) \models \varphi$

state S_j satisfies as future state S_k satisfies φ and S_k lies within l (in this example 3) time units of S_j.



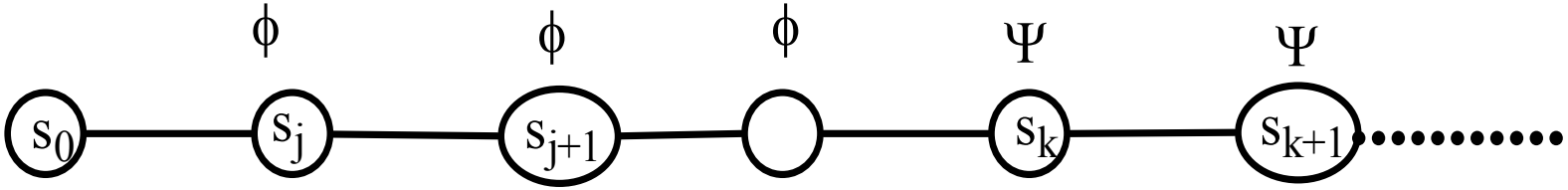
RTCTL Temporal Operator

Until : (Ψ $U^{\leq l}$ φ)

$(M, S_j) \models \Psi \cup \varphi \Leftrightarrow \exists k, 0 \leq k (M, S_k) \models \varphi$ and

$\forall k, j < k \, (M, S_j) \models \Psi$ where $k\text{-}j \leq l$

S_j satisfies (Ψ $U^{\leq 3}$ φ) because Ψ is true for all states S_i $j<=i<k$ and then φ is true for state S_k .



Some Tools

•NuSMV Model checker based on BDDs

•UPPAAL is an integrated tool environment for modeling, validation and verification of real-time systems modeled as networks of timed automata, extended with data types (bounded integers, arrays etc.).

•Ptolemy: A framework for simulating and prototyping heterogeneous systems,

References

•Lam, William K., "Hardware Design Verification: Simulation and Formal Method-Based Approaches, 1st Edition, Prentice Hall PTR, 2008.

•Fujita, Masahiro, Ghosh, Indradeep and Prasad, Mukul, "Verification Techniques for System-Level Design, 1st Edition, Morgan Kaufmann Publishers Inc, 2007

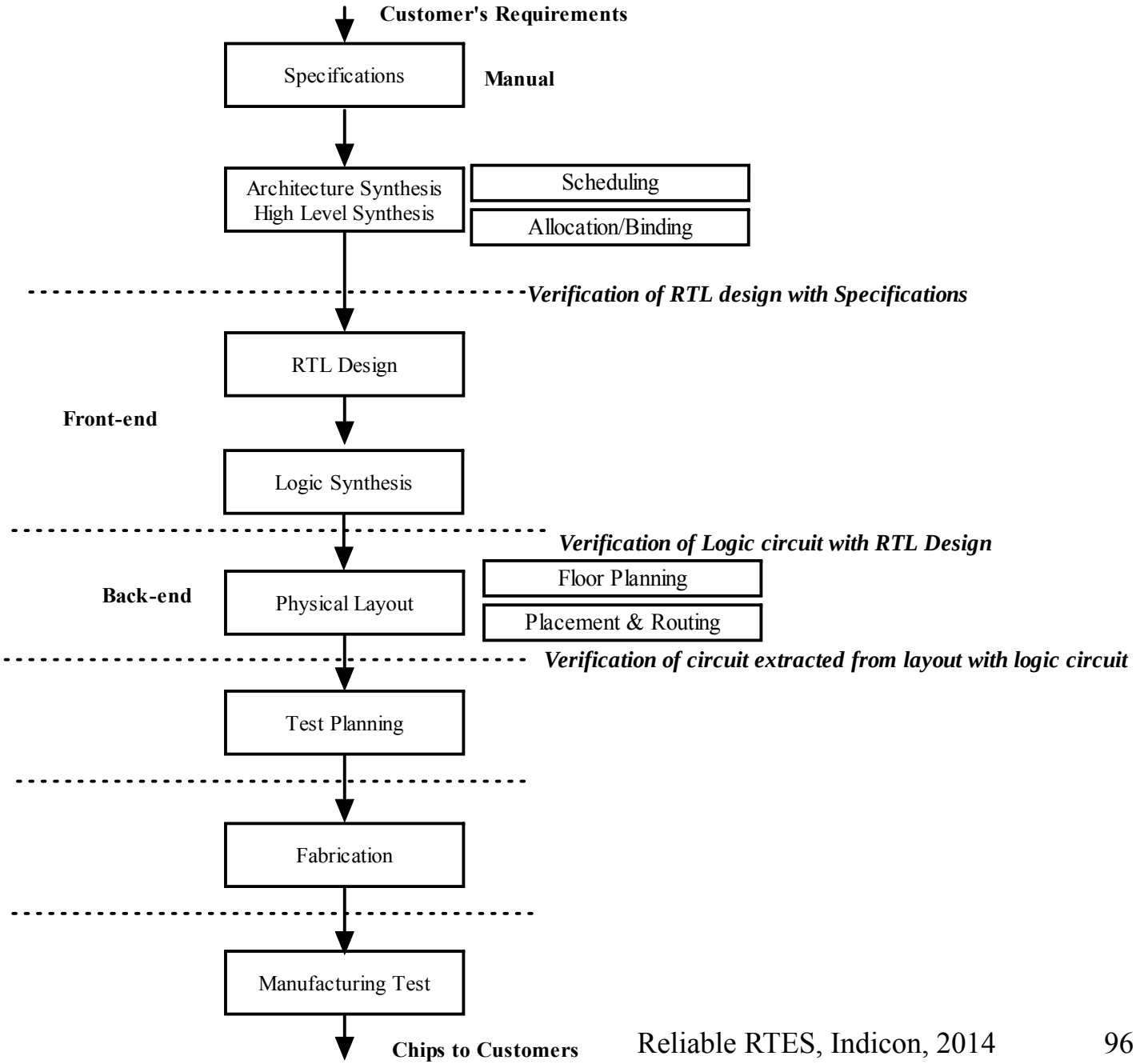
•Michael Huth and Mark Ryan, "Logic in Computer Science: Modelling and Reasoning about Systems", 2nd edition, Cambridge University Press, New York, NY, USA

- E.A. Emerson , "Temporal and modal logic", 1995
- E.A. Emerson et al. "Quantitative temporal reasoning", 1992
- E.A. Emerson et al. "Parametric Quantitative temporal Reasoning", 1999

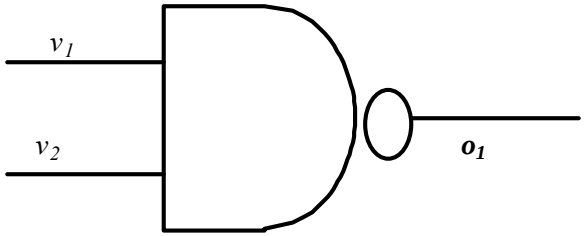
•Santosh Biswas and Jatindra K Deka, NPTEL Video lectures and Web course VLSI Design Verification and Test (http://www.nptel.ac.in/courses/106103116/)

Testing of RT Embedded Systems

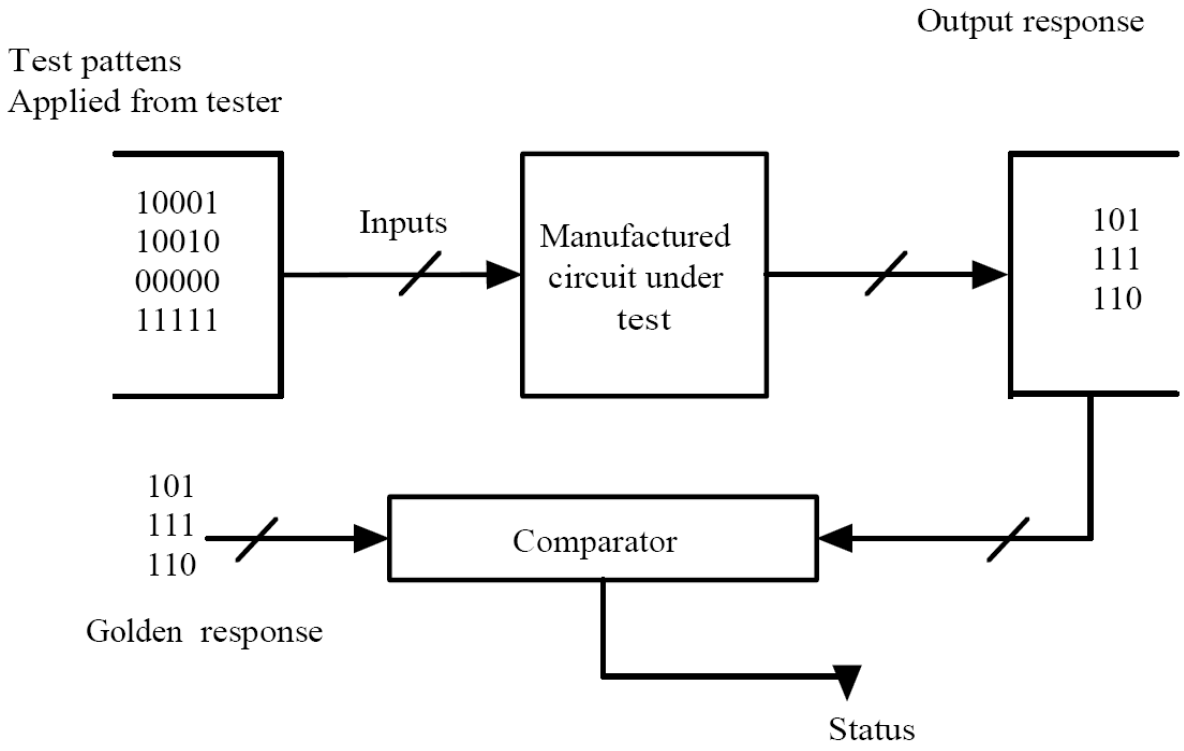
Hardware Design and Test flow



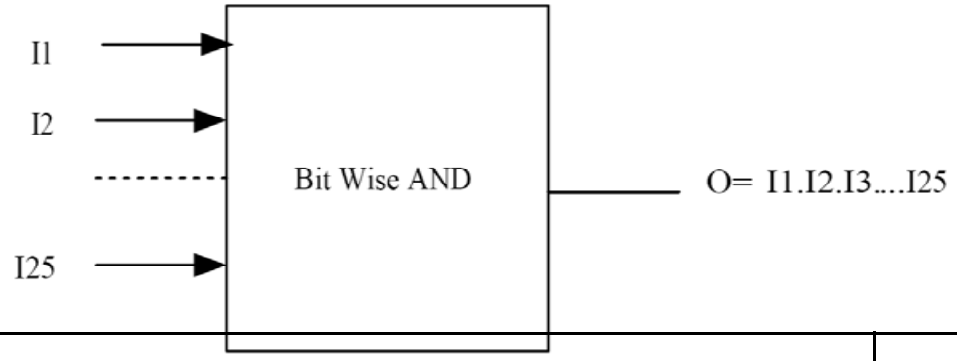
What is testing ?



Input		Output
v_1	v_2	o_1
0	0	1
0	1	1
1	0	1
1	1	0

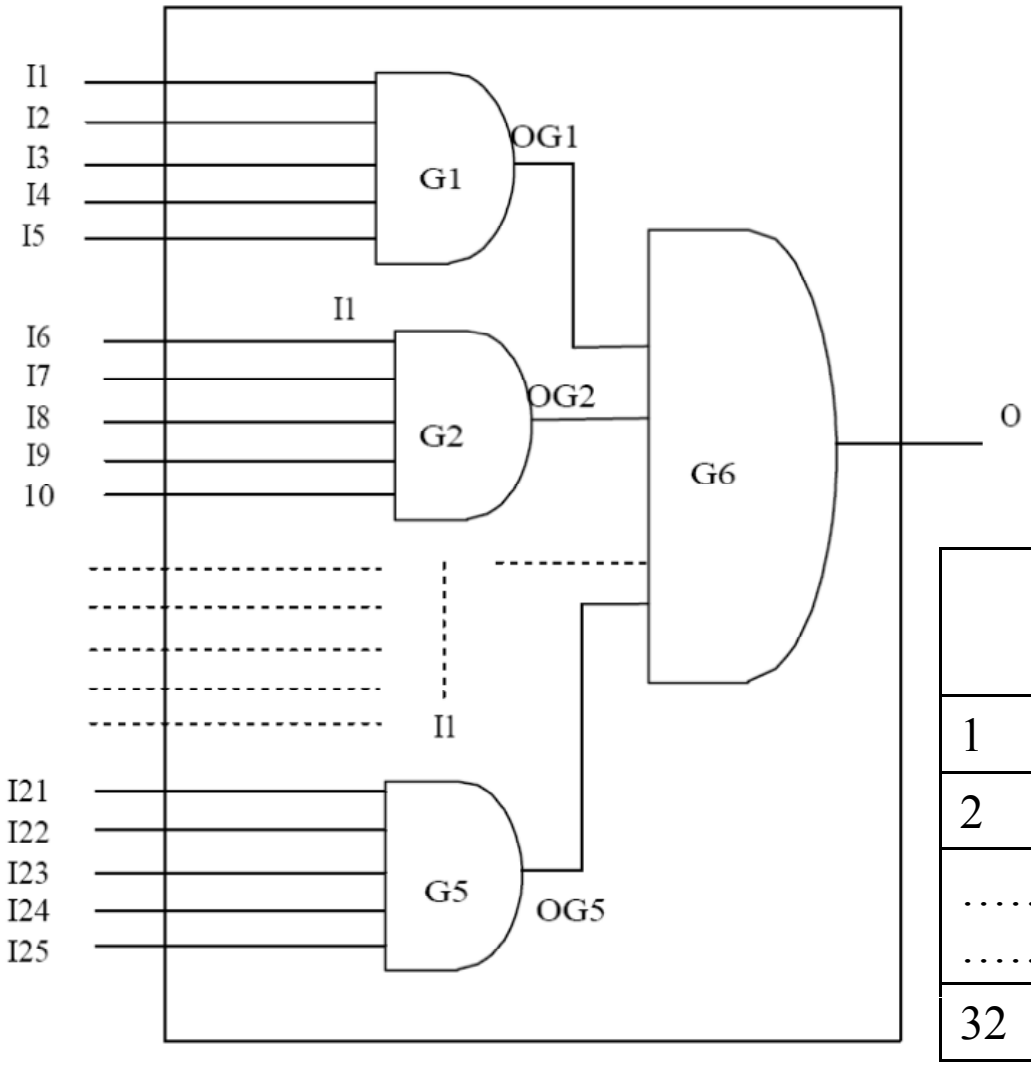


Is Functional Testing Possible



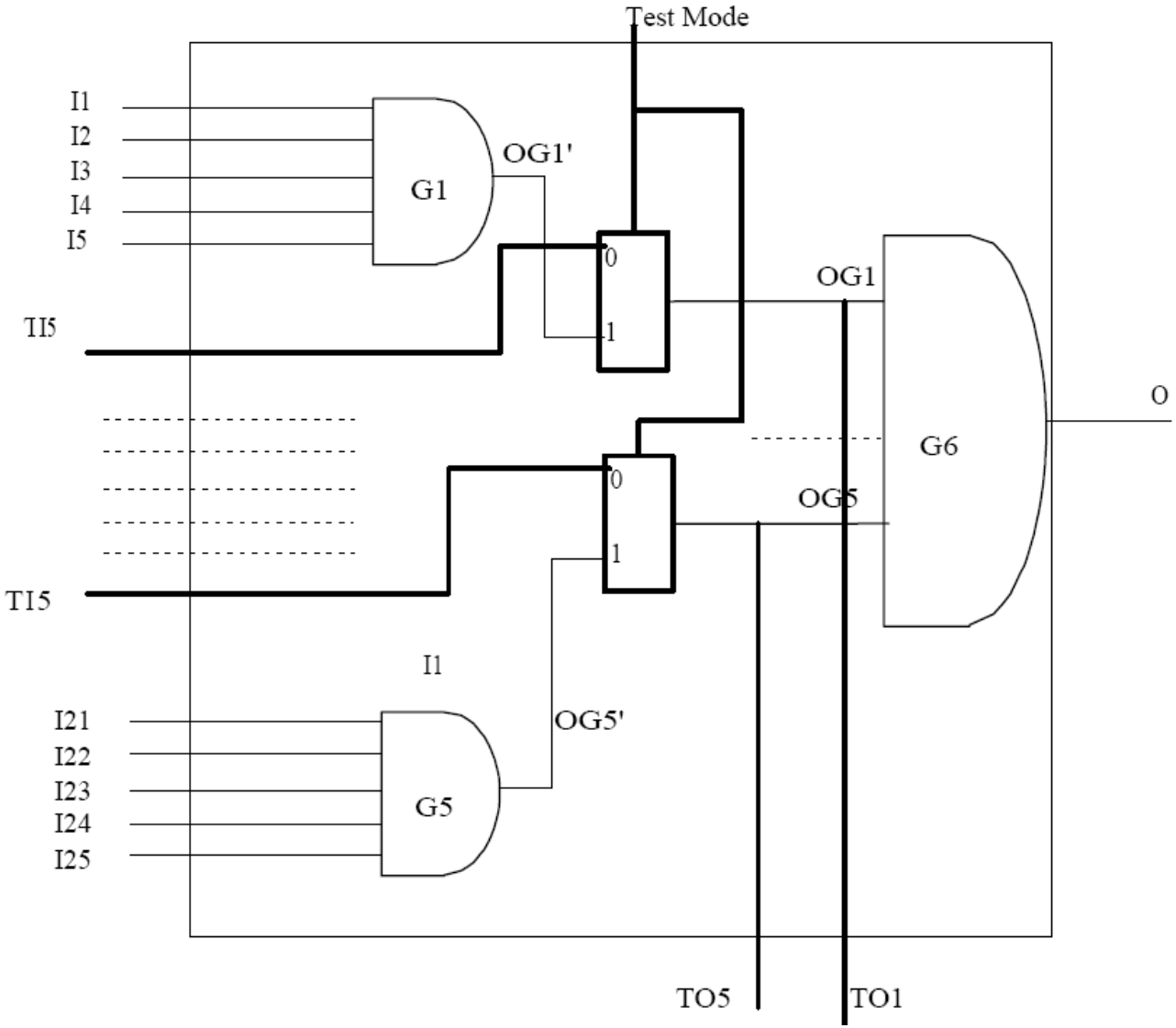
Test Pattern No.	Test Pattern	Output
1	00000000000000000000000000	0
2	00000000000000000000000001	0
……. …	……. …….	0
2^{25}	11111111111111111111111111	1

Is Functional Testing Possible

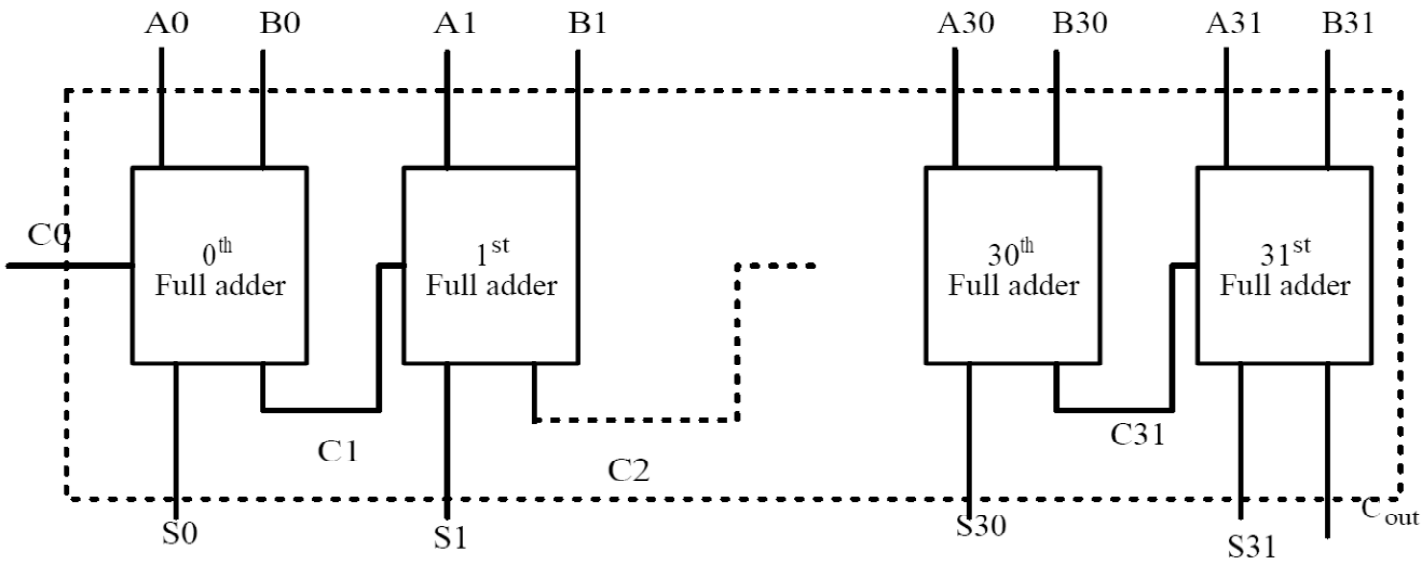


	Test Pattern (I1,I2,I3,I4,I5)	Output (OG1)
1	00000	0
2	00001	0
....	....	0
32	11111	1

Is Functional Testing Possible

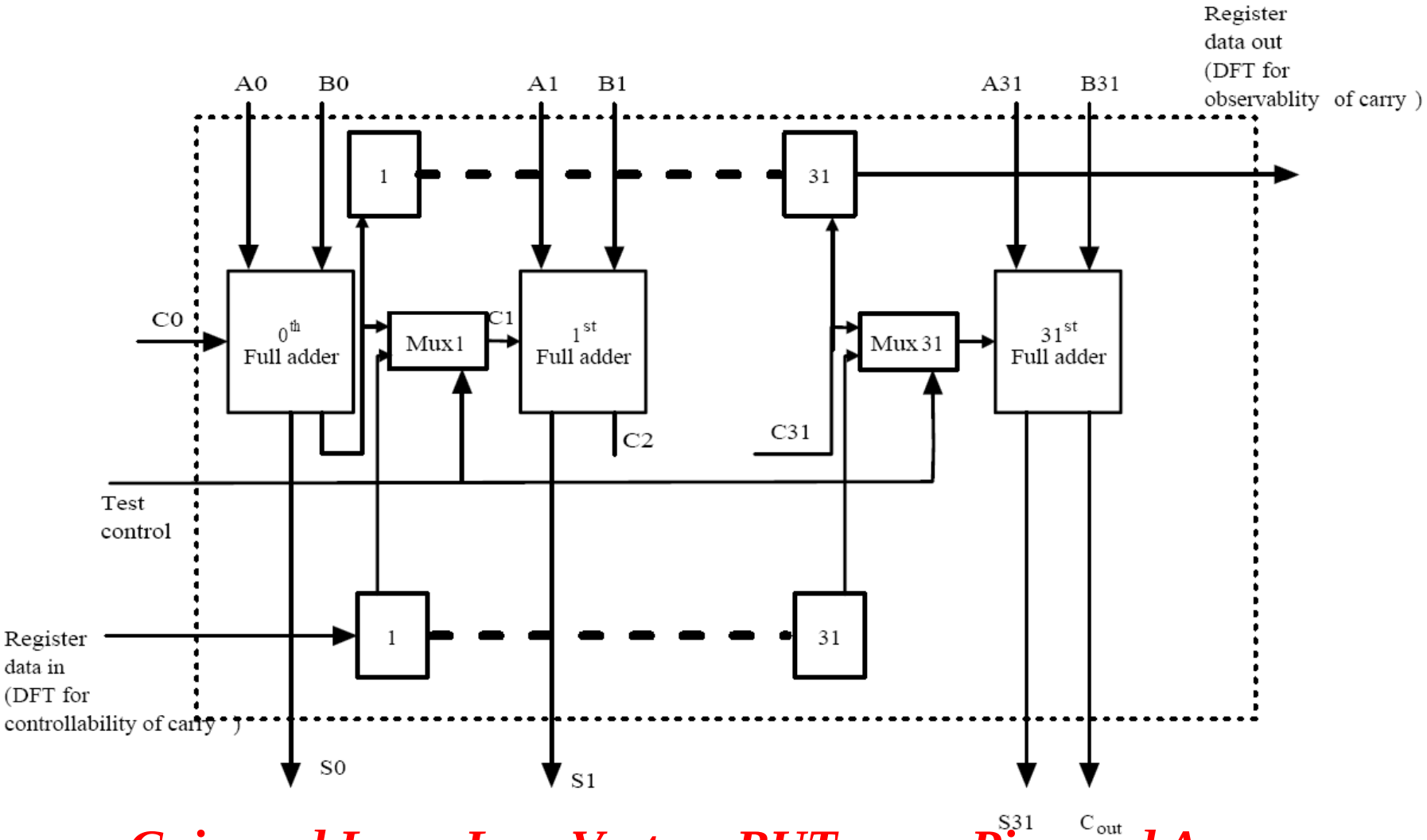


An example of Structural Testing



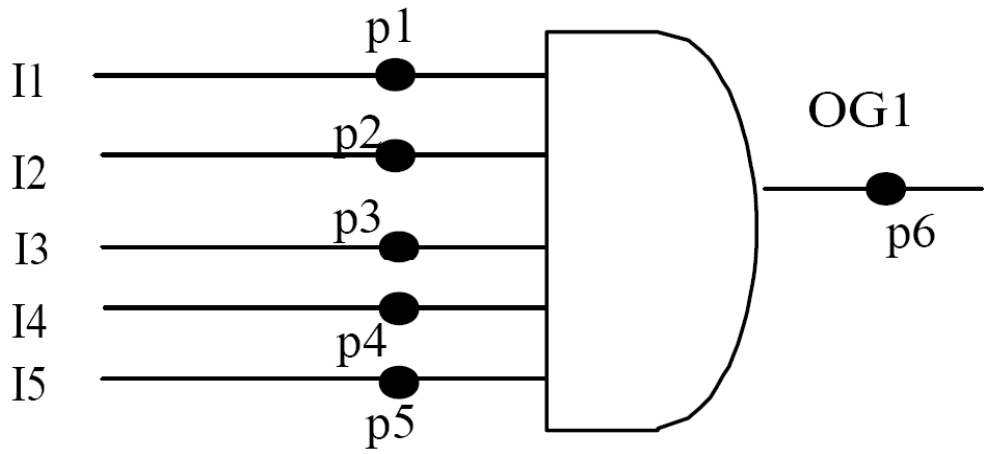
No. of Test Vectors– 2^64

An example of Structural Testing



Gain and Loss: Less Vectors BUT more Pins and Area

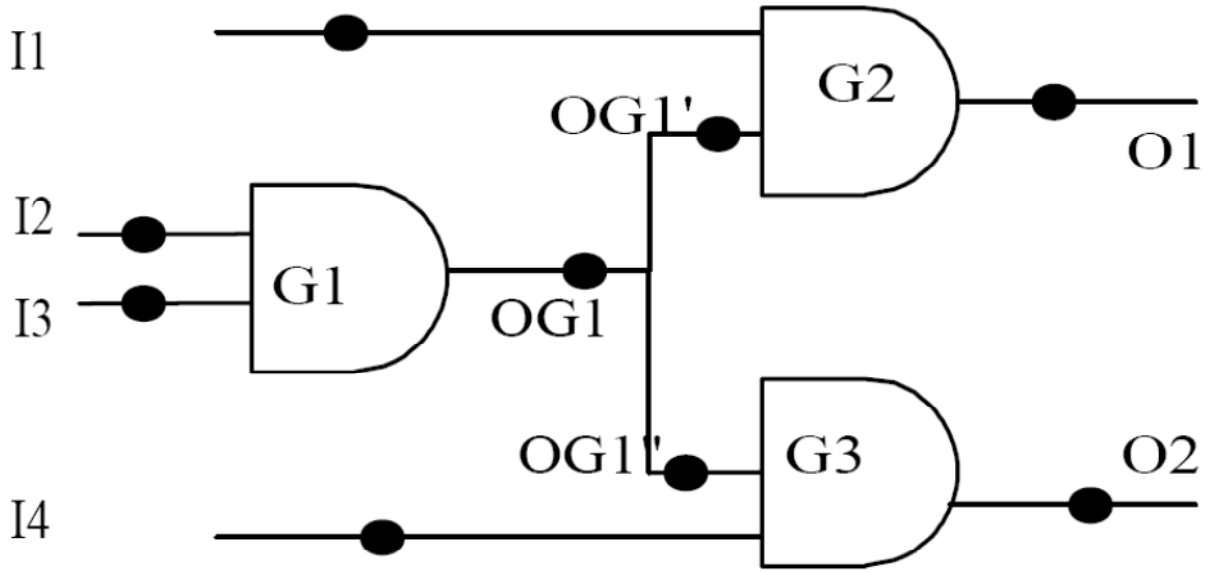
Structural Testing and Fault Model



Each net can have three states: normal, stuck-at-1 and stuck-at-0

Verify No faults

Stuck-at fault locations in a circuit with fanouts

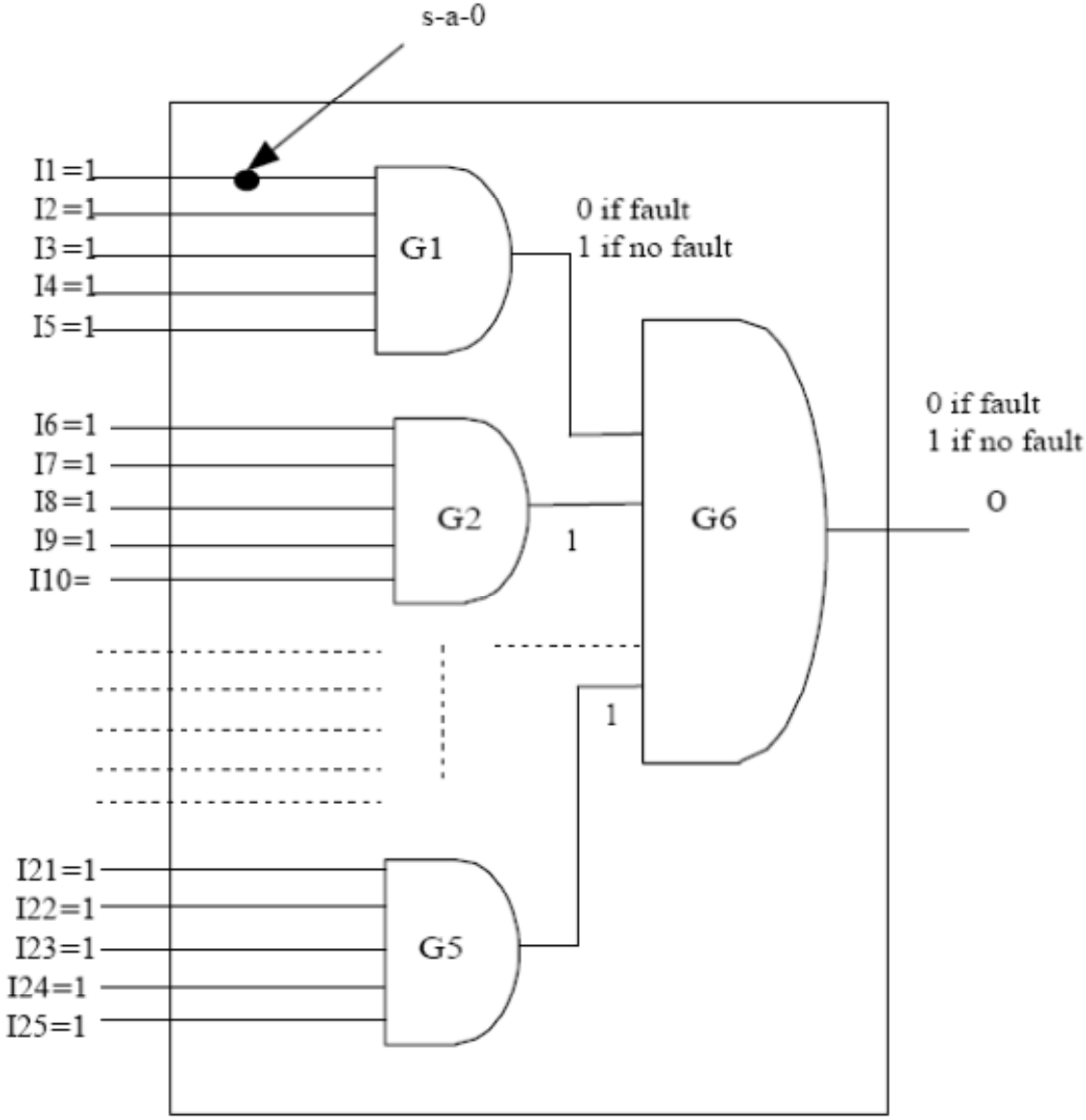


S-a Fault Model

Single stuck-at fault model is characterized by three assumptions:

- Only one net is faulty at a time.
- The faulty net is permanently set to either 0 or 1.
- The branches of a fanout net are independent with respect to locations and affect of a stuck-at fault.

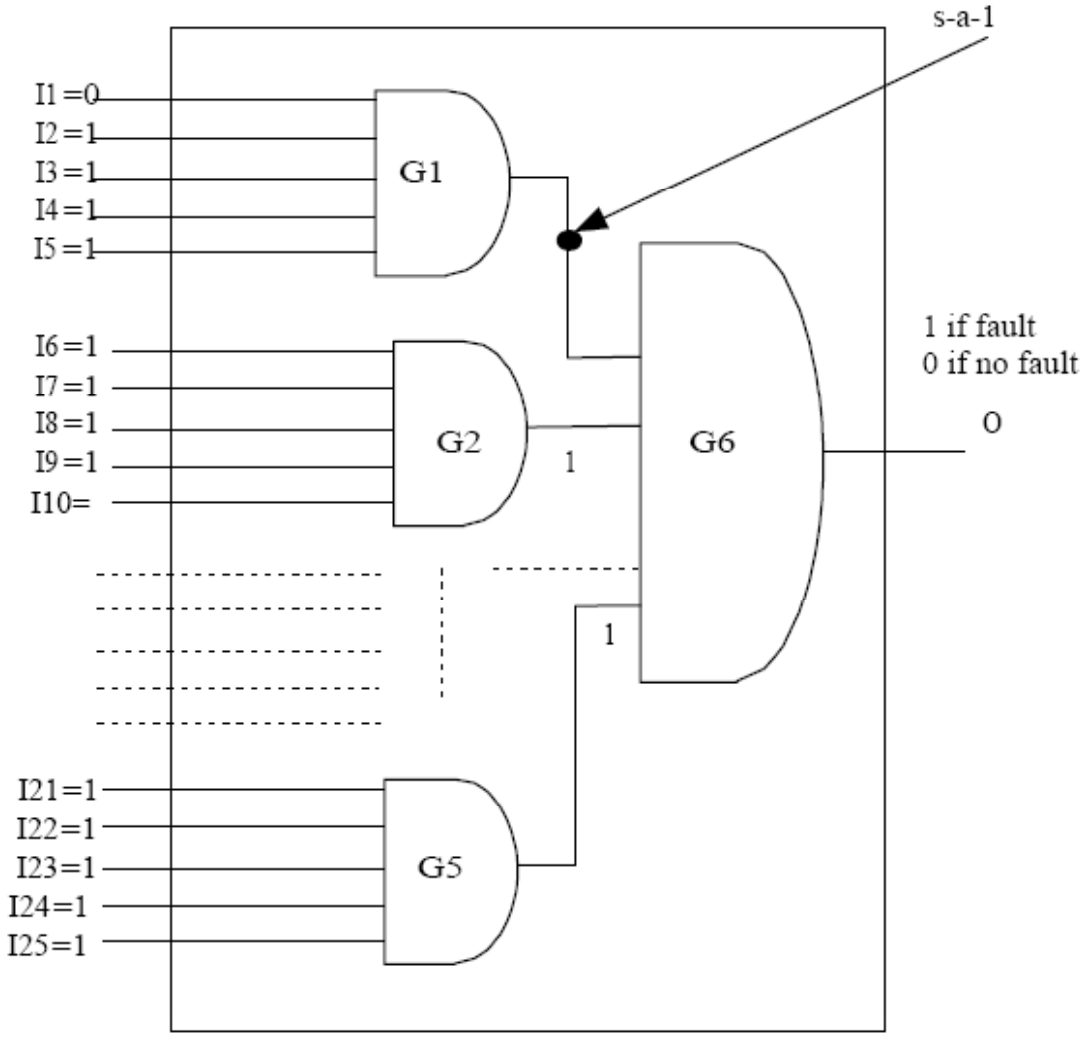
Example: Testing using s-a fault



Comparison of structural and functional testing

Functional testing	Structural Testing
Without fault models.	With fault models.
Manually generated design verification test patterns.	Automatic test pattern generation (ATPG).
Slow and labor intensive.	Efficient and automated.
Fault coverage not known	Fault Coverage is a quantified metric.
More Test Patterns	Less Test Patterns
Can be applied at the operating speed.	Difficult to be applied at the speed the design is expected to work.

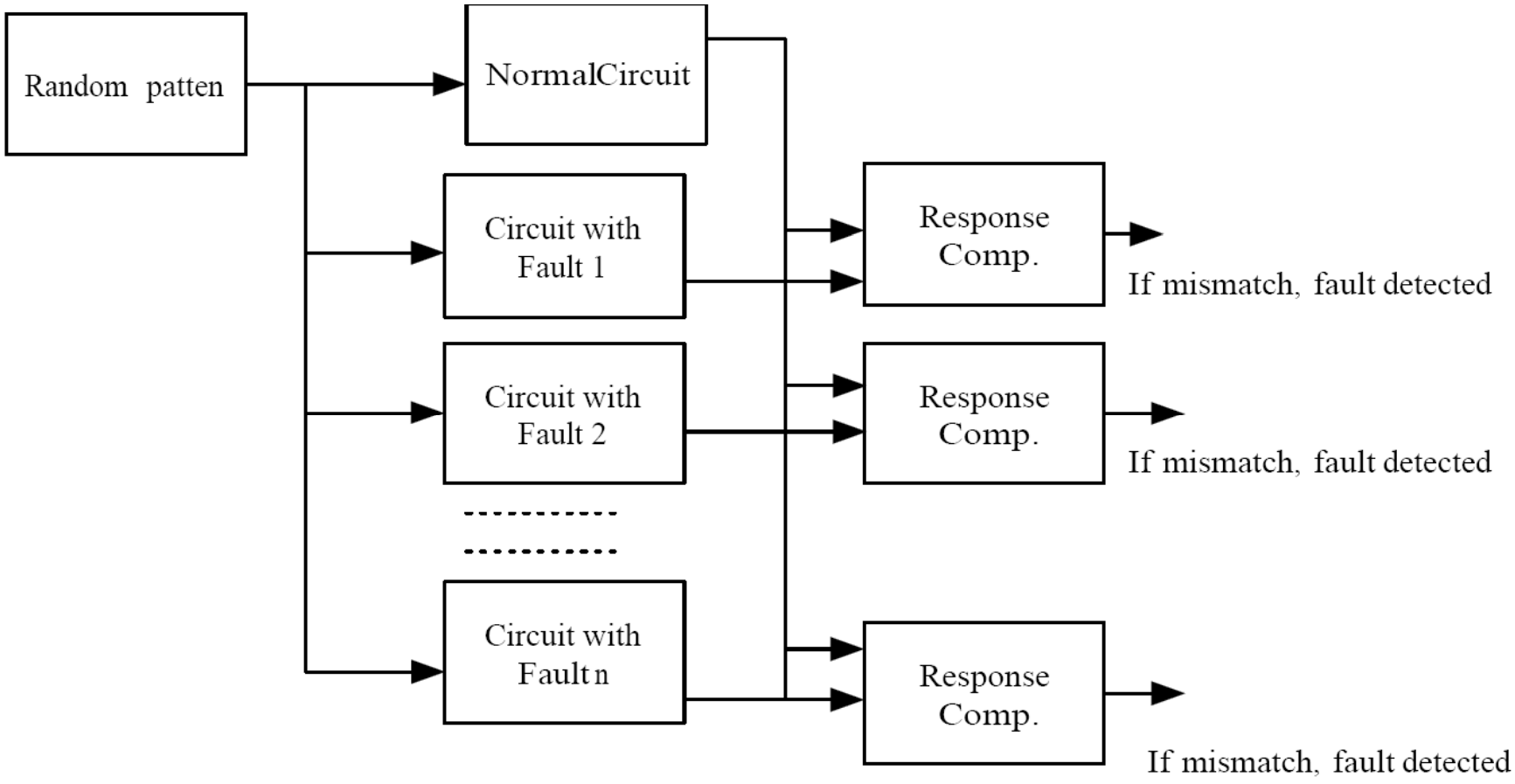
Automatic Test Pattern generation



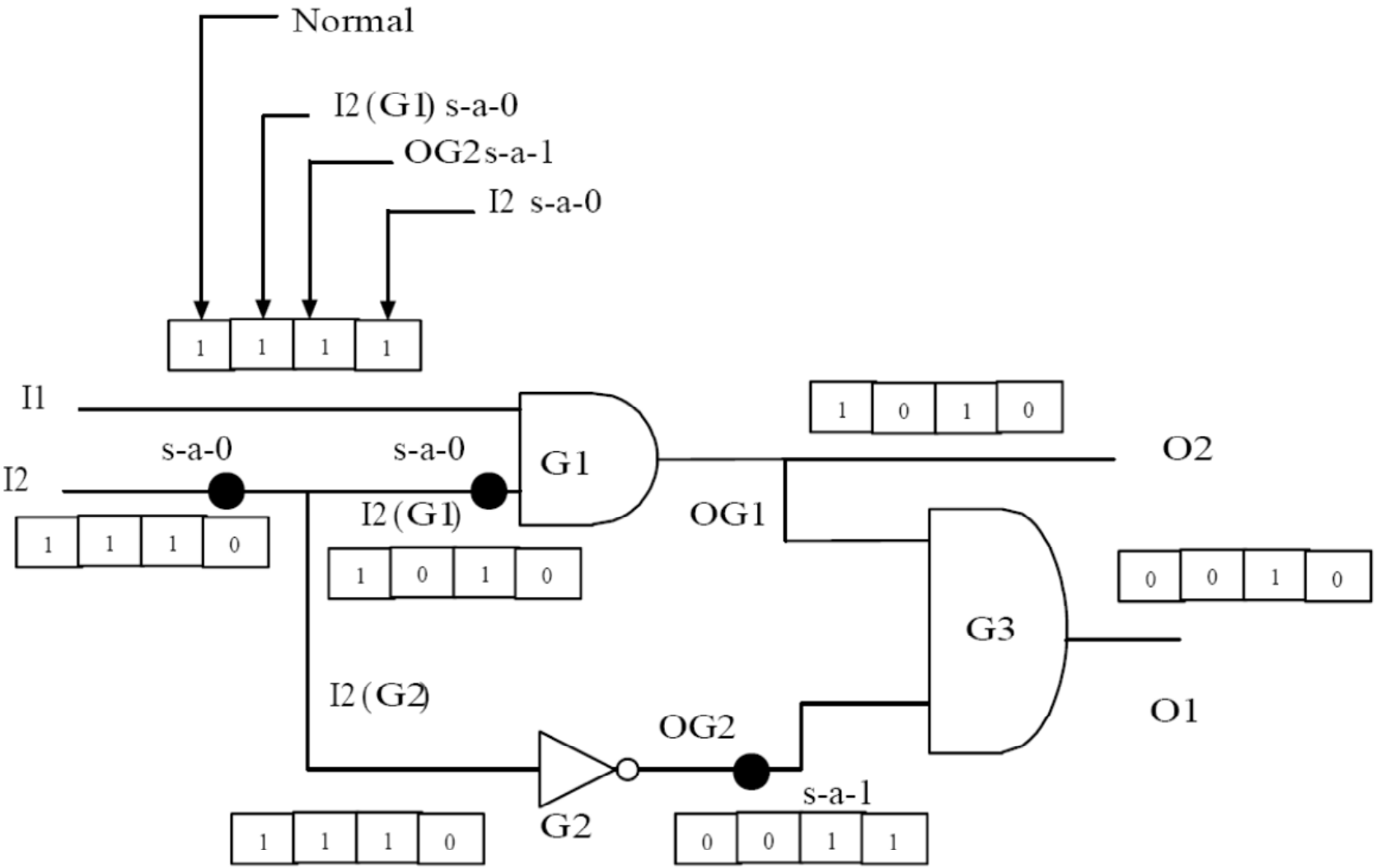
Random ATPG

Pattern No.	Random Pattern I1 I2 I3 I4 I5 I6……I25	Faults Detected
1	1 0 0 0 1 1111111111111111111	s-a-1 at net “output of G1” s-a-1 at net “output of G6”
2	1 1 1 1 1 1111111111111111111	s-a-0 faults in all the nets of the circuit

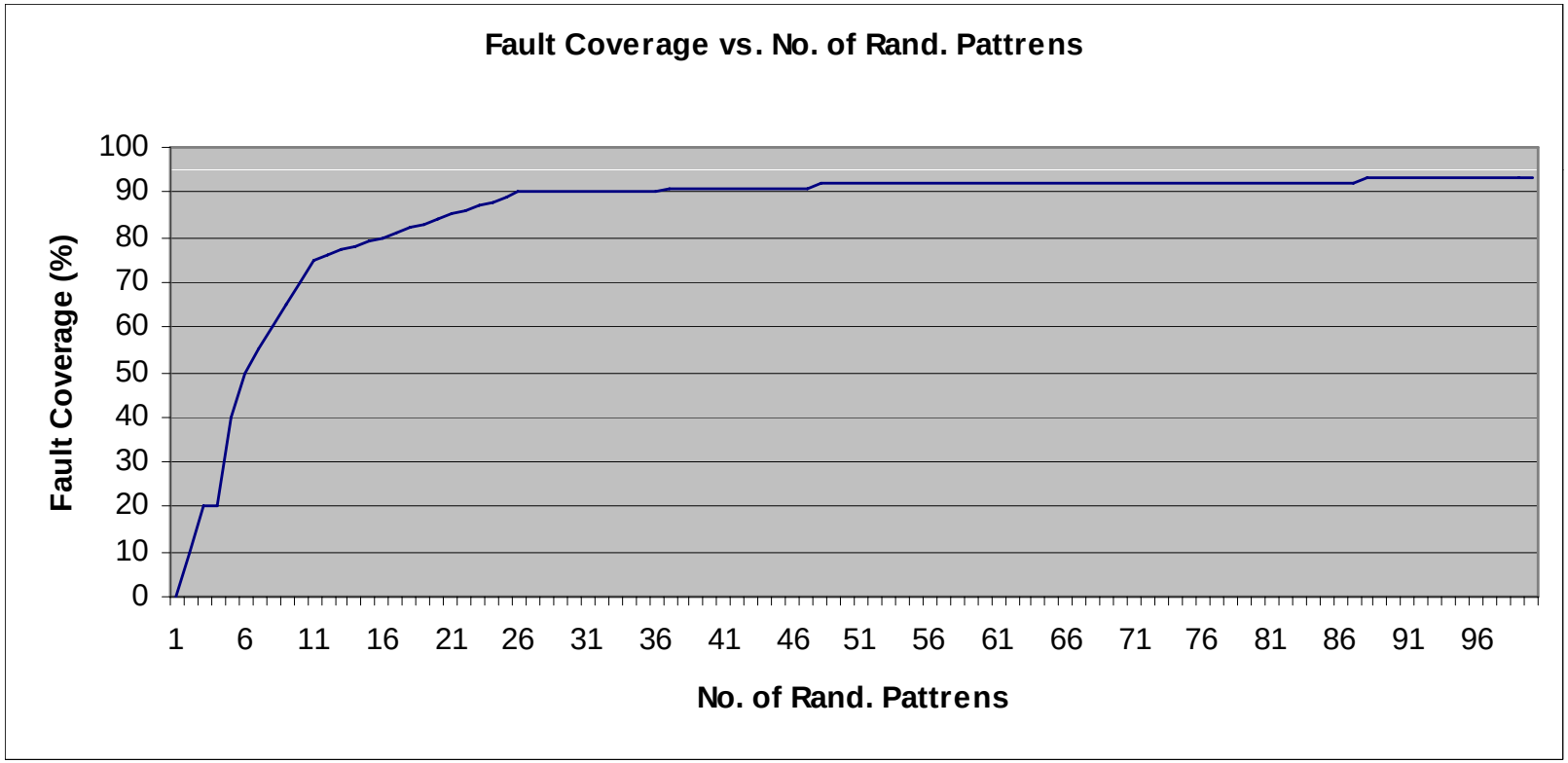
Random ATPG


Random patten
NormalCircuit
Circuit with Fault 1
Circuit with Fault 2
Circuit with Fault n
Response Comp.
Response Comp.
Response Comp.
If mismatch, fault detected
If mismatch, fault detected
If mismatch, fault detected

Random ATPG Example

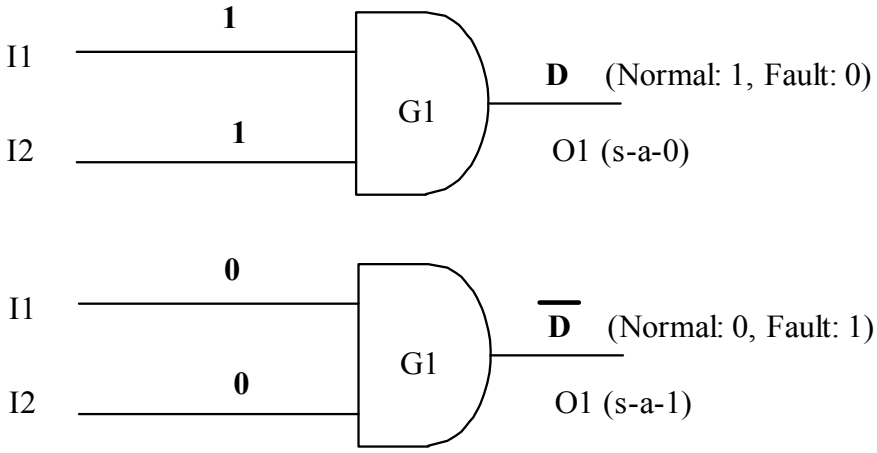


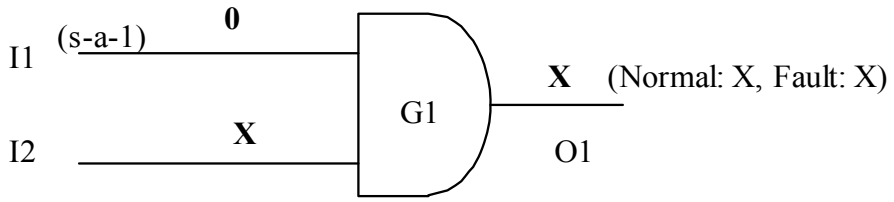
Random ATPG—How Long ?



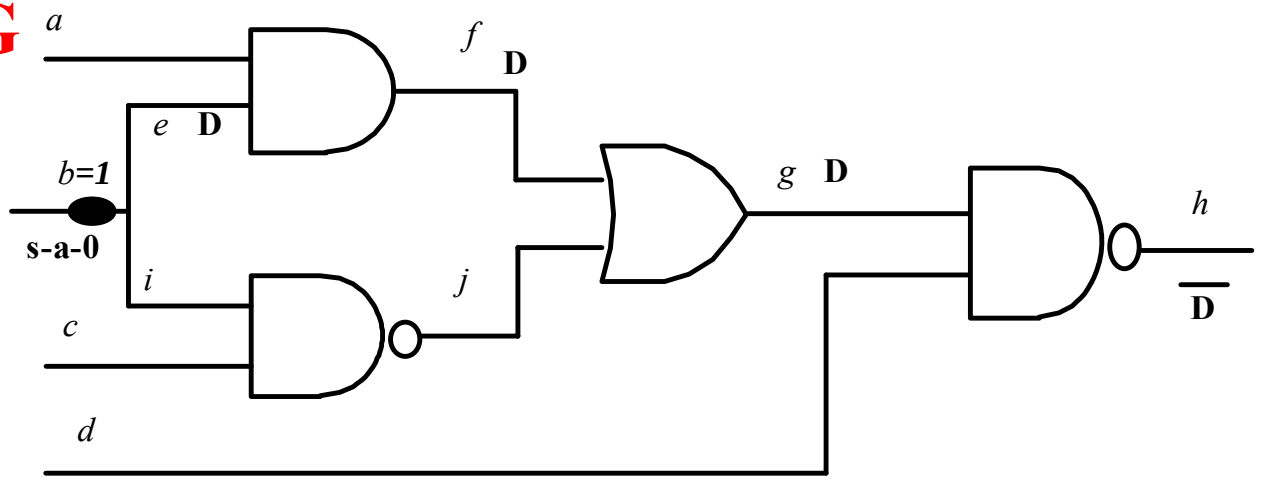
Deterministic ATPG Algebra

Symbol	Implication	Normal Circuit	Faulty Circuit
0	(0/0)	0	0
1	(1/1)	1	1
X	(X/X)	X	X
D	(1/0)	1	0
$\bar{D}$	(0/1)	0	1

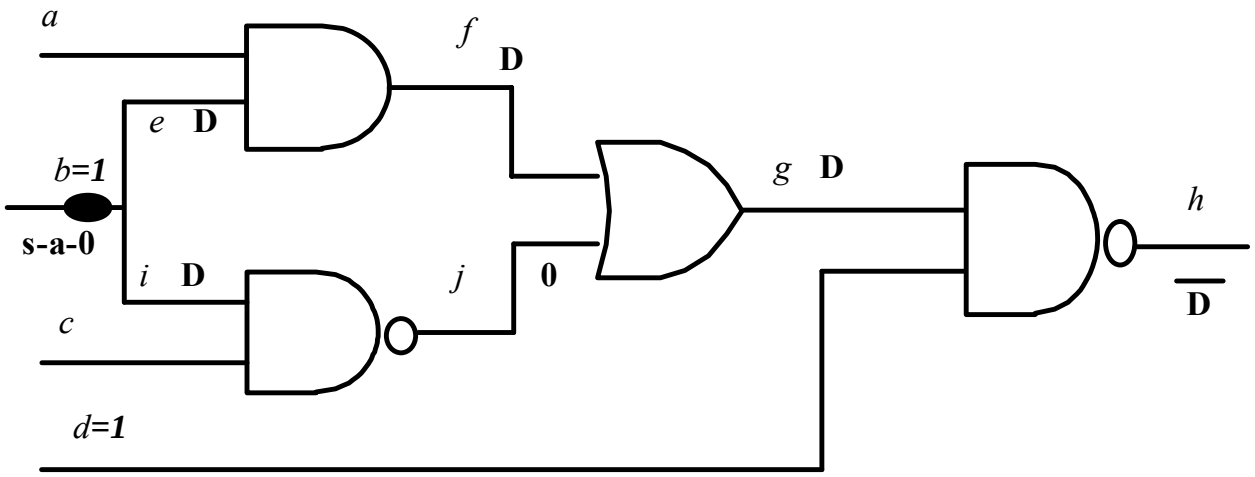




Deterministic ATPG

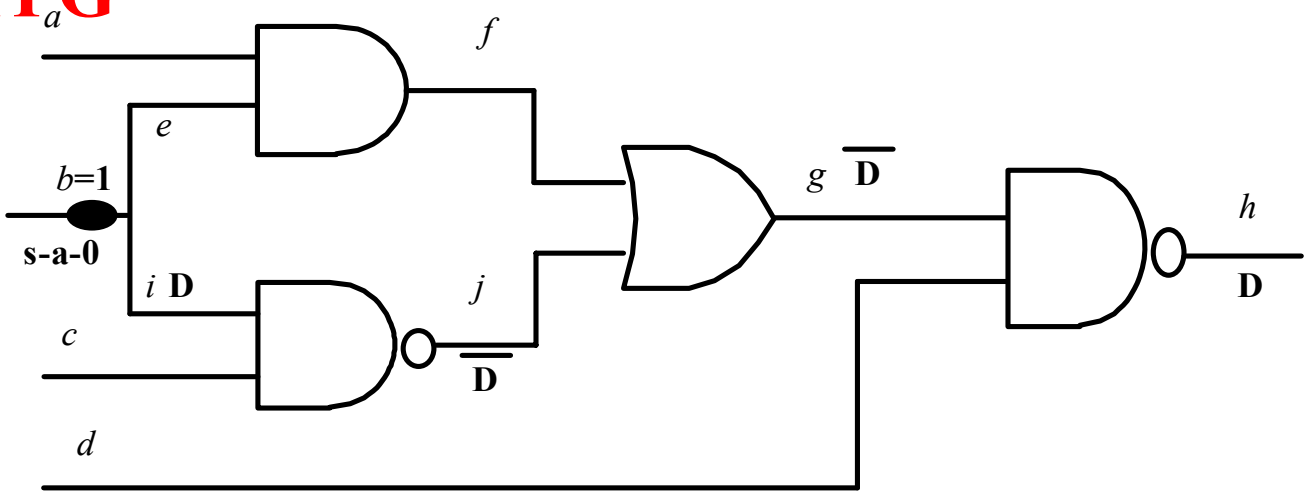


(a) Sensitization by *b*=1
Propagation by path e-f-g-h

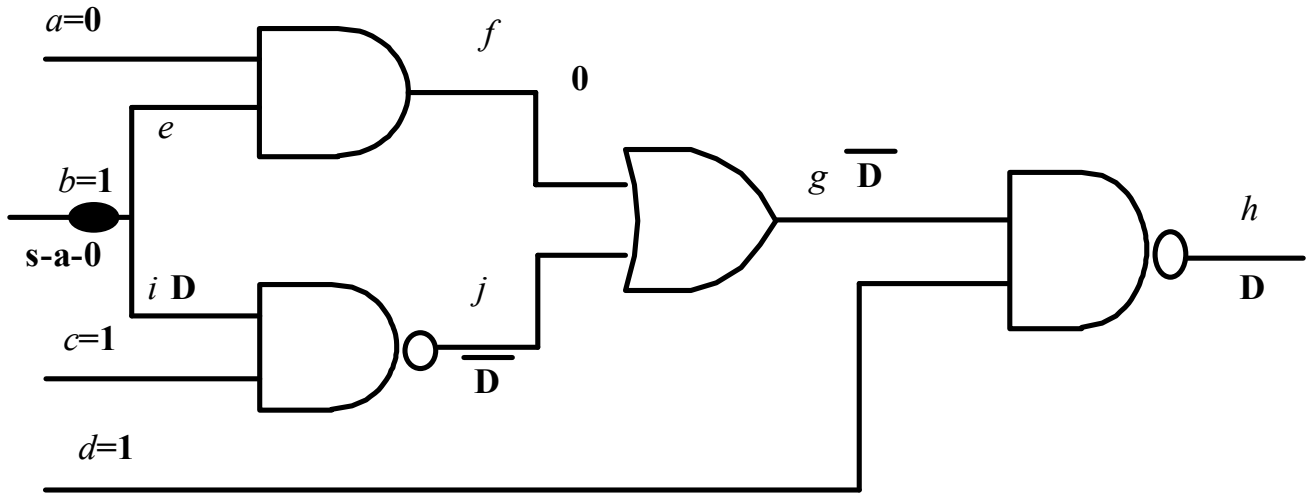


(b) Justification j=0
j*=D, *if c=1 and j=1 if c=0
(conflict at j)

Deterministic ATPG



(a) Sensitization by *b*=1
Propagation by path i-j-g-h

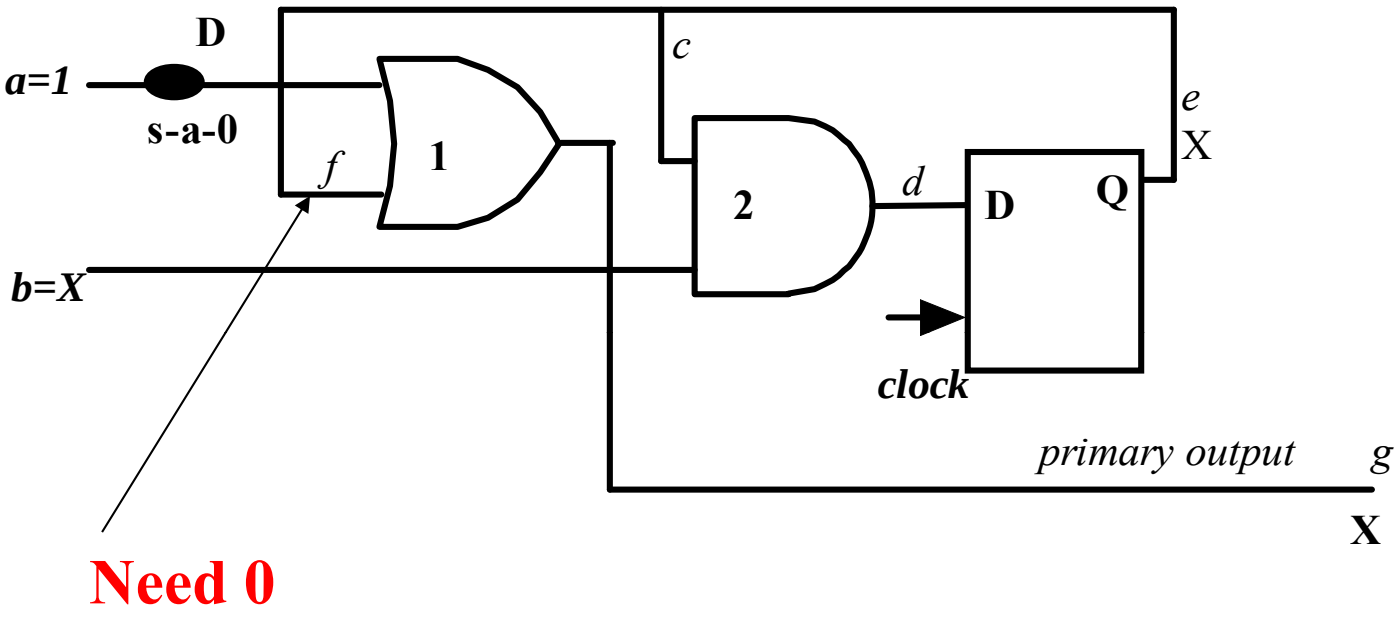


(b) Justification *d*=1,*c*=1,*f*=0,*a*=0
(Successful)

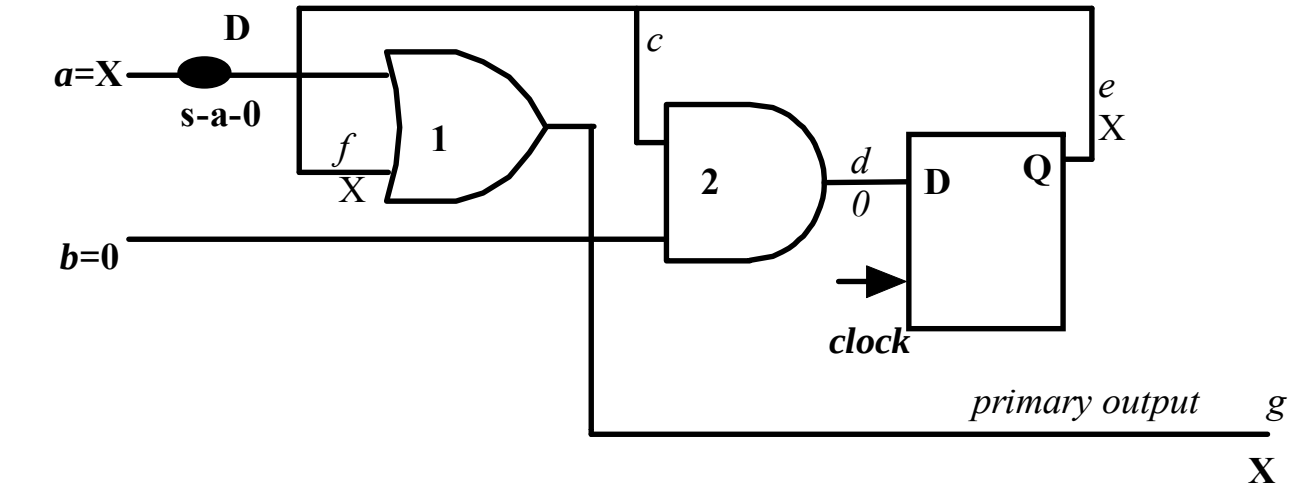
Deterministic ATPG steps

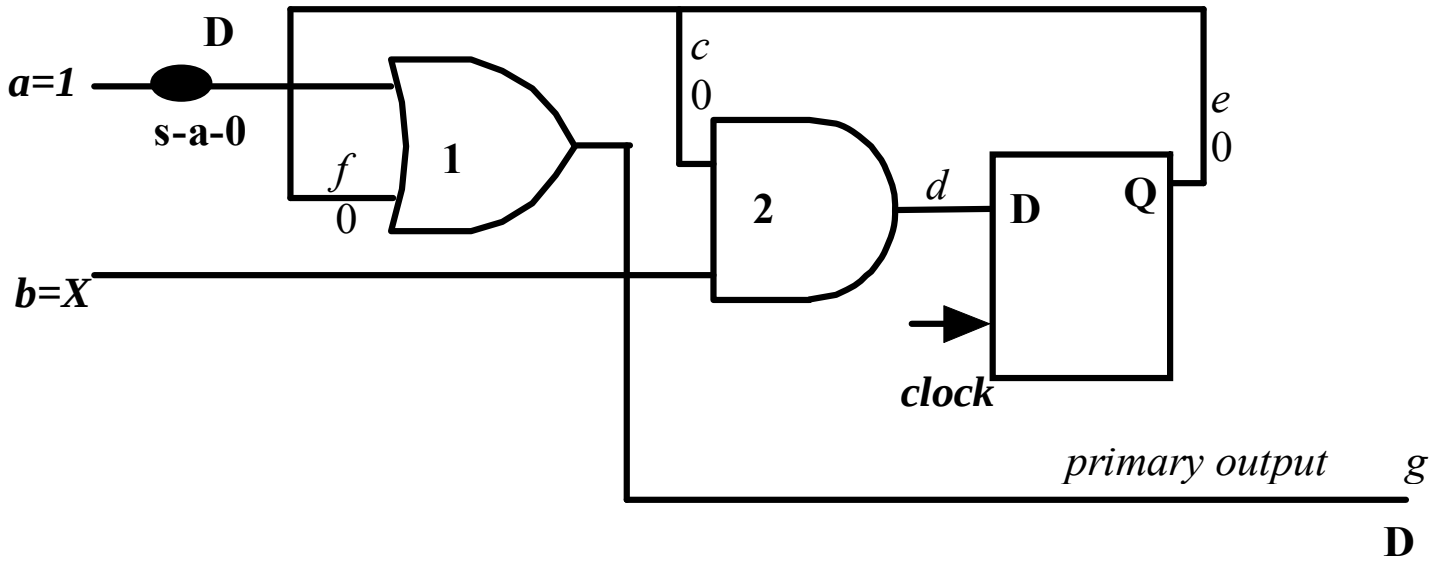
- **Fault sensitization:** In this step a stuck-at fault is activated by setting the signal driving the faulty net to an opposite value from the fault value.
- **Fault propagation**: In this step a path is selected from the fault site to some primary output, where the effect of the fault can be observed for its detection.
- **Line justification**: In this step the signals in (internal) nets or some primary inputs, which were assigned for fault sensitization/propagation, are justified by setting (remaining) primary inputs of the circuit.

How Complicated are Sequential Circuits ?

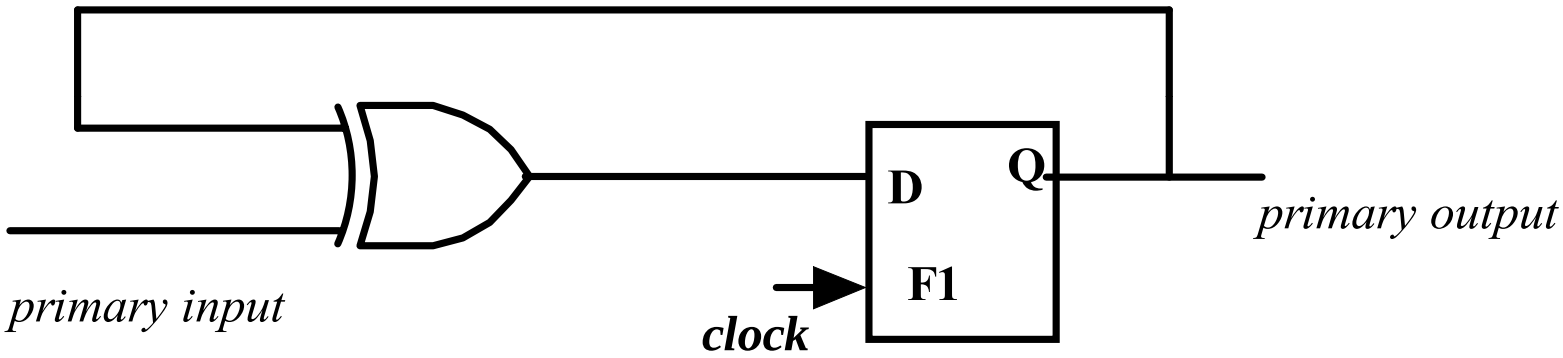


Indirect Control

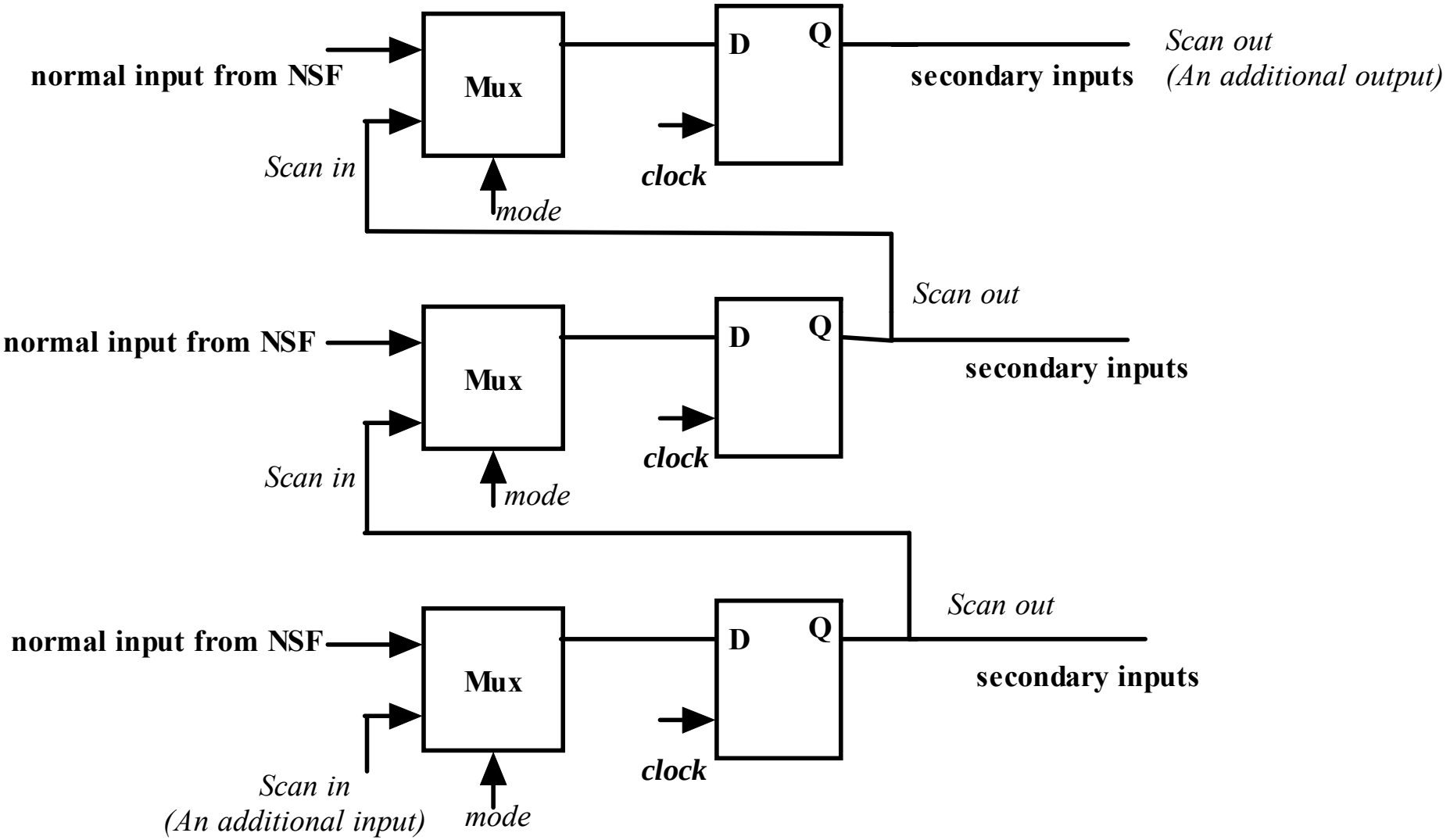




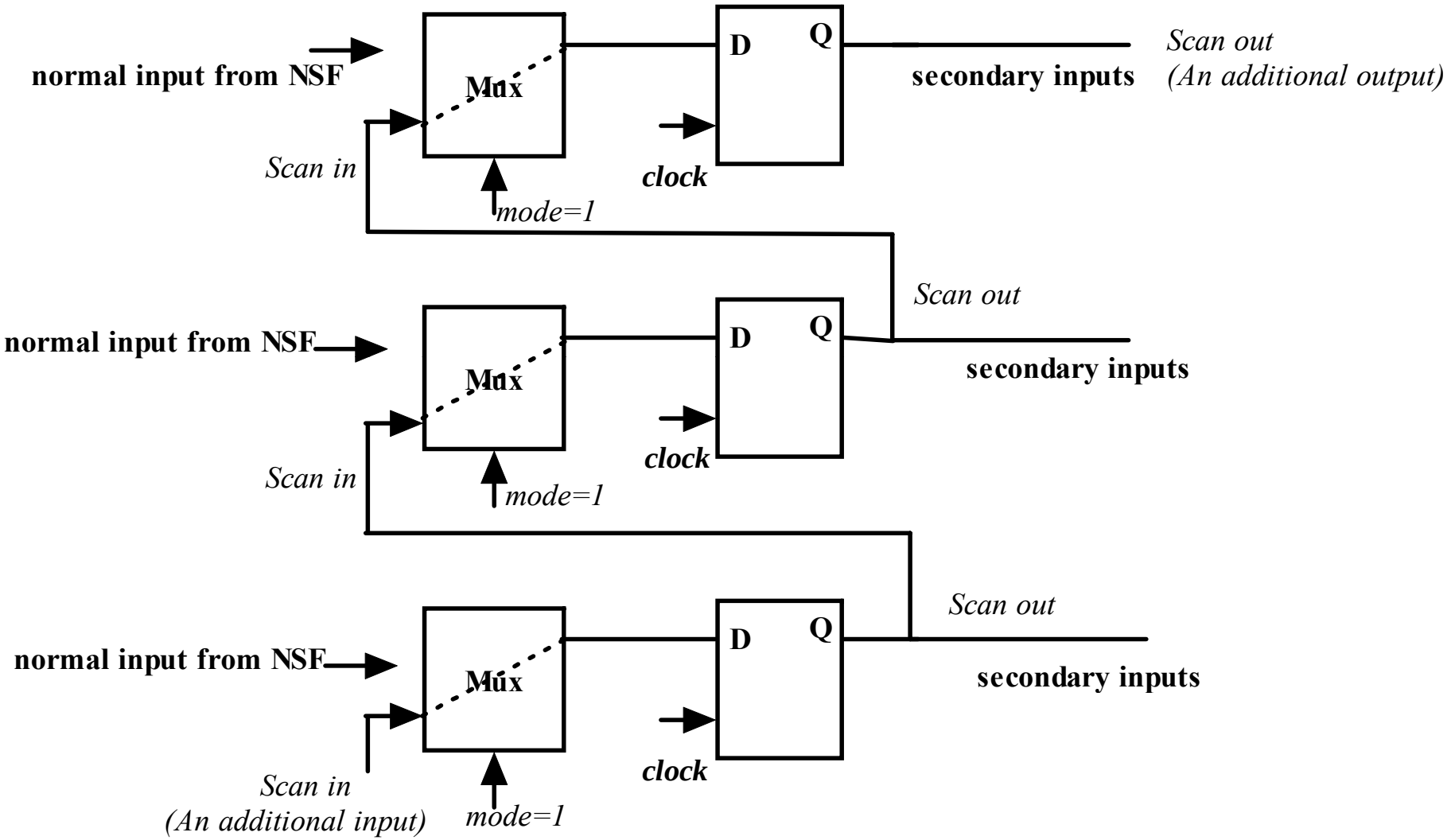
Is it always possible ?



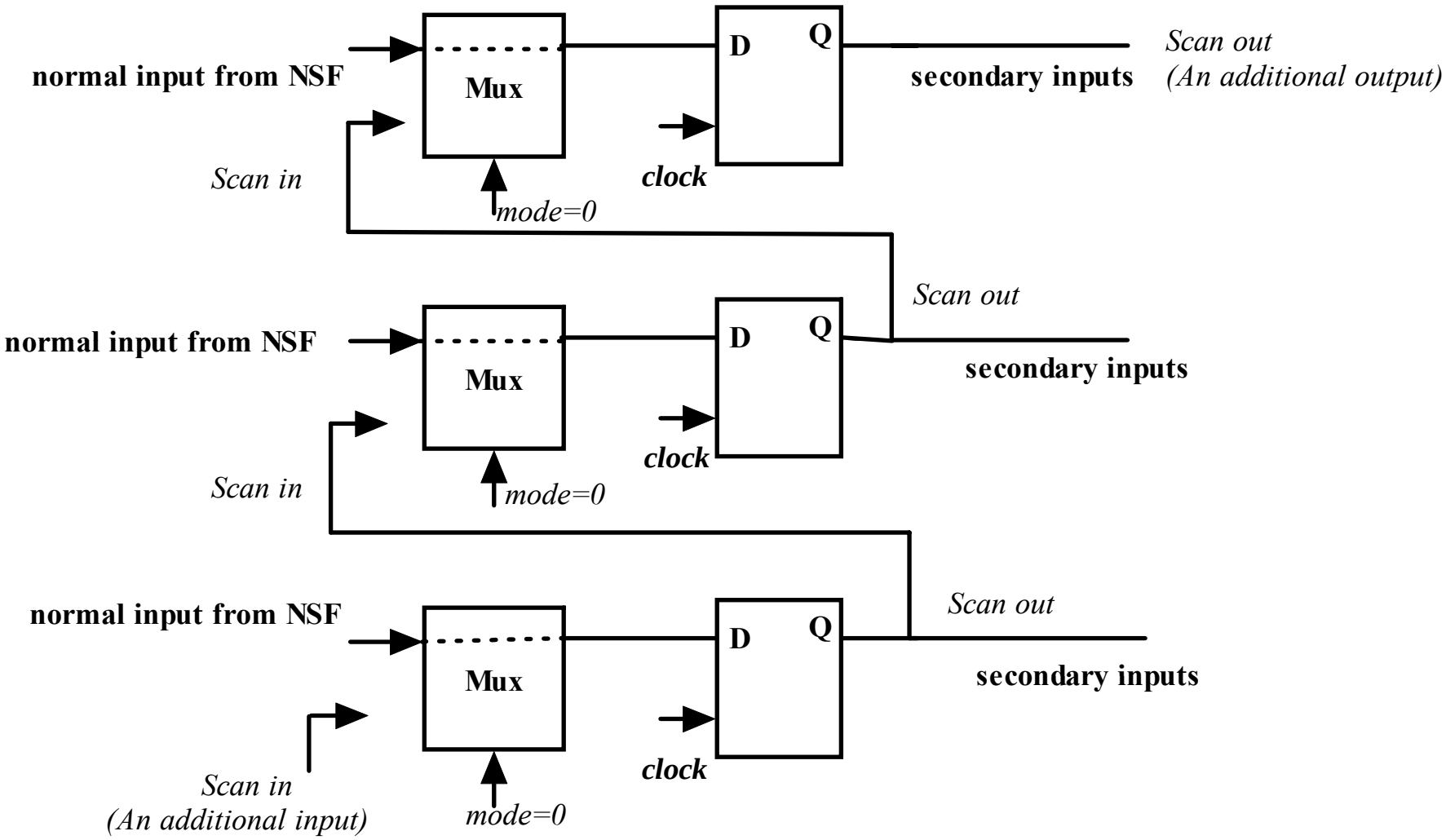
Scan Chains ?


normal input from NSF
Mux
D
Q
secondary inputs
Scan out
(An additional output)
Scan in
mode
clock
Scan out
normal input from NSF
Mux
D
Q
secondary inputs
Scan in
mode
clock
Scan out
normal input from NSF
Mux
D
Q
secondary inputs
Scan in
(An additional input)
mode
clock

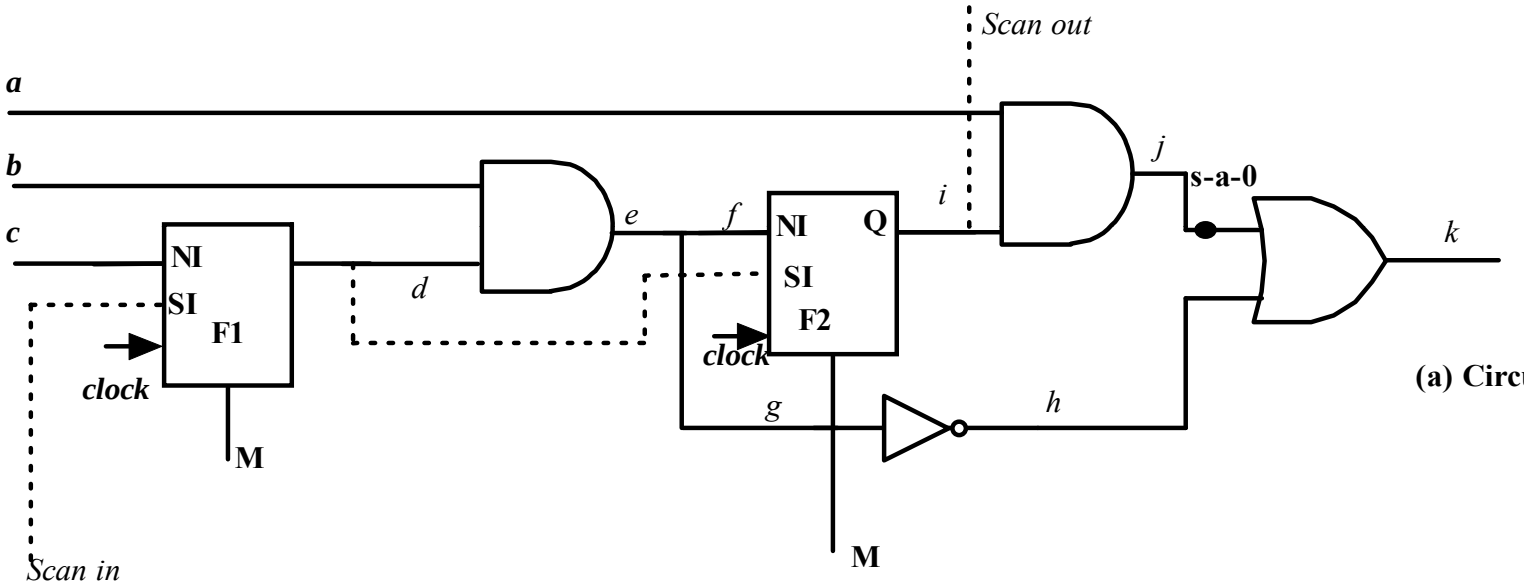
Scan Chains ?


normal input from NSF
Mux
D
Q
clock
Scan in
mode=1
Scan out
(An additional output)
secondary inputs
normal input from NSF
Mux
D
Q
clock
Scan in
mode=1
Scan out
secondary inputs
normal input from NSF
Mux
D
Q
clock
Scan in
(An additional input)
mode=1
Scan out
secondary inputs

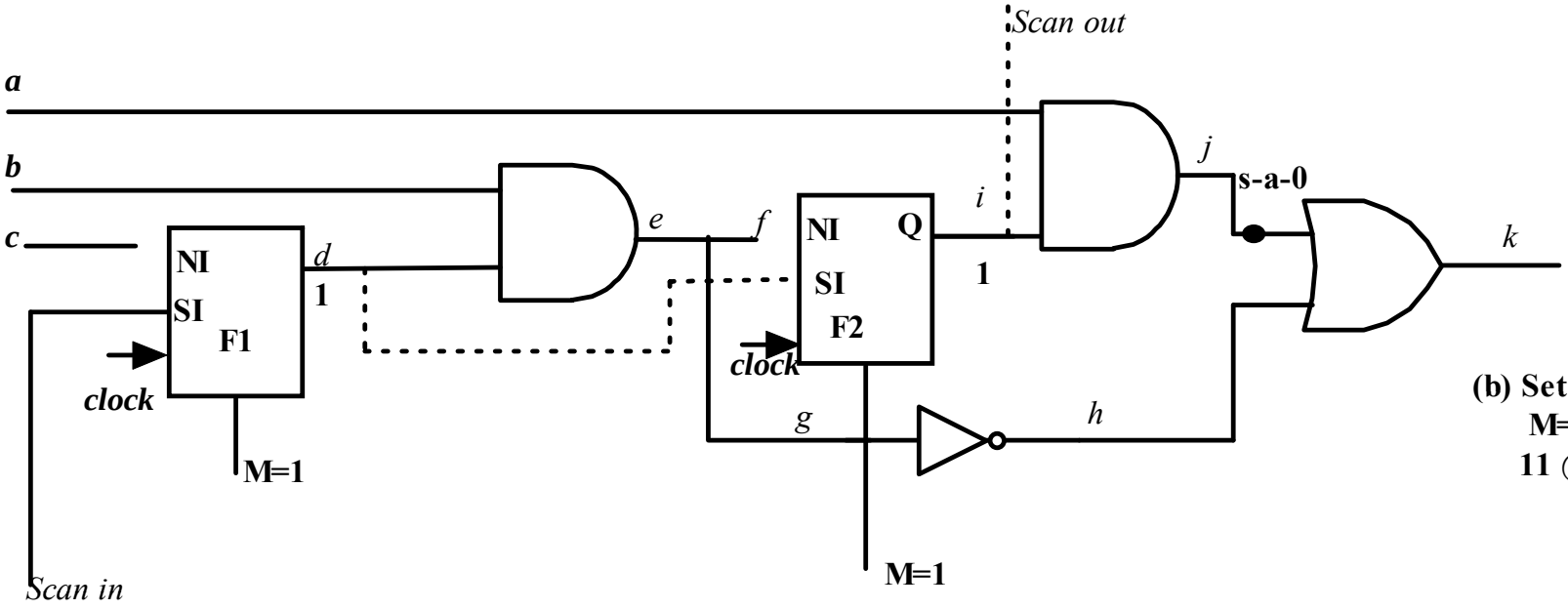
Scan Chains ?


normal input from NSF
Mux
D
Q
secondary inputs
Scan out
(An additional output)
Scan in
mode=0
clock
Scan out
normal input from NSF
Mux
D
Q
secondary inputs
clock
Scan in
mode=0
Scan out
normal input from NSF
Mux
D
Q
secondary inputs
clock
Scan in
(An additional input)
mode=0

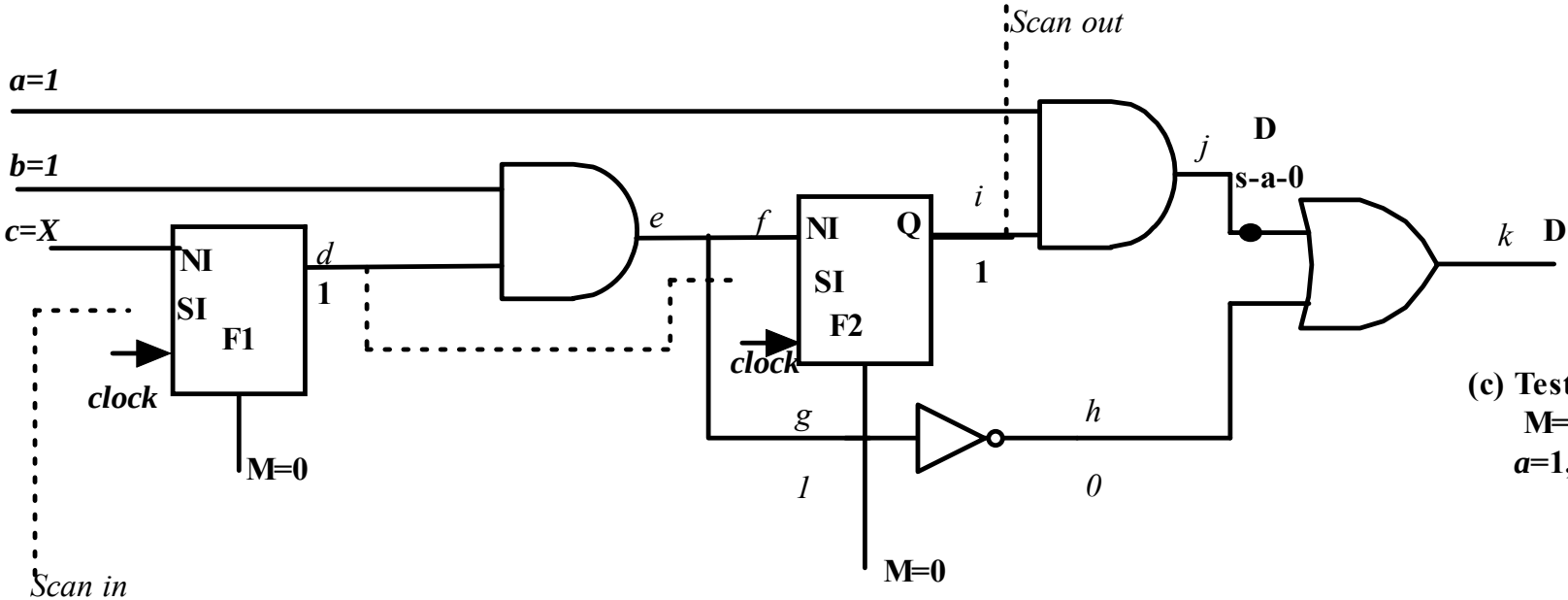
Testing Using Scan Chains - Example



(a) Circuit with Scan Chain



(b) Setting Scan Chain
M=1
11 @ two clock pulses



(c) Testing the fault
M=0
***a*=1,*b*=1,*c*=X**

Why Delay Test and New Fault Model ?

- **Limitations of s-a fault model:**
  - Classical fault model and generally do not verify timing correctness.
  - For slower circuits, it was found that s-a fault model covers some delay faults also. This does not hold for high speed circuits

- **Circuits for RT ES:**
  - Need to meet deadlines i.e., correct output and stabilize before clock edge
  - Requires new fault models i.e., path delay fault model.

- **Complicity of Delay test vs. static s-a test:**
  - Single vector for s-a fault while two vectors for delay faults
  - More number of un-testable cases
  - Complex APTG algorithms

What is Delay Fault Model

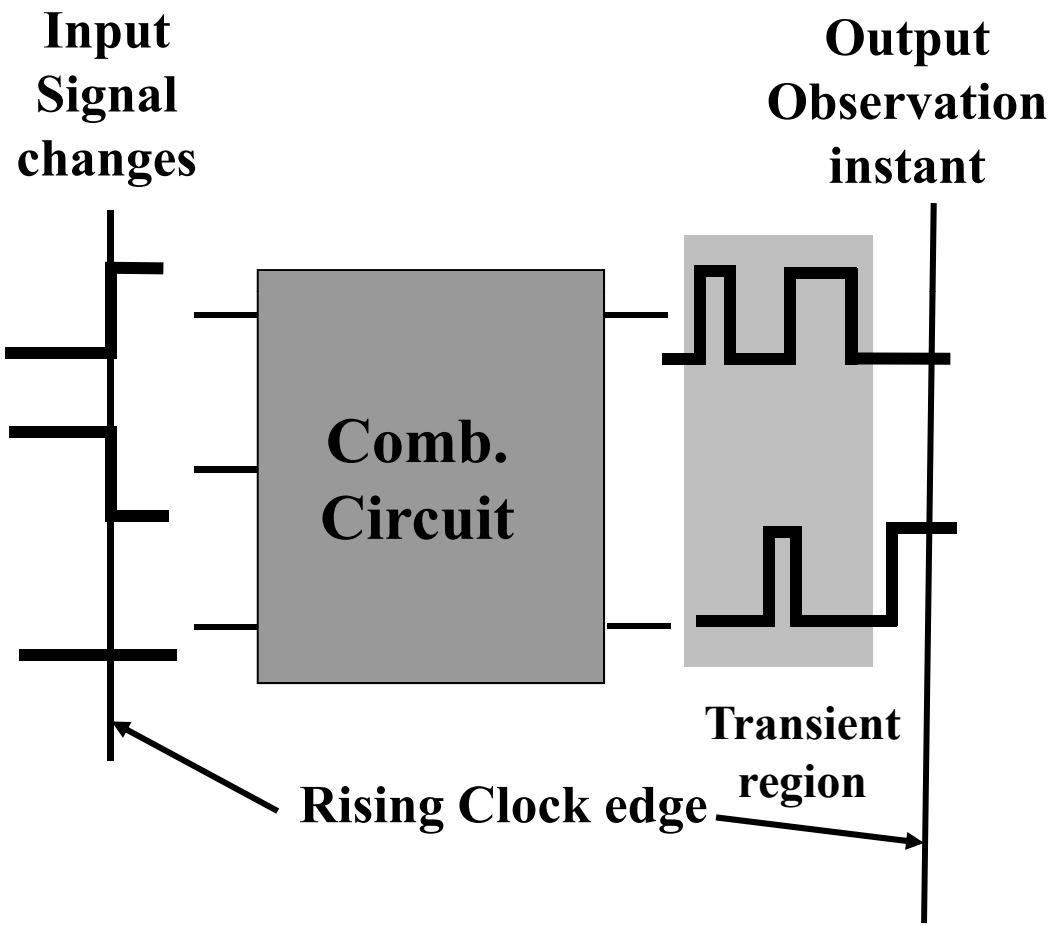
- Combinational path
  - Start: Primary input or a clocked flip-flop through levels of gates
  - Ends: primary output or a clocked flip-flop.

- Propagation delay = Gate transition delays + transport delays of interconnects on a path.

- Clock period > the propagation delay of all paths

- Delay fault
  - Propagation delay of a combinational path > Clock Period

What is Delay Fault Model

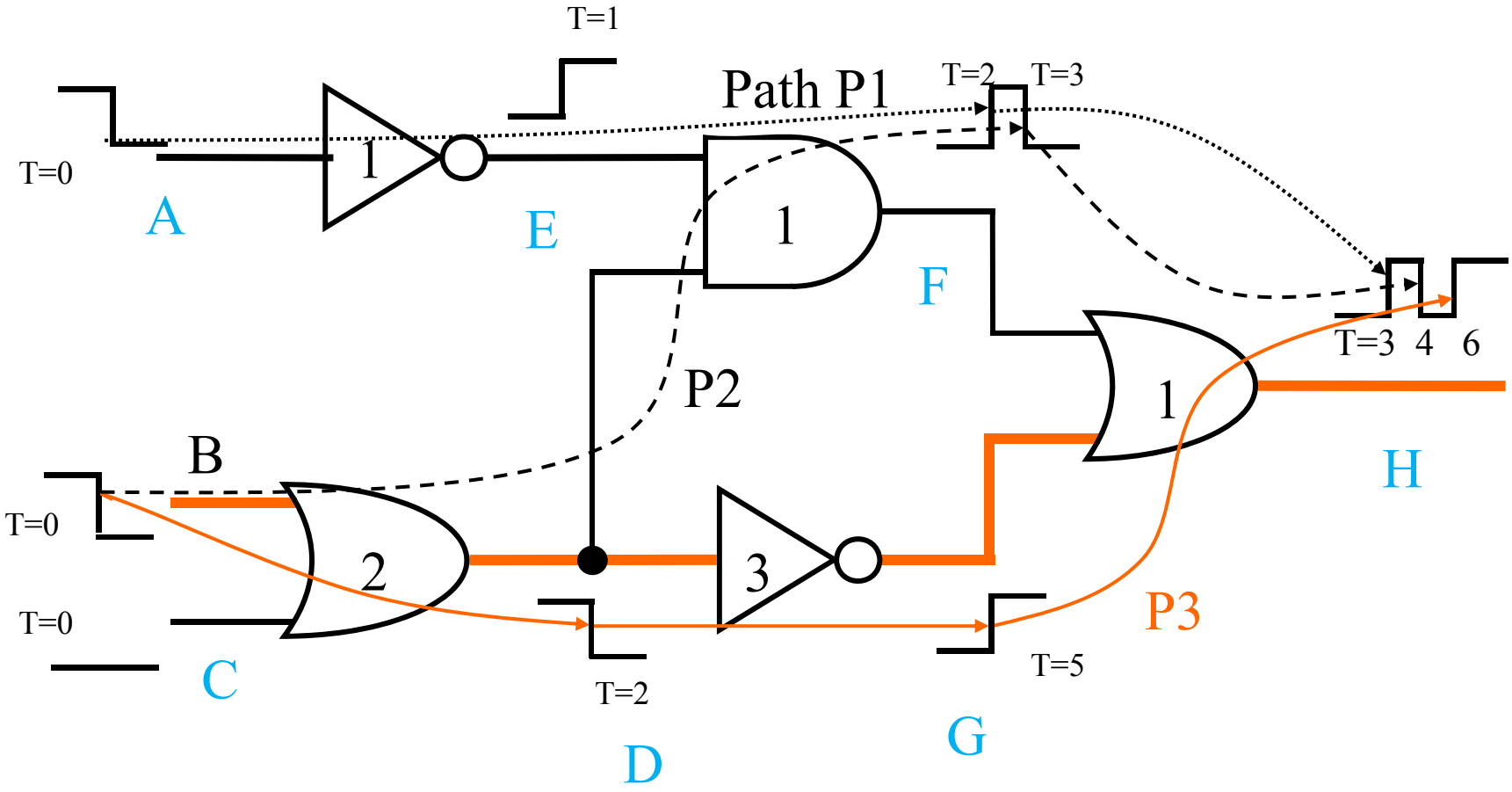


Propagation delay

- Switching or inertial delay is the interval between input change and output change of a gate:
  - Depends on input capacitance, device (transistor) characteristics and output capacitance of gate.
  - Also depends on input rise or fall times and states of other inputs (second-order effects).
  - Approximation: fixed rise and fall delays (or min-max delay range, or single fixed delay) for gate output.
- Propagation or interconnect delay is the time a transition takes to travel between gates:
  - Depends on transmission line effects (distributed *R*, *L*, *C* parameters, length and loading) of routing paths.
  - Approximation: modeled as lumped delays for gate inputs.

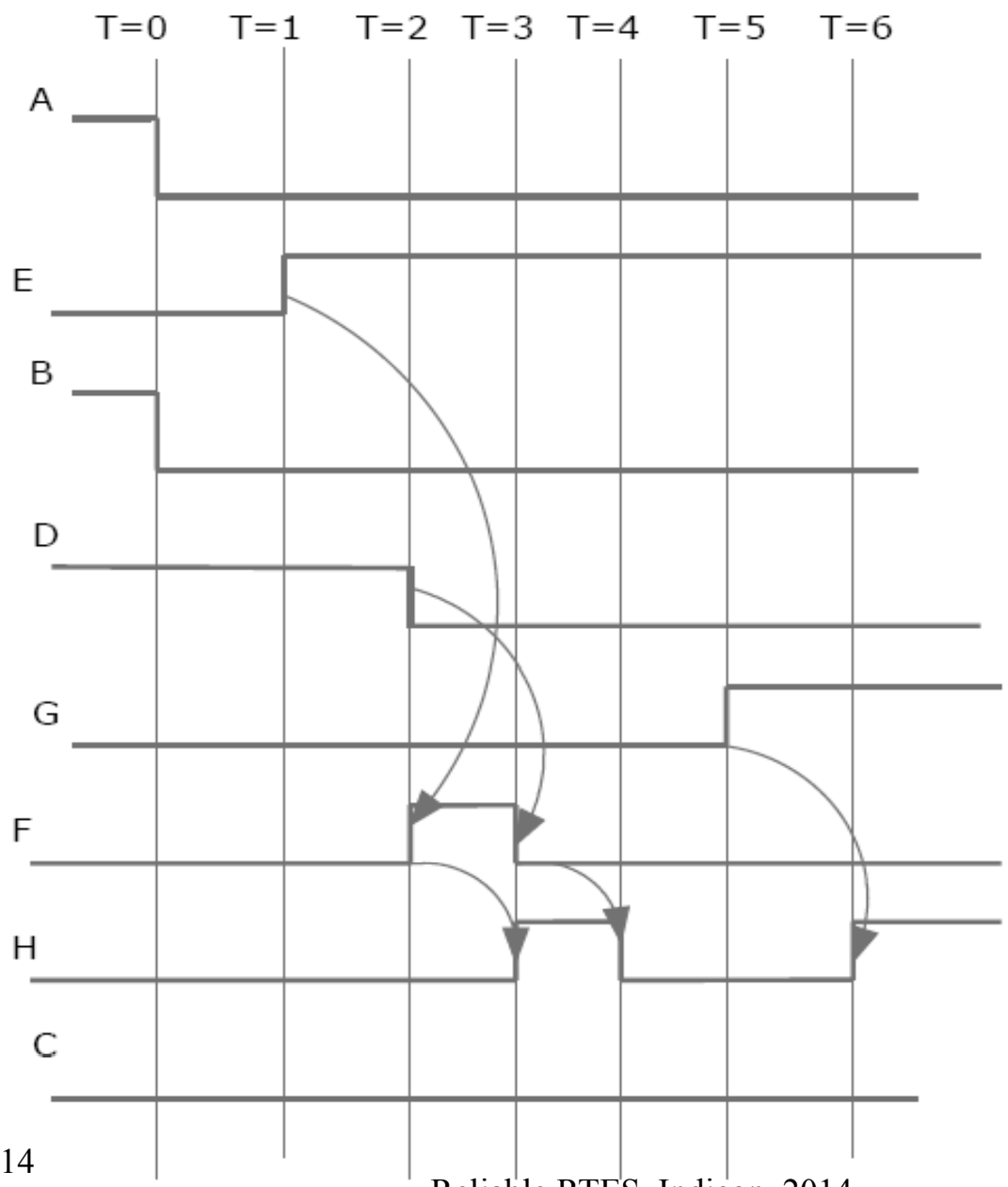
Example

PI transitions assumed to occur without time skew



Time Period is 7

Timing Diagram



Test Patten pair Conditions

On-path input --source of the output transition.

Off-path input – source that does not have effect on output transition.

- Cond-1: If event is a transition from the controlling value to non-controlling value

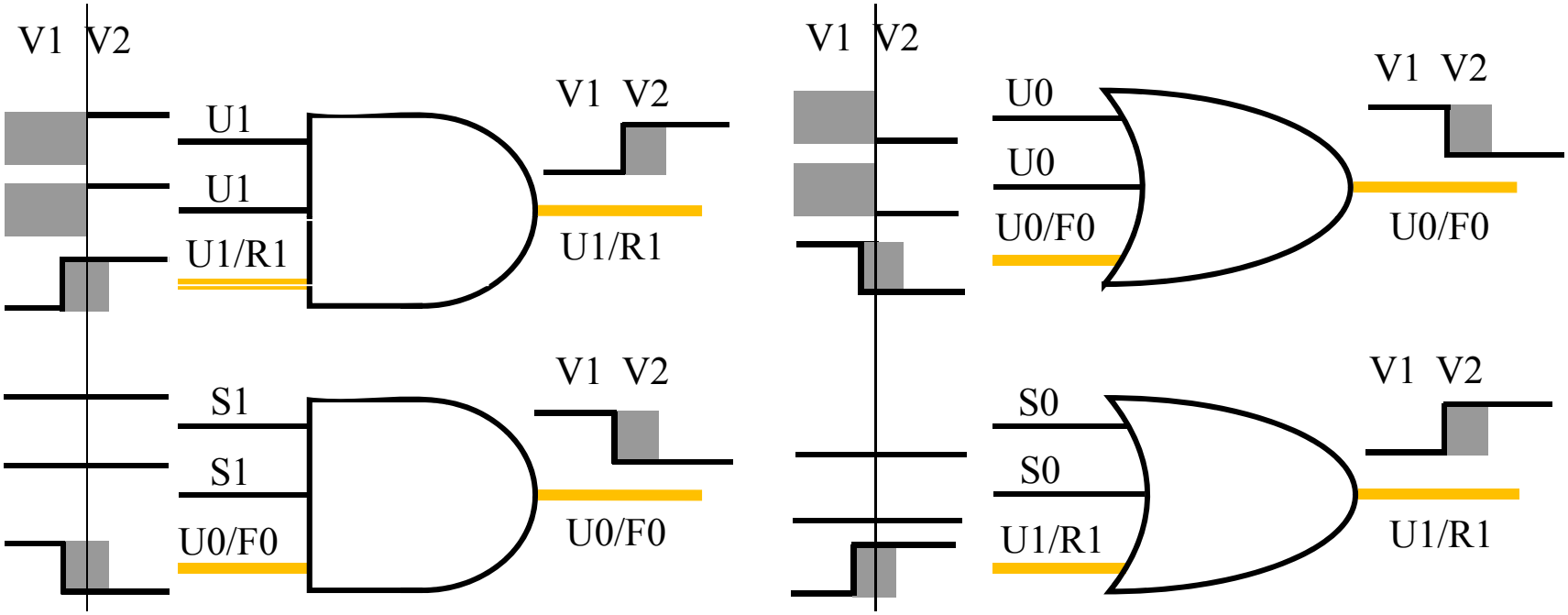
  There is no specific requirement for off-path inputs in V1.

  All off-path inputs of the gate should have non-controlling value in V2.

- Cond-2: If event is a transition from the non-controlling value to Controlling value

  All off-path inputs must have a steady non-controlling value in both V1 and V2.

Test Patten pair Conditions: Examples of Simple Gates



A Five-Valued Algebra

- Signal States: S0, S1, U0, U1, F0, R1, XX.
- On-path signals: F0 and R1.
- Off-path signals: F0=U0 and R1=U1.

AND

Input 2 \ Input 1	S0	U0	S1	U1	XX
S0	S0	S0	S0	S0	S0
U0	S0	U0	U0	U0	U0
S1	S0	U0	S1	U1	XX
U1	S0	U0	U1	U1	XX
XX	S0	U0	XX	XX	XX

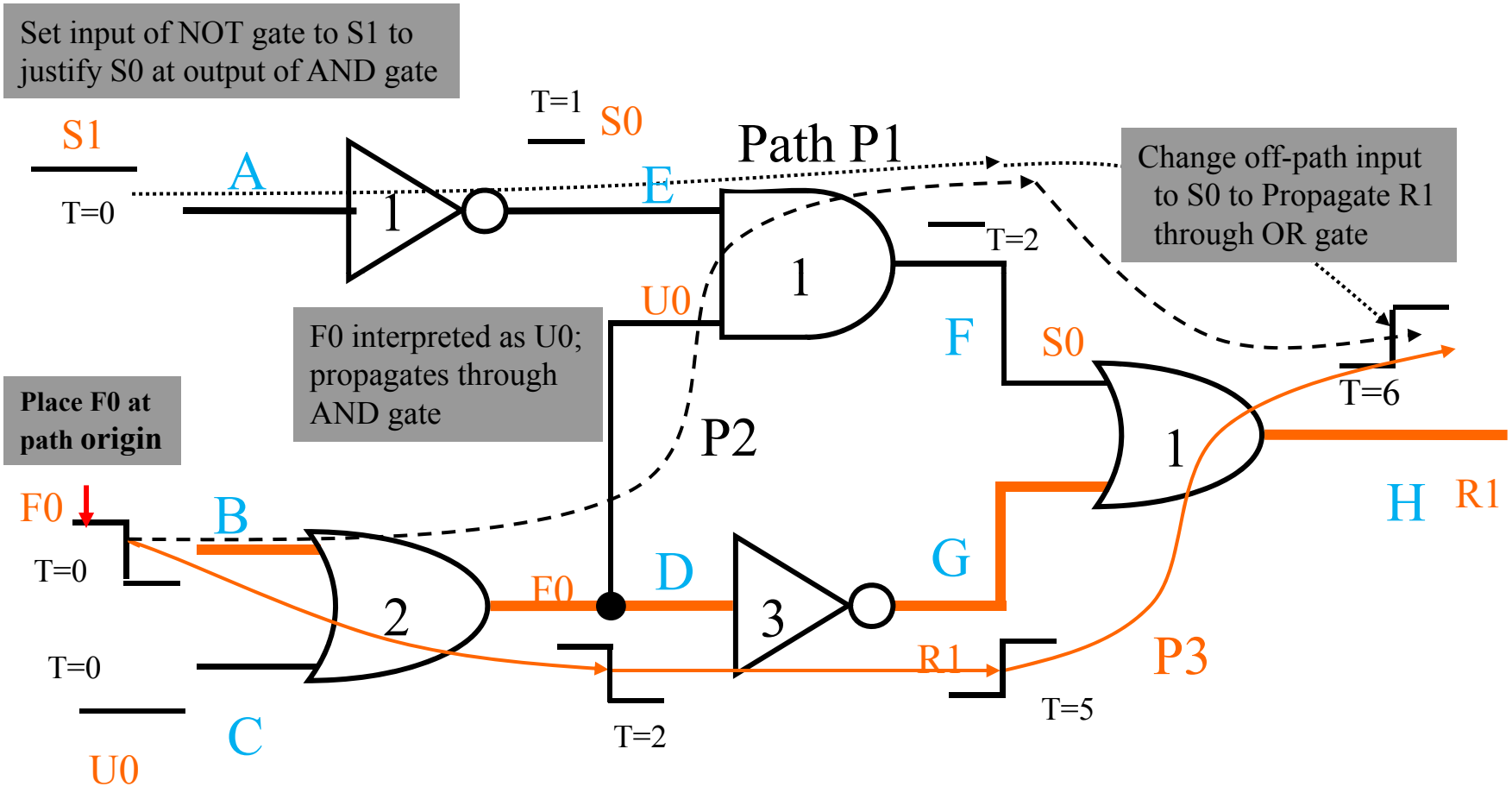
OR

Input 2 \ Input 1	S0	U0	S1	U1	XX
S0	S0	U0	S1	U1	XX
U0	U0	U0	S1	U1	XX
S1	S1	S1	S1	S1	S1
U1	U1	U1	S1	U1	U1
XX	XX	XX	S1	U1	XX

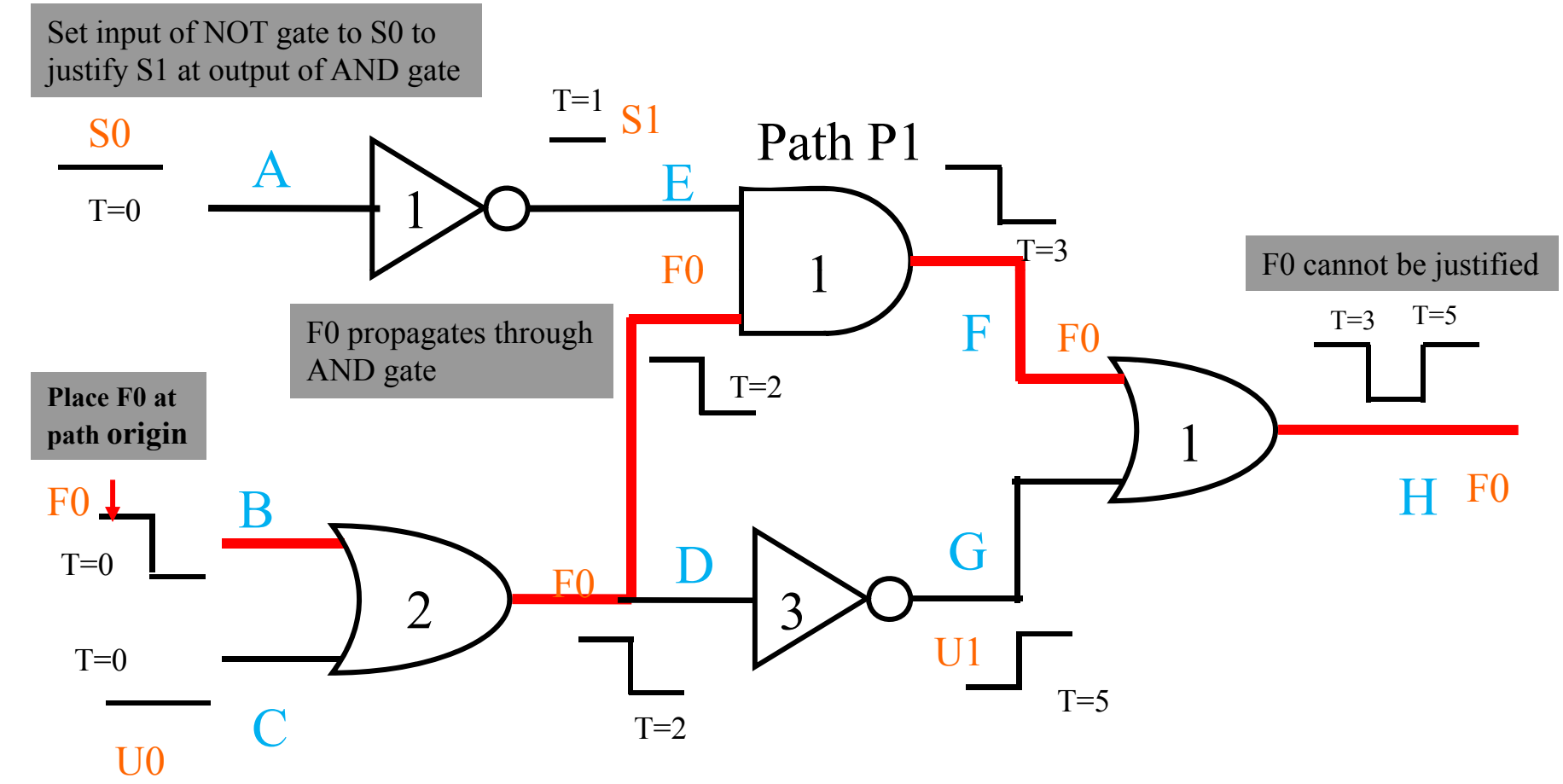
NOT

Input	S0	U0	S1	U1	XX
	S1	U1	S0	U0	XX

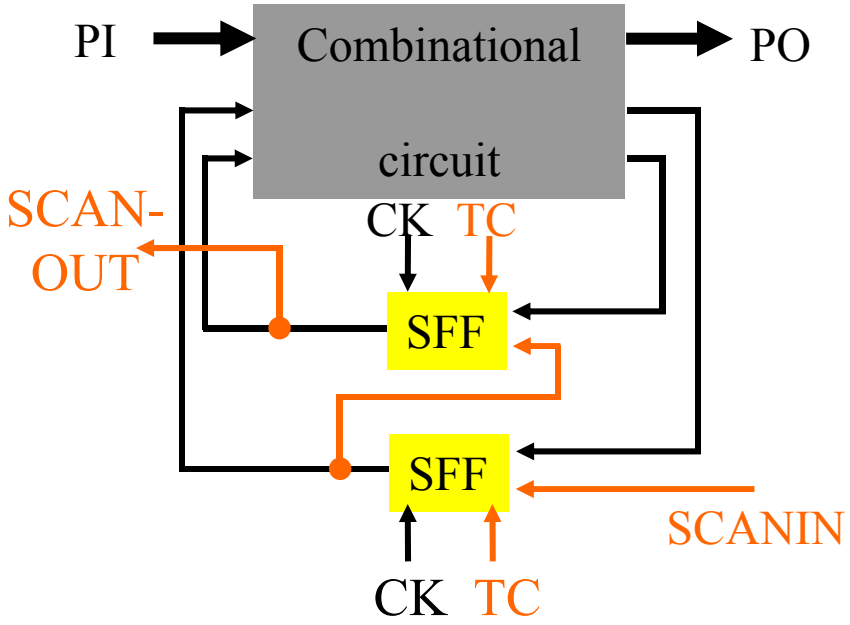
Example: TP generation
P3—Delay to Fall

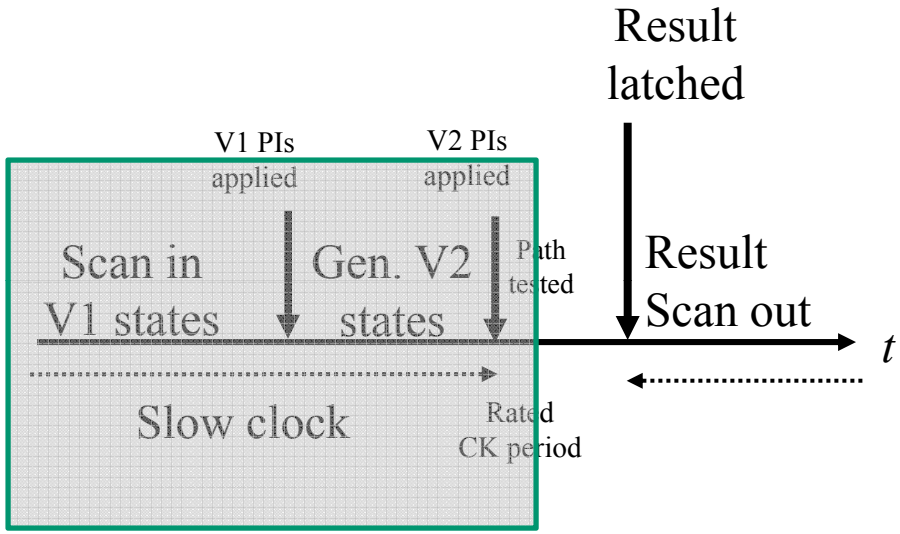


Example: TP generation
P2—Delay to Fall



What About Sequential Circuits ?





Major Challenge
V1 and V2 Related

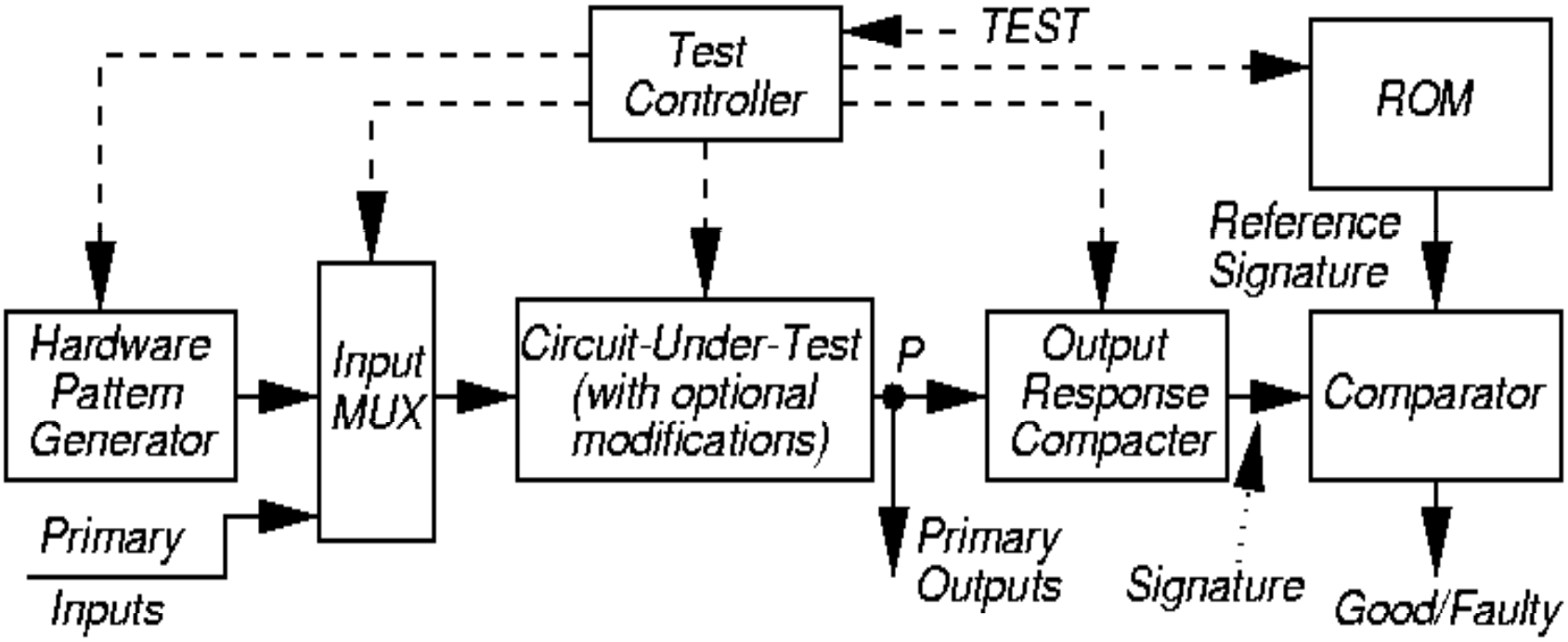
CK: system clock
TC: test control
SFF: scan flip-flop

At-Speed Functional Test

- Application of test vectors at the rated-clock speed.
- Two Schemes.
- ATE based test:
    - TPs applied and analyzed thorough high speed ATEs
    - High-speed testers are expensive.

- BIST

BIST


Test Controller
TEST
ROM
Reference Signature
Hardware Pattern Generator
Input MUX
Circuit-Under-Test (with optional modifications)
P
Output Response Compacter
Comparator
Primary Inputs
Primary Outputs
Signature
Good/Faulty

Key References

- Bushnell and Agrawal, Essentials of Electronic Testing for Digital, Memory & Mixed-Signal Circuits, Kluwer Academic Publishers, 2000

- M. Abramovici,M.A. Breuer, and A.D. Friedman. Digital Systems Testing and Testable Design. Wiley-IEEE Press, 1994.

- Santosh Biswas and Jatindra K Deka, NPTEL Video lectures and Web course
  VLSI Design Verification and Test
  (http://www.nptel.ac.in/courses/106103116/)


KGP_OLM
KGP_BIST
Thank You
KGP_DOTS1
KGP_DOTS4

Thank You